

joker-lap Codebase Reference

- [joker-lap — Codebase Reference](#)
 - [Table of Contents](#)
 - [Section 1 — Project Architecture Overview](#)
 - [Section 2 — Database Schema Reference](#)
 - [Section 3 — Pipeline Scripts Reference](#)
 - [Section 4 — Stat Calculation Reference \(CRITICAL\)](#)
 - [Section 5 — API Endpoints Reference](#)
 - [Section 6 — Frontend Reference](#)
 - [Section 7 — Known Issues & Fix List](#)
 - [Section 8 — How To Make Common Edits](#)

joker-lap — Codebase Reference

Complete developer reference for the MLB/F1/NASCAR analytics platform. Generated by a full read-through of the codebase — every pipeline script, API endpoint, database table, and frontend section documented, with a line-by-line audit of every stat calculation against the correct MLB formula.

Table of Contents

1. Project Architecture Overview
 2. Database Schema Reference
 3. Pipeline Scripts Reference
 4. Stat Calculation Reference (CRITICAL)
 5. API Endpoints Reference
 6. Frontend Reference
 7. Known Issues & Fix List
 8. How To Make Common Edits
-

Section 1 — Project Architecture Overview

1.1 What this is

`joker-lap` (formerly “sports-platform”) is a multi-sport data pipeline + analytics API. Three sports — MLB, F1, NASCAR — each live in their own fully isolated Postgres schema with no cross-sport dependencies. MLB is by far the most developed: it has proprietary quality models (BAPV+, Goose+/Goose+2/Goose+3, pK+/pHR+, Stuff Score, Juiced+2), a Monte Carlo simulation engine, daily K/HR/TB projections, Kalshi market-odds comparison, and an automated morning-brief email. F1 and NASCAR are simpler fetch→transform pipelines without the modeling layer.

1.2 Directory tree

```
joker-lap/
├── docker-compose.yml          # Local dev stack: db, api, pgadmin, scheduler, live_poller
├── docker-compose.aws.yml      # Production stack (EC2): api, scheduler, live_poller – no db/pgadmin
                                # (prod Postgres runs outside compose), TZ=America/Los_Angeles
├── .env / .env.example        # DB creds, DATABASE_URL, pgAdmin creds (EMAIL_* vars used in code
                                # but not listed in .env.example – see 1.4)
├── README.md                  # Original setup doc – DB architecture, day-1 pull instructions
├── MODEL_ROADMAP.md           # Planning doc for model development
├── allfiles.txt                # Stray/scratch file (repo-root, not part of the app)
├── icons/bat.jpg              # Custom bat icon (replaced cricket-bat emoji per recent commit)
├── api/                       # FastAPI app – the "sports_api" Docker service
│   ├── Dockerfile             # Builds the shared image used by api, scheduler, live_poller
│   ├── requirements.txt        # Python deps (FastAPI, SQLAlchemy, httpx, apscheduler, etc.)
│   ├── main.py                # App entry: creates schemas/tables on startup, mounts routers,
│   │                           # mounts /static and /frontend, redirects "/" → /mlb-dashboard
│   ├── database.py            # SQLAlchemy engine + SessionLocal + get_db() FastAPI dependency
│   ├── models/
│   │   ├── __init__.py        # Re-exports all ORM models
│   │   ├── base.py            # Shared declarative Base
│   │   ├── mlb.py              # Every mlb.* table (this is the schema source of truth – see Section 2)
│   │   ├── f1.py              # f1.* tables
│   │   └── nascar.py          # nascar.* tables
│   ├── routers/
│   │   ├── __init__.py
│   │   ├── health.py          # /health liveness check
│   │   ├── mlb.py             # All /mlb/* endpoints – 5,800+ lines, the whole MLB API surface (Section 5)
│   │   ├── nascar.py          # /nascar/* endpoints
│   │   └── f1.py              # /f1/* endpoints
│   └── static/                # Generated/served assets: homepage.html (morning brief), mlb.html
                                # (a served copy of frontend/mlb.html), bat.jpg
├── frontend/                  # Static HTML dashboards served by FastAPI's /frontend mount
│   ├── mlb.html               # The MLB dashboard SPA – vanilla JS, ~6,300 lines (Section 6)
│   ├── f1.html                # F1 dashboard
│   ├── darlington.html        # NASCAR/Darlington-specific page
│   └── logo.png
├── pipeline/                  # Everything that runs outside the request/response cycle
│   ├── daily_pipeline.py       # Master scheduler (APScheduler) + `--now <job>` CLI – orchestrates
│   │                           # every MLB pipeline script on a cron schedule (Section 3)
│   ├── send_email.py          # Gmail SMTP helper – integrity alerts, daily brief, morning email
│   ├── mlb/                   # All MLB pipeline scripts – fetch, transform, models, projections
│   │   ├── (full script-by-script reference in Section 3)
│   │   ├── models/            # Pickled per-pitch-type Stuff Score regression models (stuff_FF.pkl, etc.)
│   │   ├── *.py               # 35+ scripts: fetch*.py, build*.py, simulate*.py, transform.py, etc.
│   ├── f1/                    # F1 fetch/transform/projections (Ergast + OpenF1 APIs)
│   ├── nascar/                # NASCAR fetch/transform/live_poller (Racing Reference scrape + cache)
│   │   └── .cache/             # Parquet cache of scraped NASCAR data, keyed by season/series/race
├── migrations/
│   └── env.py                  # Alembic config, schema-aware (though in practice
                                # `Base.metadata.create_all()` in main.py handles table creation)
```

1.3 Data flow: MLB API → DB → API → frontend

```
statsapi.mlb.com (GUMBO feed)
|
| pipeline/mlb/fetch.py :: fetch_game(game_pk, db)
|
▼
mlb.raw_events (one row per game_pk, full GUMBO JSON in a JSONB `data` column – the "landing zone")
|
| pipeline/mlb/transform.py :: transform_game_pk(game_pk, db)
|
▼
Structured tables: mlb.games, mlb.at_bats, mlb.pitches, mlb.runners, mlb.linescore,
                    mlb.boxscore_batting/pitching/fielding, mlb.fielding_credits
|
| pipeline/mlb/backfill_counts.py (ball/strike count reconstruction, runs at the end of transform)
|
▼
Derived model tables, built by daily_pipeline.py's scheduled jobs on top of the structured tables:
  BAPV+ (compute_bapv.py) → Goose+/Juiced+ (build_goose.py) → Goose+2/Juiced+2 (build_goose2.py)
  → pK+/pHR+ (build_pk_phr.py) → Stuff Score (build_stuff_score.py) → Team Defense (build_team_defense.py)
  → Goose+3 (build_goose3.py) → Daily K/HR/TB projections (build_daily_projections.py, uses simulate.py's
    Monte Carlo engine) → Game-level win/run projections (build_game_projections.py)
|
| ▼ (overnight, once games are Final)
mlb.daily_actuals (build_actuals.py) → mlb.model_scores (build_model_scores.py, grades projections
  against actuals) → mlb.projection_calibration (build_calibration.py, self-adjusts K%/HR% multipliers
  nightly based on recent scoring error)
|
| ▼
mlb.pipeline_integrity – build_integrity_check.py cross-checks the whole chain daily: is mlb.games
  missing any Final game the MLB schedule API knows about, and do our PA counts match MLB's boxscore API
|
| ▼
api/routers/mlb.py – FastAPI endpoints query these tables directly with raw SQL / SQLAlchemy (Section 5)
|
| ▼
frontend/mlb.html – vanilla-JS dashboard fetches those endpoints and renders Pitcher Profile, Hitter
  Profile, Game Companion, Leaderboard, Today's Games, Metrics, and Parlay Builder sections (Section 6)
|
| └ pipeline/mlb/generate_homepage.py also renders a static homepage.html "morning brief" (server-
  rendered, not JS-driven) that's both served at /briefing and emailed out via send_email.py
```

Live, in-progress games take a side path: `pipeline/mlb/live_poller.py` polls the MLB API continuously (independent Docker service, not on the daily_pipeline cron) and writes live state directly, so the Game Companion dashboard section can show real-time in-progress games without waiting for the “Final” status transform pipeline.

1.4 Docker services

Defined in `docker-compose.yml` (local dev) / `docker-compose.aws.yml` (production, deployed on EC2). Both build the same image from `api/Dockerfile` and differ mainly in which services run and what env vars/volumes are mounted.

Service	Container name	Command	Purpose
db	sports_db	postgres:16 image	Local-only Postgres instance. Not present in the AWS compose file – production points <code>DATABASE_URL</code> at an external Postgres instance instead.
api	sports_api	uvicorn main:app --host 0.0.0.0 --port 8000 --reload	The FastAPI app – serves all <code>/mlb</code> , <code>/f1</code> , <code>/nascar</code> , <code>/health</code> endpoints plus the static frontend and morning-brief HTML. Port 8000 (proxied by Nginx on port 80 in prod, per recent commit).
pgadmin	sports_pgadmin	dpape/pgadmin4 image	Visual DB browser at :5050. Local dev only , not in the AWS compose file.
scheduler	sports_scheduler	python3 /pipeline/daily_pipeline.py	Runs <code>daily_pipeline.py</code> in scheduler mode (APScheduler <code>BlockingScheduler</code>) – this is the cron brain that fires every pipeline job (fetch, transform, all the <code>build_*</code> model scripts, integrity check, homepage generation, emails) on a fixed daily schedule. See Section 3 for the full timetable.
live_poller	sports_live_poller	python3 /pipeline/mlb/live_poller.py	Continuously polls MLB's live game feed independent of the daily cron, so in-progress games show live state on the dashboard. Talks to the <code>api</code> service

Service	Container name	Command	Purpose
			over API_BASE_URL=http://api:8000.

All services except `db / pgadmin` mount `./api:/app` and `./pipeline:/pipeline` as live bind-mounts, so local edits take effect without rebuilding — but changes must be `docker cp`’d into the containers manually if the containers were built from an image snapshot rather than run with bind mounts (this repo’s workflow, per recent session history, copies changed files into the running containers with `docker cp` rather than relying solely on the mount).

1.5 Environment variables

Actually read via `os.environ.get()` / `os.getenv()` in the codebase (grepped across `api/` and `pipeline/`):

Variable	Used by	Controls
DATABASE_URL	api/database.py (all services)	Postgres connection string — SQLAlchemy engine target
EMAIL_FROM	pipeline/send_email.py	From-address for outgoing SMTP mail
EMAIL_TO	pipeline/send_email.py	Recipient address for integrity/brief/alert emails
EMAIL_SMTP_HOST	pipeline/send_email.py	SMTP host, defaults to <code>smtp.gmail.com</code> if unset
EMAIL_SMTP_PORT	pipeline/send_email.py	SMTP port, defaults to <code>587</code> if unset
EMAIL_SMTP_USER	pipeline/send_email.py	SMTP auth username (Gmail address)
EMAIL_SMTP_PASSWORD	pipeline/send_email.py	SMTP auth password (Gmail app password)
API_BASE_URL	pipeline/mlb/live_poller.py	Where the live poller reaches the FastAPI service (<code>http://api:8000</code> in Docker)

Also set in `docker-compose*.yaml` but consumed by the runtime/interpreter rather than read via `os.getenv()` in application code: - `PYTHONPATH=/app:/pipeline` — lets pipeline scripts `import` both the `api/` package (for `database.py`, `models/`) and sibling pipeline modules - `TZ` — `America/New_York` in local dev compose, `America/Los_Angeles` in the AWS compose file (the scheduler’s actual cron logic uses explicit `ZoneInfo("America/Los_Angeles")` / `ZoneInfo("America/New_York")` objects in `daily_pipeline.py` regardless of container `TZ`, so this mostly affects log timestamps and any naive-datetime code)

`.env.example` only documents the Postgres/pgAdmin variables (`POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB`, `DATABASE_URL`, `PGADMIN_DEFAULT_EMAIL`, `PGADMIN_DEFAULT_PASSWORD`) — it’s stale and missing all the `EMAIL_*` variables required for alerting to work. No API keys are required for either the MLB Stats API or the public Kalshi markets endpoint (`api.elections.kalshi.com`) — both are unauthenticated.

Section 2 — Database Schema Reference

All tables live in the Postgres `mlb` schema. Two different mechanisms create tables in this codebase:

- **ORM-defined** tables (`api/models/mlb.py`) — SQLAlchemy models, migrated implicitly via `Base.metadata.create_all` at API startup. These are the raw/game-level tables.
- **Raw-SQL / self-provisioning** tables — model/projection/reference tables that have no SQLAlchemy model at all. Each pipeline script embeds its own `CREATE TABLE IF NOT EXISTS` (or, for the score tables like `goose3_scores`, `pk_scores`, `phr_scores`, `stuff_scores`, `goose2_scores`, `juiced2_scores`, assumes the table already exists — no `CREATE TABLE` was found in the pipeline scripts for these six; they were provisioned once outside the scripts, likely by hand or an untracked migration) and writes to it with `INSERT ... ON CONFLICT ... DO UPDATE (upsert)`.

Raw data

`mlb.raw_events`

- **Purpose:** Landing zone for the complete raw GUMBO JSON blob per game. Everything downstream is derived from this.
- **Key columns:** `game_pk` (unique), `game_date`, `status`, `away_team / home_team` (abbrev), `data` (JSONB — the full GUMBO feed/live response), `fetch_at`, `updated_at`.
- **Populated by:** `pipeline/mlb/fetch.py::fetch_game()` — inserts on first fetch, updates the `data` blob when re-fetched. `build_integrity_check.py` deletes+re-inserts rows here when it detects incomplete data.

- **Read by:** `pipeline/mlb/transform.py` (the sole consumer — every downstream table below is derived from this one). Referenced twice in `api/routers/mlb.py`, mostly for existence/debug checks, not primary data serving.

`mlb.at_bats`

- **Purpose:** One row per plate appearance/at-bat — the core analytical unit. Transformed from the `allPlays` array in `raw_events`.
- **Key columns:** `game_pk`, `at_bat_index`, `inning`, `half_inning`, `batter_id` / `batter_name`, `pitcher_id` / `pitcher_name`, `bat_side`, `pitch_hand`, `event` / `event_type` (e.g. “Single”, “Strikeout”), `rbi`, `has_out`, `balls` / `strikes` / `outs` (count at end of AB).
- **Populated by:** `pipeline/mlb/transform.py::transform_at_bats()`.
- **Read by:** Heaviest-used raw table in the API (63 references in `api/routers/mlb.py`) — backs hitter/pitcher game logs, splits, matchup history, and most “actuals” aggregation in `build_actuals.py`, `build_model_scores.py`, `build_team_defense.py`.

`mlb.pitches`

- **Purpose:** One row per pitch — the most granular table, holds all Statcast physics (velocity, movement, spin, location, exit velo/launch angle on contact).
- **Key columns:** `game_pk`, `at_bat_index`, `pitch_index`, `pitcher_id`, `batter_id`, `pitch_type_code`, `call_code` / `is_strike` / `is_ball` / `is_in_play`, `start_speed`, `px` / `pz` (plate location), `px_x` / `px_z` (movement), `spin_rate`, `launch_speed` / `launch_angle` / `hit_distance`, `balls_before` / `strikes_before` (reconstructed — see `backfill_counts.py`).
- **Populated by:** `pipeline/mlb/transform.py::transform_pitches()`; `balls_before` / `strikes_before` are backfilled separately by `pipeline/mlb/backfill_counts.py::backfill_game()` (called automatically at the end of `transform_game_pk()`).
- **Read by:** Second most-referenced table in the API (29 refs) — all pitch-level physics/Statcast views, BAPV+, Stuff Score, count-mix models.

`mlb.runners`

- **Purpose:** Baserunner movement on every play — one row per runner per play event. Backs RE24/WPA-style situational analysis.
- **Key columns:** `game_pk`, `at_bat_index`, `runner_id`, `start_base` / `end_base` / `out_base`, `is_out`, `event`, `rbi`, `earned`, `responsible_pitcher_id`.
- **Populated by:** `pipeline/mlb/transform.py::transform_runners()`, driven by `pipeline/mlb/reconstruct_runners.py` for backfills/rebuilds (scheduled nightly-ish via `run_reconstruct_runners()` in `daily_pipeline.py`).
- **Read by:** Not directly queried in `api/routers/mlb.py` today (0 refs) — currently an internal/analytical table, not yet exposed via API.

Game/team data

`mlb.games`

- **Purpose:** One row per game — team matchup, final score, venue, weather at first pitch, decisions (W/L/S pitcher), no-hitter/perfect-game flags.
- **Key columns:** `game_pk` (unique), `game_date`, `game_type` (R/S/P), `season`, `status`, `away_team_id` / `abbrev`, `home_team_id` / `abbrev`, `away_score` / `home_score`, `venue_name`, `weather_temp` / `weather_condition` / `weather_wind`, `winning_pitcher_id` / `losing_pitcher_id` / `save_pitcher_id`.
- **Populated by:** `pipeline/mlb/transform.py::transform_game()`; `status` / `score` kept fresh without a full re-fetch via `pipeline/mlb/fetch.py::update_statuses()` (runs every 30 min via `run_update_statuses()`).
- **Read by:** Most-referenced table in the whole API (43 refs) — the join anchor for nearly every endpoint (schedule, box scores, game companion, homepage recap/outlook).

`mlb.linescore`

- **Purpose:** Runs/hits/errors per half-inning per game — inning-by-inning game flow.
- **Key columns:** `game_pk`, `inning`, `half_inning`, `runs`, `hits`, `errors`, `left_on_base`.
- **Populated by:** `pipeline/mlb/transform.py::transform_linescore()`.
- **Read by:** Lightly used directly by the API (1 ref) — mostly consumed at transform time / via `games` aggregates instead.

`mlb.boxscore_batting`

- **Purpose:** Cumulative per-player batting line for a game, as computed by MLB (saves re-aggregating pitch-by-pitch). Includes MLB’s own `avg` / `obp` / `slg` / `ops` for that single game.

- **Key columns:** `game_pk`, `player_id`, `team_id`, `batting_order`, `at_bats`, `hits`, `doubles/triples/home_runs`, `walks`, `strikeouts`, `hit_by_pitch`, `total_bases`, `sac_flies`, `avg/obp/slg/ops`.
- **Populated by:** `pipeline/mlb/transform.py::transform_boxscore_batting()`.
- **Read by:** `boxscore_batting.at_bats > 0` is also how `build_integrity_check.py::validate_pa_counts()` counts “our PA” for the nightly PA-mismatch check. 9 refs in the API (hitter game logs / box score display).

`mlb.boxscore_pitching`

- **Purpose:** Cumulative per-player pitching line for a game (MLB-computed), including game `era/whip`.
- **Key columns:** `game_pk`, `player_id`, `innings_pitched`, `hits`, `earned_runs`, `walks`, `strikeouts`, `home_runs`, `batters_faced`, `outs`, `era`, `whip`, decision flags (`wins/losses/saves/holds/blown_saves`).
- **Populated by:** `pipeline/mlb/transform.py::transform_boxscore_pitching()`.
- **Read by:** Pitcher box score display (2 refs); primary source for `build_actuals.py` pitcher actuals (`actual_k`, `actual_ip`, `actual_er`, etc. all pulled from here).

`mlb.boxscore_fielding` / `mlb.fielding_credits`

- **Purpose:** Cumulative per-player fielding line (`boxscore_fielding`: assists/putouts/errors/fielding_pct) and per-play fielding credit assignment (`fielding_credits`: who recorded the putout/assist/error on each ball in play).
- **Populated by:** `pipeline/mlb/transform.py` (`transform_boxscore_fielding()`, `transform_fielding_credits()`).
- **Read by:** Not currently queried by the API (0 refs each) — transformed and stored but not yet surfaced on the frontend.

Player models

These are the site’s proprietary pitcher/batter quality metrics. All are raw-SQL tables (no ORM model), one row per `pitcher_id/batter_id × season × pitch_type_code (± bat_side)`, upserted nightly/on-demand.

`mlb.stuff_scores` (+ `mlb.stuff_overall`, `mlb.zone_values`)

- **Purpose:** “Stuff Score” — a physics-based pitch-quality grade from velocity, movement (IVB/horizontal break), spin, and release “tunnel” similarity to the pitcher’s other pitches, independent of outcomes.
- **Key columns** (`stuff_scores`): `pitcher_id`, `season`, `pitch_type_code`, `pitch_hand`, `avg_velo`, `avg_ivb`, `avg_hbreak`, `avg_spin`, `physical_score`, `location_score`, `tunnel_score`, `stuff_score` (composite), `weighted_whiff_rate`, `pitches`. `stuff_overall` rolls this up to one score per pitcher (all pitch types blended). `zone_values` holds the run-value-by-zone lookup table used to compute `location_score`.
- **Populated by:** `pipeline/mlb/build_stuff_score.py` (scheduled `run_build_stuff_score()`, 2026 season).
- **Read by:** Pitcher profile “Stuff” tab (1 direct ref to `stuff_scores`, 4 to `stuff_overall`); also read by `build_goose3.py` as an input.

`mlb.pk_scores` / `mlb.phr_scores` (+ `pk_overall`, `phr_overall`)

- **Purpose:** “pK+” (strikeout-rate quality, from whiff/CSW/contact/chase indices vs. league) and “pHR+” (home-run/hard-contact-suppression quality, from SLG-against and hard-hit-rate indices vs. league), both scaled 100 = league average.
- **Key columns:** `pitcher_id`, `season`, `pitch_type_code`, `bat_side`, index columns (`whiff_idx`, `csw_idx`, `contact_idx`, `chase_idx` for pK+; `slg_idx`, `hard_hit_idx` for pHR+), the composite `pk_plus/phr_plus`, `pitches`.
- **Populated by:** `pipeline/mlb/build_pk_phr.py` (single script builds both; scheduled `run_build_pk_phr()`).
- **Read by:** `pk_scores/phr_scores` per-pitch-type tables have 0 direct API refs — the API reads the rolled-up `pk_overall/phr_overall` tables instead (8 and 5 refs respectively). Both also feed into `goose3_scores`.

`mlb.goose2_scores` / `mlb.juiced2_scores` (+ `goose2_overall`, `league_pitch_baselines`)

- **Purpose:** “Goose+2”/“Juiced+2” — outcome-based (not physics-based) pitcher and batter quality vs. league, using contact/whiff/CSW/SLG/hard-hit indices. `juiced2_scores` is the batter-side mirror: batter quality vs. the quality of pitching faced.
- **Key columns** (`goose2_scores`): `pitcher_id`, `season`, `pitch_type_code`, `bat_side`, `contact_idx/whiff_idx/csw_idx/slg_idx/hard_hit_idx/chase_idx`, `goose2_plus`. (`juiced2_scores`): `batter_id`, `season`, `bat_side`, `slg/ops`, `avg_goose2_faced` (quality of pitching faced), `slg_vs_expected/ops_vs_expected`, `juiced2_plus`, `bat_speed/squared_up_pct/blast_rate` (bat-tracking inputs where available).
- **Populated by:** `pipeline/mlb/build_goose2.py` (writes both `goose2_scores` and `juiced2_scores` — the `run_build_juiced2()` scheduler entry actually calls this same script).
- **Read by:** `juiced2_scores` is the most-referenced of this group (6 refs — batter profile “power quality” tiles); `goose2_scores/goose2_overall` back the pitcher “Goose+2” view.

`mlb.goose3_scores (+ goose3_overall)`

- **Purpose:** “Goose+3” — blends the physics-based Stuff Score with the outcomes-based pK+/pHR+ scores into one composite pitcher-quality number, weighted by sample size (more pitches thrown → more weight on outcomes; thin samples lean on Stuff Score).
- **Key columns:** `pitcher_id`, `season`, `pitch_type_code`, `pitch_hand`, `bat_side`, `stuff_score`, `goose2_score`, `pk_score`, `phr_score` (inputs, copied in), `stuff_weight / outcomes_weight` (blend weights), `goose3_plus` (final composite), `stuff_outcomes_gap` (diagnostic — how much stuff and outcomes disagree), `pitches`.
- **Populated by:** `pipeline/mlb/build_goose3.py` (reads `stuff_scores`, `pk_scores`, `phr_scores`, `goose2_scores` as inputs).
- **Read by:** `goose3_overall` is a heavily-used table (12 refs) — the headline pitcher grade shown across leaderboards, pitcher profile, and daily projections’ `goose_plus` field.

`mlb.pitcher_mix_profile / mlb.pitcher_count_mix`

- **Purpose:** Pitch-usage and outcome profiles split by batter handedness (`pitcher_mix_profile`) and by ball-strike count (`pitcher_count_mix`) — used to weight simulation pitch selection realistically.
- **Populated by:** `pipeline/mlb/build_pitcher_mix.py` and `pipeline/mlb/build_count_mix.py` respectively (the latter depends on `backfill_counts.py` having populated `pitches.balls_before / strikes_before` first).
- **Read by:** `pitcher_mix_profile` has 4 API refs (pitcher profile pitch-mix chart); `pitcher_count_mix` has 0 (simulation-internal only).

`mlb.batter_pitch_type_tendencies / mlb.batter_zone_tendencies`

- **Purpose:** Per-batter tendencies by pitch type (swing/whiff/contact/chase/hard-hit rates, min 50 pitches faced) and by strike-zone location (min 20 pitches in zone) — used to personalize pitch outcome simulation per matchup.
- **Populated by:** `pipeline/mlb/build_batter_tendencies.py` (weekly, Monday 4am PT).
- **Read by:** `batter_pitch_type_tendencies` has 6 API refs (hitter profile plate-discipline breakdown); `batter_zone_tendencies` 0 (simulation-internal).

Projections

`mlb.daily_projections`

- **Purpose:** One row per pitcher per snapshot date/game — the site’s core pitcher projection (K%, HR%, hit%, OPS-against, projected strikeouts for full game and first-5-innings), refreshed repeatedly through the day (every 30 min, 7:30am–11pm PT) as inputs (lineups, weather) firm up.
- **Key columns:** `snapshot_date`, `game_pk`, `pitcher_id`, `pitcher_hand`, `team_abbrev / opp_abbrev`, `goose_plus`, `bapv_plus`, `proj_k_pct`, `proj_hr_pct`, `proj_hit_pct`, `proj_ops`, `proj_ks_6inn`, `proj_bb_pct`, `proj_ks_f5 / proj_er_f5 / proj_ip_f5` (first-5-innings variants, for F5 markets), `batters_simulated / simulations_per_batter / batter_ids` (Monte Carlo provenance), and a **locked snapshot:** `proj_ks_locked`, `proj_ks_f5_locked`, `proj_k_pct_locked`, `locked_at` (see Section 4/8 — the value frozen at first-projection time, before late updates, used so Kalshi comparisons aren’t “moving the goalposts”).
- **Populated by:** `pipeline/mlb/build_daily_projections.py`, driven by `pipeline/mlb/simulate.py`’s Monte Carlo engine. Upserts on (`snapshot_date`, `pitcher_id`).
- **Read by:** 8 API refs — backs the pitcher projections table on the homepage and Today’s Games/parlay views.

`mlb.daily_batter_projections`

- **Purpose:** Batter-side mirror of `daily_projections` — one row per batter × opposing pitcher × snapshot date.
- **Key columns:** `snapshot_date`, `game_pk`, `batter_id`, `bat_side`, `opp_pitcher_id / opp_pitcher_hand / opp_goose_plus`, `primary_pitch_type / primary_pitch_velo_quintile / primary_pitch_goose_plus` (what pitch the batter is most likely to see, and its quality), `juiced_plus_vs_bucket / juiced_plus_overall`, `proj_avg / proj_obp / proj_slg / proj_ops`, `proj_hr_pct / proj_k_pct / proj_bb_pct / proj_hit_pct`, `proj_tb`, `simulations`.
- **Populated by:** `pipeline/mlb/build_daily_projections.py` (same script as pitcher projections; upserts on (`snapshot_date`, `batter_id`, `opp_pitcher_id`)).
- **Read by:** 4 API refs — homepage “high total base projections” table, hitter profile projections.

`mlb.game_projections`

- **Purpose:** Game-level run/win projection — projected away/home runs (derived from starters’ `proj_ops` via a calibrated linear formula, see Section 4), projected total/run-diff/winner, plus the actual result and whether the win call was correct (scored the next night).

- **Key columns:** `game_pk` (PK), `game_date`, `away_proj_ops / home_proj_ops`, `proj_away_runs / proj_home_runs / proj_total / proj_run_diff / proj_winner`, `actual_away_runs / actual_home_runs / actual_total / actual_run_diff / actual_winner`, `win_correct`.
- **Populated by:** `pipeline/mlb/build_game_projections.py` — projects using `OPS_SLOPE = 9.0`, `OPS_INTERCEPT = -0.876` (see Section 4), scores against actuals the next night.
- **Read by:** 0 direct API refs found — currently rendered straight from the pipeline script's own query in `generate_homepage.py` (`get_today_game_projections()`), not via a dedicated API endpoint.

Actuals & scoring

`mlb.daily_actuals`

- **Purpose:** Joins the previous day's locked projections to the real box score outcome, one row per player per game — the ground truth used to grade every model.
- **Key columns:** `game_date`, `game_pk`, `player_id`, `player_type` ('pitcher'/'batter'), `team_abbrev / opponent_abbrev`, `actual_k / actual_ip / actual_er / actual_bb / actual_hits_allowed / actual_pitches` (pitchers), `proj_k_pct / proj_hr_pct / proj_ops / proj_ks_finn / proj_ks_locked` (carried over from `daily_projections` for side-by-side comparison), `kalshi_k_line / kalshi_k_yes_price`.
- **Populated by:** `pipeline/mlb/build_actuals.py` (runs 11:15pm PT nightly, joins `boxscore_pitching / boxscore_batting` + `at_bats` to `daily_projections`).
- **Read by:** 1 direct API ref; primarily consumed by `build_model_scores.py` (the next pipeline stage) rather than served directly.

`mlb.model_scores`

- **Purpose:** Daily accuracy scorecard per model/metric — MAE, RMSE, mean projected vs. mean actual, bias, and % of predictions within 1 or 2 of actual.
- **Key columns:** `score_date`, `player_type`, `metric`, `n_players`, `mae`, `rmse`, `mean_proj`, `mean_actual`, `bias`, `within_1`, `within_2`. PK on (`score_date`, `player_type`, `metric`).
- **Populated by:** `pipeline/mlb/build_model_scores.py` (11:20pm PT nightly, right after actuals).
- **Read by:** 2 API refs — homepage "model accuracy" section.

`mlb.projection_calibration`

- **Purpose:** Nightly self-learning calibration factor log — how far off recent K and HR projections ran vs. actual, and the resulting multiplier applied to future projections.
- **Key columns:** `computed_at` (PK), `lookback_days`, `k_mean_proj / k_mean_actual / k_bias_pct / k_pct_calibration` (the actual multiplier used, see Section 4), `hr_mean_proj / hr_mean_actual / hr_rate_multiplier`, `ip_mean_actual / ip_bias`, `notes`.
- **Populated by:** `pipeline/mlb/build_calibration.py` (11:22pm PT nightly, after model scores). `get_current_k_calibration(db)` in the same file reads the latest row back out to apply to the next day's `build_daily_projections.py` run.
- **Read by:** No direct API refs (0) — purely a pipeline-internal feedback loop, not user-facing.

Reference

`mlb.players` / `mlb.player_id_map`

- **Purpose:** `players` is the GUMBO player reference table (name, bats/throws, position, DOB — populated incidentally whenever a player is seen in a fetch). `player_id_map` is the MLBAM ↔ Fangraphs ↔ Baseball-Reference ID crosswalk (via `pybaseball.playerid_lookup()`), the join key that lets `fangraphs_batting / fangraphs_pitching / bbref_batting` (which use Fangraphs/BBRef IDs) join back to GUMBO-based tables (which use MLBAM IDs).
- **Populated by:** `players` — `pipeline/mlb/transform.py`; `player_id_map` — `pipeline/mlb/build_player_map.py`.
- **Read by:** `players` has 0 direct API refs (name lookups mostly go through `at_bats.batter_name / pitcher_name`, which are denormalized at transform time); `player_id_map` has 4 refs.

`mlb.fangraphs_batting` / `mlb.fangraphs_pitching`

- **Purpose:** Season-level Fangraphs advanced stats (wRC+/wOBA/plate-discipline for batters; ERA-/FIP-/Stuff+ for pitchers) via `pybaseball`, refreshed daily during the season — used for cross-validation (see `health_check.py`'s PA cross-reference) and as BAPV+/model validation baselines.
- **Populated by:** `pipeline/mlb/fetch_fangraphs.py`.

- **Read by:** `fangraphs_pitching` 7 refs, `fangraphs_batting` 2 refs.

`mlb.bbref_batting`

- **Purpose:** Season batting totals from Baseball-Reference (`pybaseball.batting_stats_bref()`), used as the ground-truth PA count that `run_pa_validation()` in `daily_pipeline.py` checks our `at_bats`-derived PA against.
- **Populated by:** `pipeline/mlb/fetch_bbref.py`.
- **Read by:** 4 API refs; primary consumer is `daily_pipeline.py::run_pa_validation()`.

`mlb.team_defense`

- **Purpose:** Team defensive quality proxy — BABIP allowed, SLG allowed, extra-base-hit rate allowed, computed from balls in play against each team.
- **Populated by:** `pipeline/mlb/build_team_defense.py` (6:05am PT daily, right after the morning fetch).
- **Read by:** 0 direct API refs — feeds into batter projection adjustments inside `build_daily_projections.py`, not surfaced directly.

`mlb.game_weather`

- **Purpose:** Per-game weather at first pitch plus derived HR-likelihood modifiers (`wind_hr_modifier`, `temp_hr_modifier`, `combined_hr_modifier`) used to adjust HR projections for weather.
- **Populated by:** `pipeline/mlb/fetch_weather.py` (10:00am and 7:00am PT).
- **Read by:** 3 API refs (game companion weather display).

`mlb.game_lineups`

- **Purpose:** Confirmed/expected starting lineups and probable pitchers per game, polled throughout the day as they're posted.
- **Key columns:** `game_pk` (unique), `home_lineup` / `away_lineup` (JSONB), `home_pitcher_id` / `away_pitcher_id` + names, `lineup_posted` (bool — flips true once MLB confirms).
- **Populated by:** `pipeline/mlb/fetch_lineups.py` (every 30 min, 7am–11:30pm PT).
- **Read by:** 3 API refs — drives whether `build_daily_projections.py` can run for a given game and what the homepage displays as confirmed vs. probable.

`mlb.pitcher_splits` / `mlb.batter_splits` / `mlb.player_workload` / `mlb.matchup_history`

- **Purpose:** `pitcher_splits` / `batter_splits` — rate stats (K%/BB%/HR%/AVG/OBP/SLG/OPS) broken out by split type (vs. L/R, home/away, etc.), one row per player/season/split. `player_workload` — 7-day and 28-day workload load and ACWR (acute:chronic workload ratio) per player, used for fatigue/injury-risk signal. `matchup_history` — pitcher-vs-batter head-to-head history with both raw and regressed (shrunk toward league mean for small samples) K%/HR% rates.
- **Populated by:** All four in `pipeline/mlb/build_context_splits.py` (`player_workload` also has its own scheduler entry, `run_build_workload()`), for standalone rebuilds).
- **Read by:** `matchup_history` 2 refs, `pitcher_splits` / `batter_splits` / `player_workload` 1 ref each — all lightly used, likely newer/still-rolling-out features.

`mlb.game_weather` bat-tracking sibling — `mlb.bat_tracking`

- **Purpose:** Statcast bat-tracking metrics (bat speed, squared-up%, blast rate) per batter, where available — an input to `'juiced2_scores'` power-quality columns.
- **Populated by:** `pipeline/mlb/fetch_bat_tracking.py`.
- **Read by:** 0 direct API refs — pipeline-internal input only.

`mlb.pipeline_integrity`

- **Purpose:** Nightly data-completeness audit log (see Section 3 / `build_integrity_check.py`) — games checked/missing/fixed and PA-count mismatches per day.
- **Populated by:** `pipeline/mlb/build_integrity_check.py` (6:10am PT daily).
- **Read by:** 0 direct API refs; read back by `generate_homepage.py::get_latest_integrity_check()` for the morning brief's integrity status bar.

Section 3 — Pipeline Scripts Reference

Pipeline Scripts — Ingestion, Transform & Orchestration

pipeline/mlb/fetch.py

Purpose — Pulls game schedules and raw GUMBO (live-feed) JSON from the MLB Stats API for Final games and stores them in `mlb.raw_events`; also does lightweight status/score refresh without re-pulling GUMBO.

Inputs — MLB Stats API: `GET /v1/schedule` (schedule listing), `GET /v1.1/game/{game_pk}/feed/live` (GUMBO feed).

Outputs — `mlb.raw_events` (insert/update one row per `game_pk`, full JSON blob); `mlb.games` (status/score columns only, via `update_statuses`).

Key functions - `get_schedule(season: int, game_type: str = "R") -> list[dict]` — pulls the full-season schedule for a season/game_type and filters to Final games only. - `get_full_schedule(season: int, game_type: str = "R") -> list[dict]` — same as above but returns ALL statuses plus current scores (used for status refresh, not filtered to Final). - `update_statuses(season: int, game_type: str = "R", db=None) -> dict` — one API call to refresh `status/home_score/away_score` on `mlb.games` for every game in the season without re-fetching GUMBO. Only issues an UPDATE when a value actually changed (`IS DISTINCT FROM`). - `fetch_game(game_pk: int, db) -> str` — fetches one game's GUMBO feed and upserts into `mlb.raw_events`. Returns "inserted", "updated", "skipped", or "error". - `fetch_season(season, game_type="R", transform=False, delay=0.5, db=None)` — loops `get_schedule` results through `fetch_game`, optionally chaining into `transform_game_pk` for newly fetched games. - `main()` — CLI entrypoint (argparse).

Formula annotations - 7-day skip logic in `fetch_game`: if a raw row already exists, its `status == "Final"`, and it was last updated ≥ 7 days ago, the fetch is skipped entirely (assumed immutable/mature data — MLB doesn't revise final box scores after a week). This is a deliberate efficiency short-circuit; note that `build_integrity_check.py` explicitly bypasses it (deletes the stale row first) when it needs to force a genuine re-fetch of incomplete data. - `update_statuses` uses `IS DISTINCT FROM` (NULL-safe inequality) so it only issues writes for genuinely changed rows, minimizing DB churn during frequent polling.

CLI usage

```
# Fetch a single season
PYTHONPATH=/app python3 /pipeline/mlb/fetch.py --season 2024

# Fetch multiple seasons
PYTHONPATH=/app python3 /pipeline/mlb/fetch.py --season 2022 2023 2024

# Fetch and immediately transform
PYTHONPATH=/app python3 /pipeline/mlb/fetch.py --season 2024 --transform

# Spring training
PYTHONPATH=/app python3 /pipeline/mlb/fetch.py --season 2026 --game-type S

# Dry run — just count games, don't fetch
PYTHONPATH=/app python3 /pipeline/mlb/fetch.py --season 2024 --dry-run

# Status-only refresh (no GUMBO re-fetch)
PYTHONPATH=/app python3 /pipeline/mlb/fetch.py --season 2026 --game-type R --update-status
```

Schedule — Not run directly on a standalone cron entry by name; its functions are imported and driven by `daily_pipeline.py`: - `run_mlb_daily()` calls `fetch_game` per game as part of the 6:00am PT `mlb_morning` job. - `run_update_statuses()` (wraps `fetch.py --update-status`) runs at 11:30pm PT nightly (`status_final`) and every 30 min 7am–11:30pm PT (`status_updates`, `offset:15/45`), plus once inside `run_mlb_morning`.

pipeline/mlb/transform.py

Purpose — Reads the raw GUMBO JSON blob from `mlb.raw_events` for a game and normalizes it into all downstream structured tables.

Inputs — `mlb.raw_events` (one JSON blob per `game_pk`).

Outputs — `mlb.games`, `mlb.at_bats`, `mlb.pitches`, `mlb.runners`, `mlb.linescore`, `mlb.boxscore_batting`, `mlb.boxscore_pitching`, `mlb.boxscore_fielding`, `mlb.fielding_credits`. Also indirectly triggers `mlb.backfill_counts.backfill_game` for ball/strike count reconstruction on pitches.

Key functions - `transform_game(raw, db) -> MLBGame` — upserts the one `mlb.games` row: teams, score, venue, weather snapshot, decisions (W/L/S pitchers), no-hitter/perfect-game flags. - `transform_at_bats(raw, db) -> int` — deletes existing at-bats for the

game_pk, then rebuilds one `mlb.at_bats` row per play in `liveData.plays.allPlays` (batter/pitcher, count, event, RBI, scoring flags). - `transform_pitches(raw, db) -> int` — deletes existing pitches, then rebuilds one `mlb.pitches` row per `playEvent` where `isPitch` is true, capturing full Statcast payload (velocity, spin, break, location, hit data). - `transform_runners(raw, db) -> int` — deletes existing runner rows, rebuilds one `mlb.runners` row per base-runner movement event within each play. - `transform_linescore(raw, db) -> int` — rebuilds `mlb.linescore` (runs/hits/errors/LOB per half-inning). - `transform_boxscore(raw, db) -> tuple[int,int,int]` — rebuilds `mlb.boxscore_batting`, `mlb.boxscore_pitching`, `mlb.boxscore_fielding` from MLB's own pre-computed per-player boxscore stats (not derived from our pitch/at-bat data — these are MLB's authoritative cumulative numbers, including MLB's own `avg / obp / slg / ops / era / whip`). - `transform_fielding_credits(raw, db) -> int` — rebuilds `mlb.fielding_credits` (putout/assist/error credit per fielder per runner event). - `transform_game_pk(game_pk: int, db)` — orchestrates all of the above for one game_pk, in order, printing a per-step row count. - `main()` — CLI entrypoint (argparse): `--game-pk` (single) or `--all` (every game_pk currently in `mlb.raw_events`, each in its own DB session).

Formula annotations - `batting_order = int(str(batting_order)[:1]) * 100 if batting_order else None` — GUMBO's raw `battingOrder` field is a string like "100", "201" (spot 2, substitution 1); this truncates to the leading digit and multiplies by 100, so subs and the original starter both collapse to the same lineup-spot bucket (100 = 1st, 200 = 2nd, etc). - IP is stored as MLB's own decimal-ish string (e.g. "6.1" meaning 6⅓ innings) cast straight to float — see Section 4 for why this is NOT true decimal innings.

CLI usage

```
PYTHONPATH=/app python3 /pipeline/mlb/transform.py --game-pk 832034
PYTHONPATH=/app python3 /pipeline/mlb/transform.py --all
```

Schedule — Not scheduled standalone; `transform_game_pk` is called per-game from `run_mlb_daily()` (part of the 6:00am PT `mlb_morning` job) and from `build_integrity_check.py`'s `_force_fetch_and_transform`.

`pipeline/mlb/fetch_kalshi.py`

Purpose — Pulls live Kalshi prediction-market prices for MLB strikeout props (series `KXMLBKS`) and matches each market to an MLBAM pitcher ID.

Inputs — Kalshi API `GET /trade-api/v2/markets?series_ticker=KXMLBKS` (paginated); `mlb.at_bats` (for name→MLBAM ID matching, using pitcher names already seen in our own data).

Outputs — `mlb.kalshi_markets` (upsert on `(fetch_date, ticker)`).

Key functions - `fetch_kalshi_markets() -> list` — paginates the Kalshi markets endpoint (cursor-based, retries on HTTP 429) and returns all raw market dicts for the series. - `extract_player_name(market: dict) -> str` — parses the player's name out of the market title, e.g. "Cade Cavalli: 9+ strikeouts?" → "Cade Cavalli". - `match_player_to_mlbam(player_name: str, db) -> tuple` — resolves a Kalshi player name to `(mlbam_id, canonical_name)` via exact match against `mlb.at_bats.pitcher_name`, falling back to a last-name + first-initial LIKE match. - `parse_game_date(ticker: str) -> date` — decodes the game date embedded in the Kalshi ticker string (format `KXMLBKS-26MAY201305CINPHI` → `2026-05-20`). - `store_markets(markets: list, db) -> tuple[int,int]` — computes `implied_prob` per market and upserts into `mlb.kalshi_markets`; skips markets with no valid pricing. - `backfill_missing_mlbam(db, fetch_date=None)` — re-attempts name matching for any rows stored earlier in the day that couldn't be matched to an MLBAM ID at fetch time.

Formula annotations - Implied probability: `implied_prob = yes_bid / (yes_bid + no_bid)` when both bid sides are priced; falls back to `yes_ask` directly if only the ask side has a price. This is the standard "mid-market" probability estimate for a binary prediction market — bid/bid ratio approximates fair value better than using either side's ask alone.

CLI usage

```
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/fetch_kalshi.py
```

(No argparse — always fetches all current markets, no date/season flags.)

Schedule — `run_fetch_kalshi()` runs at 7:30am PT and then every 30 minutes from 8:00am–11:00pm PT (paired with `run_daily_projections` in the same slots), 32 runs/day.

`pipeline/mlb/fetch_lineups.py`

Purpose — Polls the MLB schedule API (with `lineups,probablePitcher` hydration) for today's posted starting lineups and probable pitchers.

Inputs — MLB Stats API `GET /v1/schedule?hydrate=lineups,probablePitcher`.

Outputs — `mlb.game_lineups` (creates table if missing; upsert on `game_pk`).

Key functions - `fetch_lineups_for_date(game_date: str, db) -> dict` — fetches the day's schedule with lineup hydration; skips any game where neither `homePlayers` nor `awayPlayers` is populated yet (lineup not posted). - `store_lineups(lineups: dict, db) -> int` — `CREATE TABLE IF NOT EXISTS mlb.game_lineups`, then upserts each game's home/away lineup (as JSONB) and probable pitcher IDs/names.

Formula annotations — None (pure ingestion/reshaping, no derived math).

CLI usage

```
python3 /pipeline/mlb/fetch_lineups.py --date 2026-07-22
# defaults to today if --date omitted
```

Schedule — `run_fetch_lineups()` polls every 30 minutes from 7am–11:30pm PT (`lineup_poll` job, offset :00/:30).

pipeline/mlb/fetch_bbref.py

Purpose — Downloads Baseball-Reference's public WAR CSV exports directly (no scraping library) to get OPS+/ERA+/WAR/PA that aren't available from the MLB Stats API, and syncs BBref player IDs via the Chadwick register.

Inputs — `https://www.baseball-reference.com/data/war_daily_bat.txt`, `.../war_daily_pitch.txt` (raw CSV text); Chadwick Bureau register CSV shards (`people-0.csv` ... `people-f.csv`) on GitHub for ID crosswalk.

Outputs — `mlb.bbref_batting` (upsert on `(mlb_id, year_id)`), `mlb.bbref_pitching` (upsert on `(mlb_id, year_id, team_id)`), `mlb.player_id_map.bbref_id` (UPDATE only, never inserts new player rows).

Key functions - `fetch_bbref_batting(season: int, db) -> int` — downloads the batting WAR CSV, filters to the season, aggregates multi-team stints per player (PA-weighted average of OPS+), upserts `mlb.bbref_batting`. - `fetch_bbref_pitching(season: int, db) -> int` — same pattern for pitching, aggregating by IPouts-weighted ERA+. - `update_bbref_ids(db)` — only runs if any `mlb.player_id_map` rows are missing `bbref_id`; downloads all 16 Chadwick shards and back-fills `bbref_id` by MLBAM ID match. - `_safe_int(val) / _safe_float(val)` — NaN-safe casters for pandas values.

Formula annotations - **Multi-team PA-weighted OPS+**: $ops_plus = \frac{\sum(OPS_plus_stint \times PA_stint)}{\sum(PA_stint)}$ across all of a player's stints in a season — a standard weighted-average approach so a 500-PA everyday player isn't diluted evenly with a 20-PA September call-up stint on another team. - Same weighted-average pattern applied to pitching `ERA_plus`, weighted by IPouts (outs recorded, i.e. IP $\times$ 3) rather than innings-pitched decimal, avoiding the baseball-notation IP rounding problem (see Section 4).

CLI usage

```
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/fetch_bbref.py --season 2026
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/fetch_bbref.py --season 2025
# flags: --batting-only, --pitching-only, --skip-bbref-ids
```

Schedule — `run_fetch_bbref()` (hardcoded `--season 2026`) runs as part of `run_mlb_morning()` at 6:00am PT, right after `run_mlb_daily()`.

pipeline/mlb/fetch_weather.py

Purpose — Fetches game-time weather (temp/wind/humidity/precip) per ballpark from the free Open-Meteo API and derives HR-probability modifiers from wind and temperature.

Inputs — Open-Meteo `GET /v1/forecast` (hourly forecast, keyed by ballpark lat/lon); `mlb.games` (to find today's games needing weather).

Outputs — `mlb.game_weather` (creates table if missing; upsert on `game_pk`).

Key functions - `create_weather_table(db)` — `CREATE TABLE IF NOT EXISTS mlb.game_weather`. - `compute_wind_modifier(wind_speed, wind_dir, cf_direction) -> float` — converts meteorological wind direction + park center-field orientation into an HR probability multiplier. - `compute_temp_modifier(temp_f) -> float` — linear temperature→HR-multiplier adjustment. - `get_weather_condition(wmo_code) -> str` — maps Open-Meteo's WMO weather code to a human label (Clear/Rain/Snow/etc). - `fetch_weather_for_game(game_pk, game_date, team_abbrev, db) -> dict` — for indoor parks, short-circuits to a fixed 72°F/no-wind/1.0-modifier row; for outdoor parks, pulls the Open-Meteo hourly forecast for the game date and reads the hour-19 (7pm ET) slot as the “gametime” weather. - `fetch_today_weather(db)` — finds today's games in `mlb.games` missing a `mlb.game_weather` row and fetches each.

Formula annotations - **Wind modifier**: ball-flight direction is the wind direction rotated 180° (`ball_wind_dir = (wind_dir + 180) % 360`, since wind *direction* is reported as where it blows *from*), compared against the park's center-field bearing. `cf_component = cos(angle_diff_in_radians)` — this is +1 when wind blows straight out to CF, -1 when blowing straight in from CF, 0 for a pure crosswind. `modifier = 1.0 + cf_component × wind_speed × 0.008`, clamped to [0.85, 1.15]. At the file's documented calibration (10

mph dead-out wind): $1 + 1.0 \times 10 \times 0.008 = 1.08$, i.e. $\sim +8\%$ HR probability per 10mph dead-out wind — matches the docstring's stated "+8% HR boost per 10mph." - **Temp modifier**: $\text{modifier} = 1.0 + ((\text{temp_f} - 70.0) / 10) \times 0.015$, clamped to $[0.92, 1.08]$ — +1.5% HR probability per 10°F above a 70°F baseline (warmer/less-dense air → ball carries farther), matching the docstring. - **Combined modifier**: $\text{combined} = \text{wind_mod} \times \text{temp_mod}$ (multiplicative, not additive). - The **hour-19 assumption** ($\text{hour_idx} = 19$, i.e. 7pm ET) is a fixed approximation of typical first-pitch time — day games or non-ET-zone games (e.g. West Coast 7pm PT starts) will get weather from the wrong actual game hour.

CLI usage

```
python3 /pipeline/mlb/fetch_weather.py --today
python3 /pipeline/mlb/fetch_weather.py --game-pk 832034 --team SD --date 2026-07-22
python3 /pipeline/mlb/fetch_weather.py --date 2026-07-22 # (falls through to fetch_today_weather regardless - see note below)
```

Note: the `--date` flag is accepted by argparse but not actually threaded into `fetch_today_weather()`, which always uses `date.today()` internally — passing `--date` alone has no effect unless `--game-pk` and `--team` are also given.

Schedule — `run_fetch_weather()` runs at 7:00am PT daily and as part of `run_mlb_morning()` at 6:00am PT.

pipeline/mlb/fetch_fangraphs.py

Purpose — Pulls season-level advanced batting/pitching stats (wOBA, wRC+, Stuff+, plate-discipline%, Statcast aggregates) from Fangraphs via the `pybaseball` library.

Inputs — `pybaseball.batting_stats(season, qual=10)`, `pybaseball.pitching_stats(season, qual=5)` (wrap Fangraphs' own leaderboard scrape/cache).

Outputs — `mlb.fangraphs_batting`, `mlb.fangraphs_pitching` (full delete-and-reinsert per season, not upsert).

Key functions - `fetch_batting(season, db) -> int` — deletes existing `mlb.fangraphs_batting` rows for the season, bulk-inserts fresh rows mapped from the Fangraphs dataframe (avg/obp/slg/ops, wOBA, wRC+, OPS+, barrel%/hard-hit%/exit velo, plate-discipline%, sprint speed). - `fetch_pitching(season, db) -> int` — same pattern for pitching (ERA/WHIP, ERA+/FIP/xFIP/SIERA, K%/BB%, Stuff+, plate-discipline%). - `safe_float(val) / safe_int(val)` — NaN/empty-string-safe casters.

Formula annotations — None computed locally; all values are Fangraphs' own pre-computed metrics, passed through as-is.

CLI usage

```
PYTHONPATH=/app python3 /pipeline/mlb/fetch_fangraphs.py --season 2026
PYTHONPATH=/app python3 /pipeline/mlb/fetch_fangraphs.py --season 2023 2024 2025 2026
```

Schedule — `run_fangraphs_daily()` (hardcoded to `datetime.now().year`) runs as part of `run_mlb_morning()` at 6:00am PT. Not otherwise on a standalone cron entry.

pipeline/mlb/fetch_bat_tracking.py

Purpose — Pulls Statcast bat-tracking leaderboard data (bat speed, swing length, squared-up%, blast rate, attack angle) from Baseball Savant.

Inputs — Baseball Savant `GET /leaderboard/bat-tracking?csv=true` (per season, per batter/pitcher type).

Outputs — `mlb.bat_tracking` (creates table if missing; upsert on `(mlbam_id, season, player_type)`).

Key functions - `fetch_bat_tracking_leaderboard(season, player_type="batter") -> pd.DataFrame` — downloads the Savant CSV leaderboard. - `create_bat_tracking_table(db)` — `CREATE TABLE IF NOT EXISTS mlb.bat_tracking + index on (mlbam_id, season)`. - `store_bat_tracking(df, season, player_type, db) -> int` — resolves the MLBAM ID from whichever ID column Savant used that export (`player_id / mlbam_id / batter / pitcher / id`), computes `swords_pct`, upserts.

Formula annotations - Swords%: $\text{swords_pct} = \text{swords_count} / \text{pa_val} \times 100$ — "swords" (Statcast's term for a badly-fooled swing) as a percentage of competitive swings (`swings_competitive`), rounded to 2 decimals. Only computed when both values are present and `pa_val > 0`. - Column-name fallback chains (e.g. `row.get('avg_bat_speed', row.get('bat_speed'))`) exist because Savant has renamed these leaderboard columns across seasons — the comment block in the file documents the as-of-2025-2026 canonical names.

CLI usage

```
python3 /pipeline/mlb/fetch_bat_tracking.py --season 2025 2026
# defaults to [2025, 2026] if --season omitted
```

Schedule — Not referenced anywhere in `daily_pipeline.py` — **not currently scheduled**. Must be run manually.

`pipeline/mlb/build_integrity_check.py`

Purpose — Post-fetch data-completeness auditor: finds games missing or truncated in our DB versus the MLB schedule API, auto-repairs them, cross-checks PA counts against the live MLB boxscore API, and alerts on anything unresolved. (Built earlier in this session — see prior turn for full build/verification detail.)

Inputs — MLB Stats API `GET /v1/schedule` (per-day, `gameType=R,S,P`) and `GET /v1/game/{game_pk}/boxscore`; `mlb.games`, `mlb.raw_events`, `mlb.boxscore_batting`.

Outputs — `mlb.pipeline_integrity` (creates table if missing; upsert on `(check_date, check_type)`); indirectly repairs `mlb.raw_events` / `mlb.games` / etc. via forced fetch+transform; sends alert email via `send_pipeline_integrity_alert`.

Key functions - `_force_fetch_and_transform(game_pk: int, db) -> bool` — deletes the existing `mlb.raw_events` row for the game (to defeat `fetch.py`'s 7-day skip guard), then calls `fetch_game + transform_game_pk` fresh. - `find_and_fix_missing_games(db, lookback_days=30) -> dict` — walks the schedule day-by-day over the lookback window; for each Final game, flags `missing_from_db` (no `mlb.games` row and no raw row), `missing_raw_data` (no raw row but a games row somehow exists), or `incomplete_raw_events` (raw row exists but its `liveData.plays.allPlays` array has fewer than `MIN_RAW_EVENTS=10` entries), then force-repairs each. Returns counts plus per-game detail lists. - `validate_pa_counts(db, lookback_days=7) -> list` — for each Final game in the window, counts `mlb.boxscore_batting` rows with `at_bats > 0` and compares to the count of batter IDs in the MLB boxscore API's `teams.{side}.batters` list whose `players["ID<id>"].stats.batting.atBats > 0`; flags a discrepancy when `abs(diff) > PA_MISMATCH_THRESHOLD (2)`. - `write_integrity_results(db, check_date, missing_summary, pa_discrepancies)` — creates the table if needed, upserts one row per `(check_date, 'nightly')`, derives `status (ok/warning/error)` from whether anything was missing, unfixable, or mismatched. - `main()` — orchestrates the 4 steps end-to-end, `argparse --lookback` (default 30) and `--pa-lookback` (default 7).

Formula annotations - `MIN_RAW_EVENTS = 10` — an arbitrarily-chosen floor; any real MLB game has 60-90+ plays, so single digits is treated as a truncated/failed raw pull. - `PA_MISMATCH_THRESHOLD = 2` — allows up to a 2-PA discrepancy (e.g. late scratches, pinch-hit edge cases) before flagging.

CLI usage

```
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/build_integrity_check.py --lookback 30
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/build_integrity_check.py --lookback 30 --pa-lookback 7
```

Schedule — `run_integrity_check()` runs daily at 6:10am PT (`integrity_check_daily`), after the 6:00am fetch and before the 6:30am homepage generation. Also available via `--now integrity`.

`pipeline/mlb/health_check.py`

Purpose — Standalone data-quality report for a single date: cross-references game/PA/pitch counts against the MLB schedule API and Fangraphs, flags coverage gaps, duplicates, and stuck non-Final games. Prints a pass/fail summary; does not write to the DB.

Inputs — MLB Stats API `GET /v1/schedule` (Final-game count for the date); `mlb.games`, `mlb.raw_events`, `mlb.at_bats`, `mlb.pitches`, `mlb.runners`, `mlb.fangraphs_batting`.

Outputs — None (console report only — no DB writes).

Key functions - `get_schedule_count(date_str, db) -> int` — count of Final games on the MLB schedule for the date. - `run_health_checks(date_str, db) -> bool` — runs 7 checks in sequence and returns overall pass/fail: 1. Game coverage (stored vs. scheduled count) + raw-event count. 2. Fangraphs season-PA cross-reference (with a per-player diagnostic breakdown when the gap is large). 3. Plate-appearance sanity (25–45 PA/game expected range). 4. Pitch count per game (<200 pitches flagged as a failure). 5. Statcast coverage (spin-rate/launch-speed/location % populated). 6. Duplicate `game_pk` detection in `mlb.games`. 7. Non-Final game-status detection (games stuck in “In Progress”/“Postponed”/etc past their date). 8. Runner-data completeness (games with `mlb.runners` rows vs. games with `mlb.at_bats` rows). - `main()` — CLI: `--date YYYY-MM-DD`, `--yesterday`, or `--today` (mutually exclusive, required).

Formula annotations - PA/game sanity band of 25–45 is an empirical range (9 batters × ~4-5 trips through the order after accounting for extra innings/blowouts on either side). - 200-pitch floor is roughly the minimum for two full 9-inning pitching staffs even in a fast, low-scoring game. - Fangraphs diff diagnostic only runs `if diff != 0 and abs(diff) > 50` — surfaces individual player-level PA mismatches >5 to help pinpoint which games/players are causing an aggregate gap.

CLI usage

```
PYTHONPATH=/app python3 /pipeline/mlb/health_check.py --date 2026-04-01
PYTHONPATH=/app python3 /pipeline/mlb/health_check.py --yesterday
PYTHONPATH=/app python3 /pipeline/mlb/health_check.py --today
```

Schedule — Not scheduled standalone; `run_health_checks()` is imported and called inline at the end of `run_mlb_daily()` (part of the 6:00am PT `mlb_morning` job) for yesterday's date.

pipeline/daily_pipeline.py

Purpose — The master orchestrator: defines every scheduled job (APScheduler `BlockingScheduler`, all times America/Los_Angeles) and wraps every pipeline script behind a uniform `--now <name>` manual-trigger CLI.

Inputs/Outputs — None directly; it's a thin wrapper that imports and calls functions from every other `pipeline/mlb/*.py` script (and `pipeline/nascar/`, `pipeline/f1/`), or shells out to them via `_run_script` (subprocess with `PYTHONPATH=/app:/pipeline`, 1800s timeout, captures stdout/stderr).

Key functions - `run_mlb_daily()` — fetches + transforms yesterday's Final MLB games, then runs `health_check.run_health_checks`.
- `get_schedule_for_date(date_str) -> list` — thin duplicate of `fetch.get_schedule` scoped to a single date (used by `run_mlb_daily`).
- `run_fangraphs_daily()`, `run_fetch_bbrief()`, `run_fetch_weather()`, `run_fetch_kalshi()`, `run_fetch_lineups()`, `run_integrity_check()`, `run_generate_homepage()`, `run_update_statuses()`, `run_build_*` (BAPV+, pitcher mix, Goose+/Goose+2/Goose+3, team defense, Stuff Score, pK+/pHR+, Juiced+2, context splits/workload, actuals, model scores, calibration, game projections, batter tendencies, daily projections) — each a one-line wrapper calling `_run_script(...)` with the target script path and CLI args, OR (for `run_mlb_daily` / `run_fangraphs_daily` / `run_pa_validation`) calling the target module's functions in-process.
- `run_pa_validation() -> dict` — compares season-total PA between `mlb.bbref_batting` and `mlb.at_bats` (excluding non-PA event types like caught stealing/wild pitch/passed ball), flags a warning if the aggregate gap exceeds 2%, and logs individual player-level gaps >10 PA. Distinct from `build_integrity_check.py`'s per-game PA check — this one is season-aggregate, BBref-sourced, and log-only (no DB write, no email of its own — its result dict feeds into the mlb-morning integrity email).
- `run_send_integrity_email(pipeline_log, pa_results)` — sends the `send_email.send_integrity_email` report (pipeline run log + PA validation summary) — separate from `build_integrity_check.py`'s alert email.
- `run_send_daily_brief()` — emails the rendered `/app/static/homepage.html` via `send_email.send_daily_brief_email`.
- `_run_script(name, script, args=None)` — generic subprocess runner; logs the last 3 non-empty stdout lines on success, or the last 600 chars of stderr on failure; 30-minute timeout.
- `run_mlb_morning()` — the big sequential 6am chain: Status Update → MLB Fetch → BBref → PA Validation → Fangraphs → BAPV+ → Pitcher Mix → Goose+ → Runners → Stuff Score → pK+/pHR+ → Juiced+2 → Team Defense → Goose+3 → Context Splits → Weather → Model Scores → Calibration → Daily Projections, each wrapped in a `tracked()` helper that records success/failure/duration/detail into `pipeline_log`, then emails the integrity report regardless of individual step failures.
- `run_nascar_sunday()`, `run_f1_sunday()` — separate-sport pipelines, unrelated to MLB.
- `main()` — `argparse --now <choice>` for manual single-job runs, or (no `--now`) builds and starts the full `BlockingScheduler`.

Formula annotations — See `run_pa_validation()` above (2% aggregate PA-gap threshold, >10 PA per-player flag threshold) — this is the one piece of non-trivial logic in this file; everything else is scheduling/orchestration.

CLI usage

```
PYTHONPATH=/app python3 /pipeline/daily_pipeline.py           # run scheduler (blocking)
PYTHONPATH=/app python3 /pipeline/daily_pipeline.py --now mlb # MLB fetch+transform+health-check now
PYTHONPATH=/app python3 /pipeline/daily_pipeline.py --now mlb-morning # full 6am chain now
PYTHONPATH=/app python3 /pipeline/daily_pipeline.py --now integrity # integrity check now
PYTHONPATH=/app python3 /pipeline/daily_pipeline.py --now pk-phr # pK+/pHR+ now
PYTHONPATH=/app python3 /pipeline/daily_pipeline.py --now goose3 # Goose+3 now
PYTHONPATH=/app python3 /pipeline/daily_pipeline.py --now weather # weather now
PYTHONPATH=/app python3 /pipeline/daily_pipeline.py --now all # run everything now
```

Full `--now` choice list: `mlb`, `mlb-morning`, `fangraphs`, `bbref`, `pa-validation`, `bapv`, `pitcher-mix`, `goose`, `stuff-score`, `pk-phr`, `juiced2`, `goose3`, `splits`, `workload`, `scores`, `game-scores`, `calibration`, `weather`, `runners`, `kalshi`, `actuals`, `homepage`, `batter-tendencies`, `daily-projections`, `lineups`, `update-status`, `team-defense`, `nascar`, `f1`, `integrity`, `all`.

Schedule — the full cron table (all times PT):

Time	Job(s)
11:15pm nightly	<code>run_build_actuals</code> — build daily actuals
11:20pm nightly	<code>run_build_model_scores</code> — score projections vs actuals
11:22pm nightly	<code>run_build_calibration</code> — update K/HR calibration factors
11:25pm nightly	<code>run_build_game_projections</code> — score game run/W-L projections
11:30pm nightly	<code>run_update_statuses</code> — final status update

Time	Job(s)
6:00am daily	<code>run_mlb_morning</code> — full ingestion→models→projections chain + integrity email
6:05am daily	<code>run_build_team_defense</code> — team defense rebuild
6:10am daily	<code>run_integrity_check</code> — data integrity check (new)
6:30am daily	<code>run_generate_homepage</code> — morning brief
6:35am daily	<code>run_send_daily_brief</code> — email morning brief
7:00am daily	<code>run_fetch_weather</code> , <code>run_build_workload</code>
7:30am daily	<code>run_daily_projections</code> , <code>run_fetch_kalshi</code>
8:00am–11:00pm, every 30 min	<code>run_daily_projections</code> , <code>run_fetch_kalshi</code> (32 runs/day each)
9am–9pm, hourly (:00)	<code>run_build_today_game_projections</code>
9am–9pm, hourly (:02)	<code>run_generate_homepage</code>
9:00am / 12:00pm / 4:00pm	<code>run_generate_homepage</code> + <code>run_send_daily_brief</code> (offset +5 min)
7am–11:30pm, every 30 min	<code>run_fetch_lineups</code> (lineup poll)
7am–11:30pm, every 30 min (offset :15/:45)	<code>run_update_statuses</code>
Mon 4:00am	<code>run_build_batter_tendencies</code> — weekly rebuild

Pipeline Scripts — Simulation & Projections

pipeline/mlb/simulate.py

Purpose — Shared Monte Carlo at-bat (PA) simulation engine. Every projection script (`build_daily_projections.py`, `build_game_projections.py`) indirectly via `daily_projections`, `simulate_innings.py`) imports its loader/simulation functions rather than duplicating logic.

Inputs — `mlb.pitcher_count_mix`, `mlb.pitcher_mix_profile`, `mlb.pitches/mlb.at_bats/mlb.games` (for zone rate), `mlb.batter_pitch_type_tendencies`, `mlb.goose2_scores`, `mlb.juiced2_scores`, `mlb.stuff_scores`, `mlb.goose_overall`, `mlb.pk_scores`, `mlb.phr_scores`, `mlb.goose3_scores`, `mlb.team_defense`, `mlb.game_weather`, `mlb.pitcher_splits`, `mlb.matchup_history`.

Outputs — None directly (pure library/function module). Callers persist results to `mlb.daily_projections`, `mlb.daily_batter_projections`, or return JSON to the API.

Key functions

- `load_pitcher_count_mix(pitcher_id, bat_side, db) -> dict` — pulls the pitcher's actual pitch mix by count (balls, strikes) from `pitcher_count_mix` for 2026 then 2025; when only 2025 exists, rescales 2025 whiff/CSW rates by the ratio of 2026 vs 2025 `pitcher_mix_profile` rates so current-season stuff changes show through even without count-level 2026 data.
- `load_league_count_mix(db) -> dict` — league-average pitch mix by count (2025, ≥100 pitches per bucket), pitch-weighted average. Cached at module level for the process lifetime (`_league_mix_cache`).
- `load_pitcher_overall_mix(pitcher_id, bat_side, db) -> list` — pitcher's overall (not count-specific) platoon pitch mix; final fallback in the chain.
- `load_pitcher_zone_rate(pitcher_id, db) -> float` — pitcher's empirical zone rate from `mlb.pitches.zone` (zones 1-9 = full weight, 11-14 shadow zone = 0.12 weight), clamped to [0.40, 0.62], defaulting to 0.510.
- `load_batter_tends(batter_id, db) -> dict` — batter's whiff/chase/hard-hit/exit-velo tendencies per pitch type, 2025 season.
- `load_goose2_pitch_adjustments(pitcher_id, bat_side, db) -> dict` — converts each pitch type's Goose+2 score into whiff/CSW/contact multipliers via $adj = 1.0 + (goose2_plus - 100) / 200$ (i.e. a 50%-dampened linear scaling around 100 = league average).
- `load_juiced2_batter_adjustments(batter_id, db) -> dict` — same linear $1.0 + (x-100)/200$ transform applied to the batter's Juiced+2 score, producing `ops_mult / contact_mult / hr_mult`.
- `load_stuff_score(pitcher_id, bat_side, db) -> dict` — Stuff Score → adjustment via $adj = clamp(1.0 + (stuff - 100)/60, 0.50, 2.00)$ (steeper scaling than Goose+2's /200 — Stuff Score moves the needle more per point).
- `load_combined_pitcher_adjustments(pitcher_id, bat_side, db) -> dict` — blends Stuff Score (35%), Goose+2 (35%), and Goose+1 (30%) into one multiplier set per pitch type; weights renormalize when a signal is missing. **Not actually used by `run_simulation`** — superseded by the Goose+3 / pK+/pHR+ chain below (dead code / legacy path retained for reference or other callers).
- `load_pk_phr_adjustments(pitcher_id, bat_side, db) -> dict` — pK+ drives whiff/CSW multipliers ($clamp(1.0 + (pk_plus - 100)/60, 0.50, 2.00)$), pHR+ drives contact/HR multipliers the same way. Fallback signal when Goose+3 has insufficient

pitches.

- `load_goose3_adjustments(pitcher_id, bat_side, db) -> dict` — the current primary pitcher-adjustment signal: Stuff Score → whiff/CSW mult, Goose+2 → contact/HR mult, same /60 scaling. Falls back to a season-level `GROUP BY pitch_type_code` average when the bat-side-specific row is missing.
- `_combine_mults(*mults, lo=0.60, hi=1.50) -> dict` — multiplies `k_mult / bb_mult / hr_mult / contact_mult` across any number of adjustment dicts and clamps the product to [0.60, 1.50].
- `load_tto_adjustment(pitcher_id, season, tto, db) -> dict` — time-through-the-order split multiplier; regression weight `w = min(0.80, tto_pa / (tto_pa + 30))` blends the specific-TTO split rate with the pitcher's overall rate before taking the ratio to overall.
- `load_h2h_adjustment(pitcher_id, batter_id, db) -> dict` — head-to-head history multiplier; SQL itself performs Bayesian shrinkage toward league average with a 20-PA regression constant (`k_pct_reg / hr_pct_reg` computed in-query), then the Python layer just divides by league average to get a multiplier.
- `normalize_mix(mix) -> list / get_mix_for_count(...) -> list` — pitch-selection fallback chain: (1) pitcher's count-specific mix if ≥ 15 pitches in that count, else (2) a 60/40 blend of league-count-mix and the pitcher's overall mix, else (3) pitcher's overall mix alone.
- `select_pitch(mix) -> dict` — weighted-random draw over the pitch mix (cumulative-probability roulette wheel).
- `get_contact_probs(pitch_type, batter_tends, defense=None) -> dict` — converts a batter's hard-hit rate and average exit velo into single/double/triple/HR/field-out probabilities on a ball in play; see Formula Annotations below.
- `resolve_contact(...) / resolve_contact_v2(...) -> str` — draws the actual BIP outcome from `get_contact_probs`, v2 additionally scales HR probability by `hr_mult × weather_hr_mult` and hit probability by `contact_mult`.
- `load_team_defense(opp_team_id, db) -> dict` — opposing team's BABIP-allowed/XBH-rate/etc. from `mlb.team_defense`, keyed by `team_id`.
- `load_weather_modifier(game_pk, db) -> float` — `combined_hr_modifier` from `mlb.game_weather`, default 1.0.
- `load_game_state_adjustments(pitcher_id, batter_id, on_first, on_second, on_third, outs, db) -> dict` — base-out-state (bases-empty / runners-on / RISP × no-outs / one-out / two-outs) K/BB/HR multiplier. Both pitcher's and batter's rate in that specific state are individually regressed toward league average (regression constant 30 PA), then the multiplier is `sqrt(pitcher_ratio × batter_ratio)` — a geometric mean of how much each side over/under-performs league average in that state — clamped to [0.70, 1.30].
- `simulate_pa(...) -> dict` — the actual pitch-by-pitch PA simulator; see Formula Annotations.
- `aggregate_outcomes(results, n_sims) -> dict` — rolls up a list of simulated PA outcomes into AVG/OBP/SLG/OPS, K%/BB%/HR%, average pitches per PA, and an expected-run-value figure using a fixed linear-weights-style `RUN_VALUES` table.
- `run_simulation(pitcher_id, batter_id, n_sims, db, ...)` -> dict — full single-matchup runner: loads all adjustment layers, prefers Goose+3 over pK+/pHR+, combines base-out-state × TTO × H2H splits multiplicatively via `_combine_mults`, runs `n_sims` `simulate_pa` calls, returns aggregated results plus sample pitch-by-pitch logs (first `record_sample` PAs) and diagnostic context (which data sources/fallbacks were used).
- `load_batter_overall_rates(batter_id, db) -> dict` — batter's overall (not pitch-type-specific) swing/whiff/chase/zone-swing/contact rates from raw `mlb.pitches`, used as a secondary fallback inside `simulate_pa` when pitch-type tendency data is thin.

Formula annotations

- *Pitch-type adjustment scaling* (lines 272, 332, 478-479, 542-544): Goose+2 uses `adj = 1.0 + (score-100)/200`; Stuff Score, pK+, and pHR+ use `adj = clamp(1.0 + (score-100)/60, 0.50, 2.00)`. In both cases 100 = league average and the adjustment multiplies the base whiff/CSW/contact rate. The /60 denominator (vs /200) means Stuff/pK+/pHR+ scores move outcome probabilities roughly 3.3× more per index point than Goose+2 does — i.e. Goose+2 is treated as a softer signal in this codebase.
- *Ball-in-play outcome model* (`get_contact_probs`, lines 740-801): Starting from a batter's `hard_hit_rate` (`hh_rate`) and `avg_exit_velo` (`avg_ev`), computed against the 2026-actuals baseline (league BABIP=.280, SLG/BIP=0.499, HR/BIP=0.040, HardHit%=39.5%):
 - `hr_from_hh = clamp(0.040 × (hh_rate / 0.395), 0.001, 0.10)` — HR rate scales linearly with the batter's hard-hit rate relative to league average.
 - `ev_boost = clamp((avg_ev - 88.0) / 600.0, -0.015, 0.015)` — a small secondary nudge from raw exit velocity (dampened 600× because contact-EV samples in GUMBO are small and don't capture launch angle).
 - `babip = clamp(0.280 × (1 + (avg_ev - 88.0)/125.0), 0.15, 0.40)` — BABIP scales gently with exit velocity around the league-average 88 mph anchor.
 - Defense adjustment: `babip *= clamp(def_babip_allowed / 0.290, 0.80, 1.25)` when an opposing `team_defense` row is supplied.
 - Non-HR hits `non_hr_hit = babip × (1 - hr_adj)` are split into single/double/triple using `double_frac = clamp(0.24 × (1 + (avg_ev-88)/60), 0.10, 0.40)` (also nudged by opposing defense's XBH rate) and a fixed `triple_frac = 0.02`; the rest goes to singles.
- *PA state machine* (`simulate_pa`, lines 1026-1198): iterates up to 20 pitches. Each pitch: selects a pitch type from the count-specific mix, determines in-zone vs. out-of-zone via `random() < zone_rate`, determines swing decision via a base zone-swing-

rate or chase-rate scaled by a count-specific multiplier table (`count_zone_adj / count_chase_adj` , e.g. batters chase 26% more on 0-2 counts than their baseline), then on a swing resolves whiff vs. foul vs. ball-in-play:

- Whiff probability blends pitcher stuff and batter contact ability: `in-zone whiff_prob = pitcher_whiff × (0.65 + 0.35×batter_whiff_mod) × k_mult` ; out-of-zone weights pitcher 70%/batter 30%. `batter_whiff_mod = batter_whiff / LEAGUE_WHIFF (0.267)` .
- Foul probability `foul_prob = clamp(base_foul[strikes] × contact_scale, 0.28, 0.58)` where `base_foul` = {0 strikes: 0.48, 1: 0.46, 2: 0.40} and `contact_scale = clamp(batter_contact/0.78, 0.85, 1.15)` — high-contact hitters foul off more pitches, extending PAs (calibrated against TruMedia reference points: Arraez 92% contact → 43.7% foul rate; Schwarber 67% contact → 38.9%).
- Walks occur at `balls >= walk_threshold`, where `walk_threshold = max(3, round(4/bb_mult))` when `bb_mult > 1.0` (a hitter drawing more walks than average can “walk” on a 3-ball count in the simulation, modeling extra patience/deeper counts translating to more free passes).
- `aggregate_outcomes` : standard sabermetric formulas — `AVG = H/AB` , `SLG = TB/AB` , `OBP = (H+BB)/n_sims` (note: this simulation-level OBP formula omits HBP/SF because the simulator doesn’t model HBP as a distinct outcome — walks are the only non-batted-ball non-strikeout/out result), `OPS = OBP+SLG` , where `AB = n_sims - walks` .

CLI usage — Not a standalone script; no `if __name__ == "__main__"` block or `argparse`. Imported by `build_daily_projections.py` , `simulate_innings.py` , and `api/routers/mlb.py` (pitcher-vs-batter simulation endpoint, `from mlb.simulate import run_simulation`).

Schedule — Not scheduled directly. Invoked live on-demand by the API (matchup simulator endpoint) and indirectly every time `build_daily_projections.py` runs (see below, 7:30am–11pm PT every 30 min).

`pipeline/mlb/simulate_innings.py`

Purpose — Full 9-inning game simulator that projects final score, win probability, and run totals using actual posted lineups, built on top of `simulate.py`’s `simulate_pa` .

Inputs — `mlb.game_lineups` (posted home/away lineups + probable pitcher IDs), plus everything `simulate.py`’s loader functions read (count mix, batter tendencies, Goose+3/pK+/pHR+, Juiced+2, weather).

Outputs — None persisted directly; returns a result dict to its caller (the game-companion API endpoint via `run_game_simulation`).

Key functions

- `_preload_pitcher(pitcher_id, db) -> dict` — loads count mix, fallback mix, and Goose+3/pK+/pHR+ adjustments for both batter handedness sides once, so the per-PA simulation loop makes zero DB calls.
- `_preload_batter(batter_id, db) -> dict` — same idea for one batter: bat side, pitch-type tendencies, Juiced+2 adjustment.
- `load_lineup(game_pk, side, db) -> tuple` — reads `mlb.game_lineups` , returns `(batter_id_list, opposing_pitcher_id)` .
- `resolve_base_state(outcome, on_first, on_second, on_third, outs) -> tuple` — deterministic (mostly) base-running state machine; see Formula Annotations for the probabilistic single/double advancement rules.
- `simulate_half_inning(pitcher_data, batter_cache, lineup, lineup_pos, lg_mix, weather_mult) -> tuple` — walks the batting order from `lineup_pos` , calling `simulate_pa` (with no per-PA game-state DB lookups — explicitly skipped for performance) until 3 outs, returns `(runs_scored, new_lineup_pos)` .
- `simulate_game(...) -> dict` — 9 innings, away bats top vs. home pitcher then home bats bottom vs. away pitcher, returns inning-by-inning run arrays and `home_win: bool` .
- `run_game_simulation(game_pk, db, n_sims=500) -> dict` — main entry point: preloads all data once, runs `n_sims` full-game simulations, aggregates average runs, win %, and an over/under probability table at lines {6.5, 7.5, 8.5, 9.5, 10.5}.

Formula annotations

- *Base-running advancement on a single* (`resolve_base_state` , lines 111-126): runner on 3rd always scores. Runner on 2nd scores 50% of the time (`random() < 0.5`), else advances to 3rd. Runner on 1st advances to 3rd 30% of the time, else to 2nd. These are fixed coin-flip constants, not read from any table — a simplification vs. real MLB baserunning-aggressiveness data.
- *Double*: runners on 2nd and 3rd both score automatically; runner on 1st scores 50% of the time, else stops at 3rd.
- *Triple/HR*: all runners score plus the batter (HR) or the batter reaches 3rd (triple).
- *Over/under table*: `over_pct = count(total_runs > line) / n_sims × 100` computed from the raw distribution of simulated `home_runs + away_runs` , for 5 fixed lines.
- Performance note in the docstring explicitly documents why `load_game_state_adjustments` (the base-out-state split from `simulate.py`) is *not* applied per-PA here: it would require a DB query per plate appearance, and at 500 games × ~75 PA/game that’s ~72,000 queries — too slow for a live-triggered endpoint.

CLI usage

```
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/simulate_innings.py --game-pk 745678 --n-sims 500
```

Also callable as a library: `from mlb.simulate_innings import run_game_simulation`.

Schedule — Not in the cron scheduler. Invoked live by the API's game-companion endpoint (`api/routers/mlb.py:3684-3685`, `from mlb.simulate_innings import run_game_simulation`) when a user views a game's simulation panel.

`pipeline/mlb/stuff_model.py`

Purpose — Trains a Ridge-regression model per pitch type predicting CSW% (Called Strike + Whiff rate) from pitch physics (velo, movement, spin, release-point consistency), normalized to a 100-scale “Stuff+” score. Appears to be an earlier/alternate implementation of the pitch-quality concept — the pipeline's actual daily Stuff Score table (`mlb.stuff_scores`) is built by the separate `build_stuff_score.py` script (documented elsewhere in this reference); this file is not wired into `daily_pipeline.py` or the API and looks like a standalone research/legacy script.

Inputs — `mlb.pitches` joined to `mlb.at_bats` / `mlb.games`, aggregated to one row per pitcher per pitch type per season (≥ 100 pitches).

Outputs — Pickled model files at `/pipeline/mlb/models/stuff_{PITCH_TYPE}.pkl` (`save_models` / `load_models`); no DB table is written.

Key functions

- `load_training_data(seasons, db) -> pd.DataFrame` — aggregates avg velo/movement/spin/release-point stats and CSW/whiff rates per pitcher-pitch type-season, requiring ≥ 100 pitches.
- `engineer_features(df) -> pd.DataFrame` — converts each raw feature to a z-score *within its pitch type* (e.g. `velo_z = (avg_velo - pitch-type mean) / pitch-type std`), so a 96mph fastball and an 84mph curveball are compared on equal footing. Also derives `release_consistency = 1 / (std_x0 + std_z0 + 0.01)` (lower release-point variance $\Rightarrow$ higher score $\Rightarrow$ better presumed command) and z-scores that too.
- `train_models(df) -> dict` — for each pitch type with ≥ 30 pitcher-seasons, fits `Ridge(alpha=5.0)` inside a `StandardScaler` pipeline on the 6 z-scored features to predict `csw_rate`; also runs a leave-one-season-out cross-season validation (train year N, test year N+1) for a more honest R^2 estimate before fitting on all data.
- `score_pitchers(models, df, season) -> pd.DataFrame` — `stuff_plus = predicted_csw / league_avg_csw * 100`.
- `save_models(models) / load_models() -> dict` — pickle I/O.

Formula annotations — The core model is a linear regression on standardized physics features; there's no hand-written formula for the final 100-scale score beyond `predicted_csw / league_avg_csw * 100` (the same “index vs. league average” convention used throughout this codebase's Goose+/pK+/pHR+/Juiced+ family). The `--evaluate` CLI path references `m['auc']` and `m['n_pitches']` keys that `train_models` never actually stores (it stores `cv_r2`, `n_pitcher_seasons`, etc.) — `--evaluate` would raise a `KeyError` if run against models saved by the current `train_models`, suggesting this branch is stale/untested.

CLI usage

```
PYTHONPATH=/app python3 /pipeline/mlb/stuff_model.py --train
PYTHONPATH=/app python3 /pipeline/mlb/stuff_model.py --train --season 2023 2024 2025
PYTHONPATH=/app python3 /pipeline/mlb/stuff_model.py --evaluate
PYTHONPATH=/app python3 /pipeline/mlb/stuff_model.py --score --season 2025
```

Schedule — Not scheduled; not referenced anywhere in `daily_pipeline.py` or `api/routers/mlb.py`. Appears to be run manually/ad hoc, if at all.

`pipeline/mlb/build_daily_projections.py`

Purpose — The core daily projection generator: for each probable starter today, simulates against the opposing lineup's top batters to produce K%/HR%/OPS/BB% projections, then converts those rates into projected strikeout counts (`proj_ks_6inn`, `proj_ks_f5`) using the pitcher's expected innings pitched and the opposing lineup's strikeout tendency. Also generates per-batter daily projections (AVG/OBP/SLG/OPS/HR%/K%/TB) against the day's probable pitchers.

Inputs — MLB Stats API schedule (probable pitchers + posted lineups), `mlb.boxscore_batting` / `mlb.at_bats` / `mlb.games` (top-batter selection, season K%), `mlb.boxscore_pitching` (avg IP), `mlb.player_workload` (ACWR fatigue factor), `mlb.projection_calibration` (nightly-learned calibration multipliers), `mlb.goose_overall`, `mlb.pitch_quality_scores` (BAPV+), `mlb.goose_scores` / `mlb.juiced_scores` (legacy per-bucket batter matchup), plus everything `simulate.py` reads.

Outputs — `mlb.daily_projections` (pitcher-level), `mlb.daily_batter_projections` (batter-level).

Key functions

- `get_schedule(target_date, include_final=False) -> list` — MLB API schedule call hydrated with `probablePitcher, lineups, teams`; returns per-game dicts with both teams' probable pitchers and posted lineup IDs (empty list if lineup not yet posted).
- `get_team_roster(team_id) -> list` — active non-pitcher roster IDs, used as a lineup fallback before lineups are posted.
- `get_pitcher_hand(pitcher_id, db) -> str` — most recent `pitch_hand` from `mlb.at_bats`.
- `get_top_batters(batter_ids, pitcher_hand, db) -> list` — top 8 batters (`TOP_N_BATTERS`) by 2026 PA specifically *against pitchers of this handedness* (min 20 AB); falls back to overall-2026 PA (min 10 AB, any handedness) if fewer than 4 qualify.
- `get_pitcher_avg_ip(pitcher_id, db) -> float` — average IP per start in 2026 (only rows where `games_started=1`), clamped to ≤ 9.0 , default 5.0 if insufficient data.
- `get_pitcher_season_k_pct(pitcher_id, db) -> tuple` — actual 2026 K% and batters-faced count, requires ≥ 30 BF, else `(None, 0)`.
- `get_pitcher_pa_per_inn(pitcher_id, db) -> float` — pitcher's actual PA-faced-per-inning in 2026, converting baseball-notation IP ($4.1 \rightarrow 4.0 + 1 \times 10/3 = 4.333$) to decimal innings via $\text{FLOOR}(\text{ip}) + (\text{ip} - \text{FLOOR}(\text{ip})) \times 10/3$; clamped to $[2.5, 6.0]$, default 4.1 (MLB average).
- `get_team_k_rate(team_id, db) -> float` — team-level season K rate (fallback when no batter-level lineup K rate is available), default 0.224.
- `get_lineup_k_factor(batter_ids, db, opp_team_id=None) -> tuple` — see Formula Annotations.
- `load_calibration_factors(db) -> dict` — reads the most recent `k_pct_calibration / hr_rate_multiplier` row from `mlb.projection_calibration` (written nightly by `build_calibration.py`); falls back to the hardcoded `K_PCT_CALIBRATION = 0.830` and `hr_rate_multiplier = 1.0` if the table is empty.
- `get_acwr_ip_factor(pitcher_id, db) -> float` — fatigue-based IP multiplier from the most recent ACWR value in `mlb.player_workload`: $\text{ACWR} > 1.5 \rightarrow 0.70 \times \text{IP}$, $> 1.3 \rightarrow 0.85 \times$, $< 0.8 \rightarrow 1.05 \times$, else $1.0 \times$ (a pitcher who has been worked unusually hard recently relative to his chronic load is projected for fewer innings today).
- `simulate_pitcher_vs_batters(pitcher_id, batters, db, k_calibration, hr_multiplier, opp_team_id=None) -> dict` — runs `N_SIMS_PER_BATTER=500` PA simulations per selected batter (using `pK+/pHR+` adjustments, not `Goose+3` — this call site uses `load_pk_phr_adjustments` directly rather than the `Goose+3`-first chain in `simulate.py::run_simulation`), aggregates, and applies the calibration multipliers to the raw simulated K%/HR%.
- `build_daily_projections(target_date, db, include_final=False)` — the pitcher-projection orchestrator; see Formula Annotations for the K-count conversion chain.
- `get_pitcher_primary_pitch(pitcher_id, db) / get_batter_juiced_vs_bucket(...) / simulate_batter_vs_pitcher(...) / build_daily_batter_projections(target_date, db)` — the batter-projection side: for each probable pitcher, simulates every rostered batter with ≥ 10 AB against that specific pitcher (500 sims each), storing rate stats plus a legacy “Juiced+ vs. velo/pitch-type bucket” lookup.

Formula annotations — CRITICAL, ties directly to Section 4's model-metrics questions

- **proj_k_pct** — is it per-PA or per-AB? It's **per-PA** (technically per-simulated-plate-appearance): `aggregate_outcomes` computes $\text{k_pct} = \text{strikeouts} / \text{n_sims} \times 100$ where `n_sims` counts every simulated PA including walks — i.e. the denominator is PA, not AB. This matches the correct definition of strikeout rate (K/PA), not the common K/AB error.
- **K_PCT_CALIBRATION = 0.830** (module-level fallback, line 40): a flat multiplier applied to the raw simulated K% because the Monte Carlo engine's league-average K% consistently runs hotter than real MLB K% (documented in user memory as a fixed root cause). `load_calibration_factors` overrides this fallback with a value learned nightly by `build_calibration.py` and stored in `mlb.projection_calibration`.
- **get_lineup_k_factor** (lines 252-283): computes the opposing lineup's actual 2026 K rate ($\text{SUM}(\text{strikeouts})/\text{SUM}(\text{at_bats})$ from `boxscore_batting`, min 50 AB) as a ratio to league average ($\text{LEAGUE_K_PCT} = 0.224$), clamps that ratio to $[0.80, 1.25]$, then **dampens it 50% toward 1.0**: $\text{dampened} = 1.0 + (\text{clamped_ratio} - 1.0) \times 0.5$. When no lineup is posted yet, it falls back to the *team's* season K rate via `get_team_k_rate` with the same clamp/dampen treatment. This factor is what the docstring/CLI calls “lineup_k_factor” in Section 4's question list.
- **K-count conversion chain** (lines 444-472) — this is the answer to “what does `proj_ks_6inn` represent and what are its assumptions”:
 1. Start from the simulated, calibration-scaled `proj_k_pct` (per-PA, vs. the pitcher's top-8 opposing batters).
 2. **Re-scale to this specific lineup's K tendency**: `lineup_k_calib = k_calib × clamp(lineup_raw_k / 0.224, 0.60, 1.30)`, then `proj_k_pct = proj_k_pct / k_calib × lineup_k_calib` — i.e. undo the flat calibration and reapply a lineup-specific one.
 3. **Blend with the pitcher's own real 2026 season K%** (`get_pitcher_season_k_pct`) to correct the simulation's known tendency to regress elite strikeout pitchers toward the mean: `blend_weight = min(0.60, batters_faced / 300)` (caps at 60% weight on actual stats once the pitcher has ≥ 300 BF this season), `blended_k = (1-weight) × sim_k_pct + weight × season_k_pct`.
 4. **Convert the final K% into a strikeout count**: `proj_ks_6inn = (proj_k_pct/100) × proj_ip × pa_per_inn × lineup_k`, where `proj_ip = avg_ip × acwr_ip_factor` (fatigue-adjusted expected innings) and `pa_per_inn` is the pitcher's own historical PA-faced-per-inning (not a flat 4.0 or 4.1 — pitchers who work around more traffic face more batters per inning).

Despite the name `proj_ks_6inn`, the actual innings figure used is the pitcher's *own* average IP (commonly less than 6), not a hardcoded 6 — the name appears to be a holdover label rather than a literal 6-inning assumption.

5. **F5 (first-5-innings) variant:** `f5_ip = min(proj_ip, 5.0)`, `proj_ks_f5 = (proj_k_pct/100) × f5_ip × pa_per_inn × lineup_k` — same formula, innings capped at 5.

- **`proj_ks_locked` / `proj_ks_f5_locked` / `proj_k_pct_locked`** (lines 494, 522-525, 544-546): on upsert, these three “locked” columns are only ever set to a value **when `used_posted_lineup` is true** (i.e. the real batting order was posted, not the roster fallback), and once set they're preserved on every subsequent upsert via `COALESCE(mlb.daily_projections.proj_ks_locked, EXCLUDED.proj_ks_locked)` — so the “locked” fields freeze the *first* lineup-confirmed projection of the day and never get overwritten by later re-runs (e.g. from the 30-minute recurring `run_daily_projections` schedule), while the un-suffixed `proj_ks_6inn` / `proj_k_pct` columns keep updating live all day. This is the mechanism referenced in the API/frontend for “locked at lineup post” vs. “still moving” projection display.
- **Run-scoring proxy for F5 earned runs:** `proj_er_f5 = lg_er_per_inn(0.50) × f5_ip × (proj_ops / 0.700)` — a crude linear scaling of league-average ER/9 (~4.5) by how much worse/better than league-average OPS (0.700) the pitcher is projected to allow, not an independently modeled quantity.

CLI usage

```
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/build_daily_projections.py --date 2026-07-22
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/build_daily_projections.py --pitchers-only
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/build_daily_projections.py --batters-only
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/build_daily_projections.py --include-final # retroactive/backfill
```

Schedule — `daily_pipeline.py::run_daily_projections()` — first run 7:30am PT, then every 30 minutes from 8:00am–11:00pm PT (`daily_pipeline.py` lines ~693-723). Also `run_build_today_game_projections` -adjacent hourly game-projection refreshes (9am–9pm PT) depend on this table already being populated for the day.

pipeline/mlb/build_game_projections.py

Purpose — Converts each game's two `mlb.daily_projections` rows (one per starting pitcher, keyed by which opposing lineup they face) into a projected final score and win pick, then scores yesterday's projections against actual results into `mlb.model_scores`.

Inputs — `mlb.daily_projections` (`proj_ops` per pitcher for that day's snapshot), `mlb.games` (actual scores), `mlb.game_lineups` (fallback join for pre-game/today's slate before `mlb.games` exists for that date).

Outputs — `mlb.game_projections` (creates the table itself via inline DDL), and `mlb.model_scores` rows under `player_type='game'` with metrics `run_total`, `team_runs`, `win_pick`.

Key functions

- `upsert_game_projections(game_date, db) -> int` — joins each game to both pitchers' `daily_projections` rows (matched via `opp_team_id` = the other team), computing projected runs (see Formula Annotations) and projected/actual winner; falls back to a `game_lineups`-based join when `mlb.games` doesn't have the date yet (i.e. today's pre-game slate).
- `build_game_model_scores(score_date, db) -> list` — aggregates MAE/RMSE/bias for total-runs and per-team-runs (2× sample size, treating home and away as separate observations), plus win-pick accuracy, for games with known actual results on `score_date`.
- `upsert_model_scores(rows, db)` — generic upsert into `mlb.model_scores` keyed on (`score_date`, `player_type`, `metric`).
- `build_for_date(score_date, db) -> int` — orchestrates project-then-score for one date.
- `create_tables(db)` — runs the inline DDL (idempotent `CREATE TABLE IF NOT EXISTS`).

Formula annotations

- **Runs-from-OPS conversion** (module docstring + lines 29-30, 171-177): `proj_runs_allowed_by_pitcher = pitcher's_opposing_proj_ops × OPS_SLOPE(9.0) + OPS_INTERCEPT(-0.876)`. Calibrated from a 2026-season sample where average `proj_ops` ≈ 0.595 and average actual runs ≈ 4.3/team ($0.595 \times 9.0 - 0.876 \approx 4.4$, roughly matching). Critically, **`proj_away_runs` uses the home pitcher's `proj_ops`** (i.e., the away team's projected runs = how many runs the home pitcher is projected to allow to the away lineup) and vice versa for `proj_home_runs` — the variable naming (`away_ops` / `home_ops` = that side's own pitcher's OPS-allowed) is easy to misread; the code comment at lines 174-175 explicitly calls this out.
- **Win pick:** `proj_winner = 'away' if proj_away_runs > proj_home_runs + 0.1, 'home' if the reverse, else 'push'` (anything within 0.1 runs is a toss-up, not assigned to either side). `win_correct` is only computed when both a non-push projected winner and a non-tie actual winner exist.

CLI usage

```
python build_game_projections.py # yesterday, 1 day
python build_game_projections.py --date 2025-05-20 --days 5
python build_game_projections.py --backfill # every date with daily_projections data
```

Schedule — Two distinct call sites in `daily_pipeline.py`: `run_build_game_projections()` (yesterday's date, nightly at 11:25pm PT — scoring completed games) and `run_build_today_game_projections()` (today's date, hourly 9am–9pm PT — building/refreshing today's pre-game projections as lineups post and pitcher projections update).

pipeline/mlb/build_batter_tendencies.py

Purpose — Computes the per-batter, per-pitch-type and per-zone swing/whiff/contact/power tendency tables that feed both the BAPV+ pitch-quality model and `simulate.py`'s `load_batter_tends`, plus stores the season's static wOBA linear weights.

Inputs — `mlb.pitches` joined to `mlb.at_bats` / `mlb.games`.

Outputs — `mlb.batter_pitch_type_tendencies`, `mlb.batter_zone_tendencies`, `mlb.linear_weights`.

Key functions

- `build_pitch_type_tendencies(season, db) -> int` — per batter × pitch type (min 50 pitches faced): swing rate, whiff rate (whiffs/swings), contact rate (contact/swings), chase rate (out-of-zone swings/out-of-zone pitches), CSW rate, in-play rate, hard-hit rate (EV≥95/BIP), avg exit velo, avg launch angle, barrel rate (EV≥98 & launch angle 26-30°/BIP). Deletes and fully rebuilds the season's rows each run (not an incremental upsert — `DELETE ... WHERE season = :s` then bulk insert).
- `build_zone_tendencies(season, db) -> int` — same idea keyed by `mlb.pitches.zone` (1-14) instead of pitch type, min 20 pitches per zone.
- `build_linear_weights(season, db)` — writes hardcoded FanGraphs-era wOBA linear weights (per-event run values `weight_out` / `weight_bb` / `weight_hbp` / `weight_single` / `weight_double` / `weight_triple` / `weight_hr`, plus `woba_scale` and `league_woba`) for 2023-2025; unrecognized seasons fall back to the 2024 values. These are **not derived from this codebase's own data** — they're transcribed constants from FanGraphs' published guts page.

Formula annotations — All rate stats use the standard call-code taxonomy documented in the module docstring: swing = call codes {S,W,T,F,D,E,X}, whiff = {S,W,T}, contact = {F,D,E,X}, in-zone = zone BETWEEN 1 AND 9. `barrel_rate` uses the codebase's own simplified single-window definition (EV≥98 & launch angle 26-30°) rather than Statcast's actual barrel classifier (a 2-D EV/LA sweet-spot region that widens as EV increases beyond 98).

CLI usage

```
PYTHONPATH=/app python3 /pipeline/mlb/build_batter_tendencies.py --season 2025
PYTHONPATH=/app python3 /pipeline/mlb/build_batter_tendencies.py --season 2023 2024 2025
```

Schedule — `daily_pipeline.py::run_build_batter_tendencies()` — weekly, Monday 4:00am PT (`id="batter_tendencies"`).

pipeline/mlb/build_count_mix.py

Purpose — Builds `mlb.pitcher_count_mix`: per pitcher × bat-side × count (balls-strikes) × pitch-type, the usage percentage and whiff/CSW/called-strike/ball rates and average velocity. This is the primary data source for `simulate.py::load_pitcher_count_mix`.

Inputs — `mlb.pitches` joined to `mlb.at_bats` / `mlb.games` (min 3 pitches per pitcher-bat_side-count-pitch_type group).

Outputs — `mlb.pitcher_count_mix` (upsert keyed on `pitcher_id`, `season`, `bat_side`, `balls`, `strikes`, `pitch_type_code`).

Key functions

- `build_count_mix(season, db)` — single function: aggregates via SQL, then computes `usage_pct` in pandas as `pitches / total_in_count` where `total_in_count` is the group sum over (`pitcher_id`, `bat_side`, `balls`, `strikes`) — i.e. usage percentage is normalized within each specific count, not globally.

Formula annotations — Straightforward rate stats (`whiff_rate` = whiffs/pitches where whiff = call codes {S,W,T}; `csw_rate` adds C; `called_strike_rate` = C alone; `ball_rate` = {B, *B}). No modeling or regression — this is a pure raw-aggregation script.

CLI usage

```
PYTHONPATH=/app python3 /pipeline/mlb/build_count_mix.py --season 2025 2026
```

(defaults to [2025, 2026] if `--season` omitted)

Schedule — Not present anywhere in `pipeline/daily_pipeline.py` or `api/routers/mlb.py`. Despite being load-bearing for the simulation engine's primary pitch-selection signal, it does not appear to be scheduled — it must be run manually/ad hoc (e.g., periodically during the season to pick up new pitch data). This looks like a gap worth flagging in Section 7.

`pipeline/mlb/build_context_splits.py`

Purpose — Builds four related “situational” tables used by `simulate.py`'s split-adjustment loaders: pitcher/batter performance splits by time-through-the-order/platoon/base-state/inning, player workload (ACWR fatigue ratio), and head-to-head matchup history with pre-computed regression.

Inputs — `mlb.at_bats / mlb.games` (splits, matchup history — requires `runners_reconstructed = TRUE`), `mlb.pitches` (workload — pitch counts per pitcher-game), `mlb.at_bats` (workload — PA counts per batter-game).

Outputs — `mlb.pitcher_splits`, `mlb.batter_splits`, `mlb.player_workload`, `mlb.matchup_history` (all four created via inline DDL if missing).

Key functions

- `create_tables(db)` — runs the module's inline multi-statement DDL.
- `build_pitcher_splits(seasons, db) / build_batter_splits(seasons, db)` — single big SQL statement per side using `CROSS JOIN LATERAL (VALUES ...)` to fan each at-bat out into every split bucket it belongs to (`tto_1 / tto_2 / tto_3plus`, `vs_lhh / vs_rhh` (or `vs_lhp / vs_rhp` for batters), `bases_empty`, `risp`, `early_innings` (≤ 3) / `mid_innings` (4-6) / `late_innings` (≥ 7), `overall`), then aggregates `K%/BB%/HR%/AVG/OBP/SLG/OPS` per bucket (min 5 PA). One at-bat can appear in several buckets simultaneously (e.g. `tto_2 AND risp AND mid_innings AND overall`), since the LATERAL join is one-row-per-matching-split, not mutually exclusive partitions.
- `build_player_workload(db, calc_date=None)` — see Formula Annotations.
- `build_matchup_history(seasons, db)` — see Formula Annotations.

Formula annotations

- ACWR (Acute:Chronic Workload Ratio):** `load_7d = SUM(load in last 7 days) / games_played_in_last_7_days` (average per-game load over the acute window), `load_28d = SUM(load over 28 days) / games_played_in_28_days` (average per-game load over the chronic window), `acwr = load_7d / load_28d`. Pitcher load = pitches thrown per game; batter load = plate appearances per game. An ACWR > 1 means recent per-game workload is running above the player's own longer-term baseline (this is the exact signal `build_daily_projections.py::get_acwr_ip_factor` consumes to discount a fatigued starter's projected innings).
- Matchup history regression** (lines 499-509): `k_pct_reg = raw_k_pct × (PA/(PA+20)) + 22.0 × (20/(PA+20))`, and symmetrically `hr_pct_reg = raw_hr_pct × (PA/(PA+20)) + 3.5 × (20/(PA+20))` — standard PA-weighted shrinkage toward league average (`LG_K_PCT=22.0`, `LG_HR_PCT=3.5`) with regression constant `H2H_K_REG=20`; at PA=20 the matchup gets 50% weight vs. league average, at PA=100 it gets ~83% weight. `sample_weight = PA/(PA+20)` is stored alongside for downstream callers.
- Pitcher/batter split rates:** `avg_against/avg = hits / (PA - BB)` — note the denominator approximates AB as `PA - BB` rather than a true AB count, and since sacrifice flies are not separately excluded from the underlying `valid_abs` CTE's event filter, SF at-bats are implicitly counted as AB in this denominator (a minor deviation from the strict AB definition, though SF are rare enough that the effect is small). `obp_against/obp = (H+BB)/PA` — this omits HBP and SF from both the numerator and the “PA” proxy, since the underlying event list doesn't track HBP as a distinct event type separately from other categories filtered out; see Section 4/7 for the cross-file OBP discussion.

CLI usage

```
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/build_context_splits.py --seasons 2024 2025 2026
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/build_context_splits.py --workload-only
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/build_context_splits.py --matchup-only --seasons 2024 2025 2026
```

Schedule — `daily_pipeline.py::run_build_context_splits()` — full splits+workload+matchup rebuild, nightly (part of the 6:00am PT `run_mlb_morning` sequence, tracked as “Context Splits”). `run_build_workload()` — workload-only rebuild, daily 7:00am PT (`id="workload_morning"`).

`pipeline/mlb/reconstruct_runners.py`

Purpose — Reconstructs pre-play baserunner state (`on_first / on_second / on_third / outs_before`) for every at-bat from the raw GUMBO JSON, since GUMBO's `postOn*` fields describe runner state *after* a play, not before. This is a prerequisite for all base-out-state-aware features (`simulate.py::load_game_state_adjustments`, the `risp / bases_empty` splits in `build_context_splits.py`).

Inputs — `mlb.raw_events.data` (the GUMBO JSON blob, `liveData.plays.allPlays`).

Outputs — Updates `mlb.at_bats` columns `on_first`, `on_second`, `on_third`, `outs_before`, `runners_reconstructed`.

Key functions

- `reconstruct_game(game_pk, db) -> int` — iterates `allPlays` sorted by `atBatIndex`; carries forward the *previous* play's post-play runner state as the *current* play's pre-play state, resetting to empty bases/0 outs whenever the inning or half-inning changes (or whenever outs reach 3, as a safety net). Only updates rows still flagged `runners_reconstructed = FALSE` (idempotent/incremental).
- `reconstruct_season(season, db)` — iterates every game in a season missing reconstruction, with error tolerance (continues past per-game exceptions, logs first 5).
- `validate(db)` — sanity-check query: % of at-bats with any runner on base (expected 33-38%) and average outs-before (expected ~1.0), plus a sample breakdown of Home Run frequency by base-out state.

Formula annotations — Not a statistical formula; a deterministic state-carry-forward algorithm. The one design choice worth noting: `prev_on_first = post_first` is not `None` and `bool(post_first)` treats GUMBO's `postOnFirst` (a dict describing the runner if present, absent/null if the base is empty) as a boolean presence check.

CLI usage

```
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/reconstruct_runners.py --season 2024 2025 2026
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/reconstruct_runners.py --validate
```

Schedule — `daily_pipeline.py::run_reconstruct_runners()` — part of the 6:00am PT `run_mlb_morning` sequence, tracked as "Runners", defaults to `--season 2026`.

pipeline/mlb/build_player_map.py

Purpose — Builds `mlb.player_id_map`, a crosswalk from MLBAM player IDs (used throughout GUMBO/this codebase) to Fangraphs IDs and Baseball-Reference IDs, using the `pybaseball` library's Chadwick register. Required for joining our GUMBO-derived tables to `fetch_fangraphs.py` / `fetch_bbref.py` data.

Inputs — `pybaseball.chadwick_register()` (external), `pybaseball.playerid_lookup()` (external), `mlb.at_bats` (to find GUMBO batter IDs missing from the map).

Outputs — `mlb.player_id_map` (full rebuild via `DELETE` + bulk insert, or incremental via `update_missing`).

Key functions

- `build_full_map(db) -> int` — pulls the entire Chadwick register (all professional players with a valid MLBAM or Fangraphs ID), deduplicates on MLBAM ID, clears the existing table, and bulk-inserts in batches of 5,000.
- `update_missing(db) -> int` — finds GUMBO `batter_ids` with no matching `mlbam_id` in the map, splits the GUMBO "First Last" name string, and looks each one up individually via `pybaseball.playerid_lookup(last, first)`, taking the first result if there are name collisions.

Formula annotations — None; pure ID-crosswalk ETL.

CLI usage

```
PYTHONPATH=/app python3 /pipeline/mlb/build_player_map.py # full rebuild + missing-player check
PYTHONPATH=/app python3 /pipeline/mlb/build_player_map.py --update-missing # incremental only
```

Schedule — **Not present in `daily_pipeline.py`**. Docstring says "Run this once before the season, then as needed when new players appear" — confirmed as an intentionally manual/occasional script, not part of the automated nightly/hourly cadence.

pipeline/mlb/backfill_counts.py

Purpose — Reconstructs `balls_before` / `strikes_before` for every pitch from the sequence of call codes within each at-bat (GUMBO doesn't always populate these directly), since `simulate.py::get_mix_for_count` and `build_count_mix.py` both key off count. Also the function `backfill_game` is called live by `transform.py::transform_game_pk` for every newly transformed game (per the codebase's transform pipeline), not just in bulk-backfill mode.

Inputs — `mlb.pitches` (rows where `balls_before` IS NULL).

Outputs — Updates `mlb.pitches.balls_before` / `strikes_before`.

Key functions

- `backfill_game(game_pk, db) -> int` — walks each at-bat's pitches in `pitch_number` order, maintaining running `balls / strikes` counters: ball-type call codes (`{B,*B,H,W,I,P}`) increment balls (capped at 4); strike-type codes (`{C,S,T,K,Q,R}`) increment strikes (capped at 3); foul-type codes (`{F,L,O,M,N,E,D}`) increment strikes only if `strikes < 2` (fouls with 2 strikes don't create a 3rd strike); in-play code (`X`) ends the at-bat with no further count change. Records the *pre-pitch* count for each pitch before applying that pitch's effect.
- `backfill_season(season, db)` — iterates every game in a season with any NULL `balls_before` pitch.
- `validate_sample(db)` — cross-checks reconstructed `strikes_before` against GUMBO's own `strikes_after` field for called/swinging/foul-tip strikes with `strikes_before < 2`, flagging any `strikes_before + 1 != strikes_after` mismatches.

Formula annotations — Deterministic count-reconstruction state machine (not a statistical formula); the call-code taxonomy (`BALL_CODES / STRIKE_CODES / FOUL_CODES / END_CODES`) is the key domain logic and is fully documented in the module docstring.

CLI usage

```
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/backfill_counts.py --season 2024 2025 2026
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/backfill_counts.py --season 2026 --validate
```

Schedule — Not scheduled as a standalone bulk job in `daily_pipeline.py`. Its per-game function `backfill_game(game_pk, db)` is called automatically as the last step of `transform.py::transform_game_pk` for every game transformed during the normal fetch→transform flow (confirmed via `transform.py` line ~446-448: `from mlb.backfill_counts import backfill_game`), so in practice counts are backfilled continuously as new games are ingested; the standalone `--season` CLI mode exists for catching historical gaps.

Pipeline Scripts — Models, Actuals & Scoring

Naming note (2 discrepancies from the requested script list): the user's original spec asked for `build_pk_score.py` and `build_phr_score.py` as separate files, and `build_juiced2.py` — none of these exist. In the actual repo, `pipeline/mlb/build_pk_phr.py` builds **both** pK+ and pHR+ in one script, and `pipeline/mlb/build_goose2.py` builds **both** Goose+2 and Juiced+2 in one script (the `daily_pipeline.py` wrapper function `run_build_juiced2()` just calls `build_goose2.py` a second time with the same args as `run_build_goose2()` — same script, same output, run twice under different log labels).

`pipeline/mlb/bapv_config.py`

Purpose — Central constants module for BAPV+ v2.0: base pitch values, contact thresholds, wOBA linear weights, call-code sets.

Inputs/Outputs — None; pure constants module, no DB/API access.

Key values — `BASE_WHIFF_VALUE=0.170`, `BASE_CS_VALUE=0.025`, `BASE_FOUL_VALUE=0.020`, `BASE_BALL_VALUE=-0.040`, `BASE_HBP_VALUE=-0.150`; `HARD_HIT_THRESHOLD=95` mph, `SOFT_HIT_THRESHOLD=85` mph, `HARD_HIT_OUT_MULT=0.50`, `SOFT_CONTACT_MULT=0.85`; league fallbacks (`LEAGUE_WHIFF_RATE=0.267`, `LEAGUE_CHASE_RATE=0.310`, etc.); `WOBA_WEIGHTS` dict (single 0.888, double 1.271, triple 1.616, HR 2.101, BB 0.690, HBP 0.720, out -0.098); call-code classification sets (`WHIFF_CODES`, `CS_CODES`, `FOUL_CODES`, `BALL_CODES`, `SWING_CODES`).

⚠ **Maintenance risk found while documenting:** `pipeline/mlb/compute_bapv.py` does **not** import from this module — it hardcodes its own identical copy of `BASE_WHIFF_VALUE / BASE_CS_VALUE` /etc. at the top of the file. The two sets of constants currently match exactly, but they will silently drift apart the next time someone tunes BAPV+ and only edits one of the two files. See Section 7.

Formula annotations — This file documents BAPV+ v2.0's version history in its docstring: v1.0 used equal called-strike/whiff weighting; v2.0 reduced `BASE_CS_VALUE` from 0.080→0.025, added chase-rate adjustment, added the hard-hit-out 0.5× penalty, and fixed the whiff-rate denominator to be swings-only (matching Baseball Savant's definition) rather than all pitches.

CLI usage — N/A (imported, not run directly).

Schedule — N/A (not a script; a config module some other script *should* import).

`pipeline/mlb/build_pitcher_mix.py`

Purpose — Builds each pitcher's pitch-usage/outcome profile split by opposing batter handedness (L/R) — usage%, whiff%, CSW%, velo, hard-hit% per pitch type per side.

Inputs — `mlb.pitches` joined to `mlb.at_bats / mlb.games`, filtered to regular season.

Outputs — `mlb.pitcher_mix_profile` (upsert on (`pitcher_id`, `season`, `pitch_type_code`, `bat_side`)).

Key functions — `build_pitcher_mix(season, db)`: single SQL aggregate (whiff_rate/csw_rate/avg_velo/hard_hit_rate per pitcher × pitch type × bat_side, min 5 pitches), then computes `usage_pct = pitches / total_pitches_for_that_pitcher_and_side` via a pandas `groupby().transform('sum')`, then upserts.

Formula annotations — $\text{whiff_rate} = \text{whiffs} / \text{swings}$ where $\text{whiffs} = \text{call_code in } \{S, W, T\}$ and $\text{swings} = \text{call_code in } \{S, W, T, C\}$ for CSW, or $\{S, W, T, F, D, E, X\}$ for whiff-rate proper (foul/in-play/whiff, excluding called strikes) — consistent with the whiff-rate convention used everywhere else in the codebase (Savant-style, swings-only denominator).

CLI usage

```
PYTHONPATH=/app python3 /pipeline/mlb/build_pitcher_mix.py --season 2026
```

(default `--season 2025` in argparse, but always invoked with `--season 2026` by the scheduler)

Schedule — `run_build_pitcher_mix()` runs as part of `run_mlb_morning()` at 6:00am PT (between BAPV+ and Goose+ in the chain).

pipeline/mlb/build_team_defense.py

Purpose — Computes each team's defensive quality (BABIP allowed, SLG allowed, extra-base-hit rate) from balls in play against them, for use as a context adjustment in batter projections and the inning simulator.

Inputs — `mlb.at_bats` (home team's *fielding* performance, derived from `half_inning = 'top'` rows — i.e. plate appearances where the home team was pitching/fielding) joined to `mlb.games`.

Outputs — `mlb.team_defense` (creates table if missing; upsert on `(team_abbrev, season)`), min 100 PA required per team.

Key functions — `build_team_defense(season, db) -> int`: one aggregate query, then per-team Python-side ratio computation and upsert.

Formula annotations - $\text{babip} = \text{hits} / \max(1, \text{bip} - \text{hrs})$ — standard BABIP formula (hits on balls in play, excluding HRs which aren't "in play" for BABIP purposes, over balls in play excluding HRs). - $\text{slg} = \text{total_bases} / \max(1, \text{pa_count})$ — ⚠️ this divides by PA, not AB, so it is not true SLG (true SLG = TB/AB); it's really "total bases per PA allowed," a defense-quality proxy, not a batting-stat-comparable SLG. Flagged in Section 4/7 as a naming inconsistency (not a bug per se, since it's self-consistent and only used relatively, but the column name invites confusion). - $\text{xbh_rate} = (\text{doubles} + \text{triples}) / \max(1, \text{hits})$ — extra-base-hit share of allowed hits (HR excluded from numerator, oddly, despite HR obviously being extra bases — likely intentional, isolating "defense-preventable" extra bases like doubles/triples vs. HRs which defense can't prevent). - Balls in play (`bip_count`) excludes HR/K/BB/IBB/HBP by definition (true BIP), but the query's exclusion list also removes baserunning-only rows (`Runner Out`, caught stealing, wild pitch, passed ball) that aren't real PAs at all.

CLI usage

```
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/build_team_defense.py --season 2026
```

Schedule — `run_build_team_defense()` runs twice daily: once inside the `run_mlb_morning()` chain (6:00am PT) and again standalone at 6:05am PT (`team_defense_daily` job) — the second run re-computes with the freshest data just after the 6am ingestion finishes, since the in-chain run happens before some late data may have settled.

pipeline/mlb/build_splits.py

Purpose — Contextual-splits library: batter/pitcher performance broken out by situational context (runners on/RISP/bases loaded, times through the order, inning group, close game, outs, first-pitch-strike%), each compared to a league-average baseline in the same context.

Inputs — `mlb.at_bats` joined to `mlb.games` (filtered to `runners_reconstructed = TRUE`, i.e. only games where `reconstruct_runners.py` has already backfilled base-state); `mlb.pitches` (for first-pitch-strike detection).

Outputs — None to the DB — `compute_batter_splits` / `compute_pitcher_splits` / `compute_league_splits` all **return** dicts; nothing is persisted. This is a library/CLI-report tool, not a table-builder (contrast with the similarly-named `build_context_splits.py`, covered in the simulation/projections batch, which *does* write to a table).

Key functions - `compute_batter_splits(batter_id, season, db) -> dict` — 12 named splits (bases_empty, runners_on, risp, bases_loaded, first/second/third+_time facing the pitcher, early/middle/late innings, close_game, no/one/two_outs), each computed via a `split_stats()` helper returning PA/AVG/OBP/SLG/OPS/HR%/K%/BB%. - `compute_pitcher_splits(pitcher_id, season, db) -> dict` — same context buckets from the pitcher's side (avg_against/slg_against/ops_against/k_pct/bb_pct/hr_pct), plus `first_pitch_strike_pct`. - `compute_league_splits(season, db) -> dict` — league-average AVG bases-empty vs RISP, and K% overall vs RISP, for comparison baselines.

Formula annotations - `split_stats()`'s OBP: $\text{obp} = (\text{hits} + \text{bb}) / \max(n, 1)$ where n = PA count in that split — this is **closer to correct** than several other OBP implementations found elsewhere in the codebase (see Section 4): it includes walks, though it still omits HBP and doesn't exclude sac bunts from the PA denominator the way strict OBP does. - $\text{slg} = \text{tb} / \max(\text{ab}, 1)$ where $\text{ab} = n - \text{bb}$ — this **does** divide by a proper AB proxy (PA minus walks), unlike `build_team_defense.py`'s and `build_goose2.py`'s SLG-like metrics

which divide by PA. - Times-through-the-order uses `ROW_NUMBER() OVER (PARTITION BY game_pk, pitcher_id, batter_id ORDER BY at_bat_index)` — a clean, standard way to number a batter's successive PAs against the same pitcher within one game.

CLI usage

```
python3 /pipeline/mlb/build_splits.py --season 2026 --pitcher-id 694973
python3 /pipeline/mlb/build_splits.py --season 2026 --batter-id 665742
```

Schedule — not scheduled anywhere in `daily_pipeline.py`. No `run_build_splits()` wrapper exists; this must be invoked manually as a CLI report tool. (Do not confuse with `build_context_splits.py`, which *is* scheduled and *does* persist to a table — covered in the simulation/projections section.)

pipeline/mlb/build_actuals.py

Purpose — After each night's games finish, joins yesterday's locked-in projections to the real box-score outcomes, producing one row per pitcher/batter-game in `mlb.daily_actuals` — the ground truth that `build_model_scores.py` and `build_calibration.py` grade against.

Inputs — `mlb.boxscore_pitching`, `mlb.boxscore_batting`, `mlb.games`, `mlb.at_bats` (for name lookups), `mlb.daily_projections`, `mlb.daily_batter_projections`, `mlb.kalshi_markets`.

Outputs — `mlb.daily_actuals` (upsert on `(game_date, game_pk, player_id, player_type)`).

Key functions - `build_pitcher_actuals(game_date, db)` — one big `INSERT...SELECT` joining `boxscore_pitching` (filtered to `games_started = 1`, i.e. starters only) to that day's `daily_projections` snapshot and the closest Kalshi K-line (by `ABS(implied_prob - 0.5)`), i.e. picks the market's most "coin-flip" line as the reference line). Captures `actual_k`, `actual_ip`, `actual_er`, `actual_bb`, `actual_hits_allowed`, `actual_pitches`, plus the frozen `proj_k_pct` / `proj_hr_pct` / `proj_ops` / `proj_ks_6inn` / `proj_ks_locked` for later scoring. - `build_batter_actuals(game_date, db)` — same pattern for `boxscore_batting` (filtered to `at_bats >= 2`), joined to `daily_batter_projections` (picks the projection row with the most `simulations` if there are multiple `bat_side` variants). - `build_actuals(game_date, db)` — orchestrates both, commits once.

Formula annotations - `actual_ip` = `bp.innings_pitched` copied straight from `mlb.boxscore_pitching.innings_pitched` — ⚠ this column is itself populated by `transform.py`'s naive `float()` cast of MLB's baseball-notation IP string (e.g. "6.1" meaning 6⅓ innings, misread as 6.10 decimal innings). This bug propagates from raw box score → `boxscore_pitching` → `daily_actuals.actual_ip` → every downstream ER-rate/IP calculation (`build_model_scores.py`'s ER proxy, `build_calibration.py`'s K-rate-per-BF diagnosis). See Section 4/7 — this is the single most consequential formula bug in the pipeline because of how many things read `actual_ip`. - `proj_tb` = `dbp.proj_slg * 4` — explicitly commented `-- rough TB proxy`; not a real total-bases projection, just SLG scaled by a flat factor of 4 (implicitly assuming every batted ball that becomes a hit is a double, since SLG's max meaningful per-AB value is 4 for a HR — this is a crude linear proxy, not a distributional projection). - The Kalshi-line join picks `ORDER BY ABS(implied_prob - 0.5) LIMIT 1` — i.e. among all strike-count lines offered for that pitcher/game, it stores the one closest to a 50/50 market, treating that as "the line" for later reference.

CLI usage

```
python3 /pipeline/mlb/build_actuals.py --date 2026-07-21
# defaults to yesterday if --date omitted
```

Schedule — `run_build_actuals()` runs nightly at 11:15pm PT (`actuals_nightly`), first in the overnight scoring chain (before `model_scores` → `calibration` → `game_projections` → final status update).

pipeline/mlb/build_model_scores.py

Purpose — Grades yesterday's (or a range of dates') projections against `mlb.daily_actuals`, producing MAE/RMSE/bias/within-N-accuracy metrics per player type × metric, for both pitchers (K count, K rate, ER proxy, IP distribution) and batters (TB, K rate, HR Brier score).

Inputs — `mlb.daily_actuals`, `mlb.daily_batter_projections` (for batter K-rate context).

Outputs — `mlb.model_scores` (creates table if missing; upsert on `(score_date, player_type, metric)`).

Key functions - `build_pitcher_scores(score_date, db) -> list` — one CTE (`pd`) computing `proj_k_count`, `proj_er`, `actual_k_rate` per pitcher-game, then an aggregate query producing 3–4 metric rows: `k_count` (MAE/RMSE/bias/within-1/within-2), `k_rate` (MAE/bias only), `ip_dist` (mean/stddev of actual IP, no projection — baseline tracking), and `er_proxy` (MAE/RMSE/bias, only if any rows have both actual and proj ER). - `build_batter_scores(score_date, db) -> list` — CTE (`bd`) joining `daily_actuals` to `daily_batter_projections` for K-rate context, producing `tb`, `k_rate`, and `hr_brier` metric rows. - `upsert_scores(rows, db)` — bulk upsert into `mlb.model_scores`. - `build_scores_for_date(score_date, db) -> int` — orchestrates both, one date at a time.

Formula annotations - proj_k_count fallback chain: `COALESCE(proj_ks_locked, proj_ks_6inn, proj_k_pct/100 * 21)` — prefers the pre-game locked K projection, falls back to the (potentially-updated) 6-inning projection, and as a last resort (for legacy rows from before 2026-06-02 per the code comment) derives a K count from the rate assuming 21 batters faced. - **ER proxy:** `proj_er = proj_ops * 4.5 * actual_ip / 9.0` — converts a projected OPS into a rough ERA via the well-known sabermetric rule of thumb “ERA ≈ OPS × 4.5” (approximate, not derived from first principles), then scales that ERA to the pitcher’s *actual* innings pitched to get a per-outing ER count. Note this uses `actual_ip`, so it inherits the baseball-notation IP bug described above. - **actual K rate:** `actual_k / (actual_ip * 3.3) * 100` — approximates batters faced as `IP × 3.3` (the file’s documented “MLB average batters faced per inning”) rather than using the real BF count (which does exist in `boxscore_pitching` — this is a proxy, not the exact figure), and again is denominated in the possibly-mis-parsed `actual_ip`. - **HR Brier score:** converts `proj_hr_pct` (a per-PA rate) into a per-game probability via `1 - (1 - hr_pct/100)^3.8`, assuming 3.8 PA/game, then scores it against the binary `hr_binary` (did this batter hit ≥1 HR that game) using the standard Brier score formula `mean((forecast_prob - outcome)^2)` — lower is better, 0 = perfect. - **within_1/within_2:** fraction of games where `|actual - proj| ≤ 1` or `≤ 2`, expressed as a percentage — a calibration-style “hit rate” metric alongside MAE/RMSE.

CLI usage

```
python build_model_scores.py --date 2026-07-21
python build_model_scores.py --date 2026-07-21 --days 7 # score last 7 days ending at --date
python build_model_scores.py --backfill # score every date with actuals not yet scored
```

Schedule — `run_build_model_scores()` runs nightly at 11:20pm PT (`model_scores_nightly`), right after `build_actuals`.

pipeline/mlb/build_calibration.py

Purpose — The pipeline’s “self-learning” step: reads the last 14 days of `mlb.model_scores` bias, and nudges two global correction factors (`k_pct_calibration`, `hr_rate_multiplier`) toward the observed proj/actual ratio, so tomorrow’s projections auto-correct for recent systematic over/under-projection. Also runs a diagnostic that decomposes K misses into “pitchers going longer/shorter than expected” vs. “K rate itself being wrong.”

Inputs — `mlb.model_scores` (14-day rolling window by default), `mlb.daily_actuals` (for the miss-decomposition diagnostic), `mlb.projection_calibration` (reads the current/most-recent factor before computing the next one).

Outputs — `mlb.projection_calibration` (creates table if missing; **append-only**, PRIMARY KEY (`computed_at`) — every run inserts a new timestamped row rather than upserting one row per day, so this table accumulates a full history of calibration drift over time). Downstream, `build_daily_projections.py` reads the latest row’s `k_pct_calibration` / `hr_rate_multiplier` to scale that day’s simulator output.

Key functions - `get_current_k_calibration(db)` / `get_current_hr_multiplier(db)` — read the most recent stored factor, or the hardcoded base (`K_CALIB_BASE=0.830`, `HR_MULT_BASE=1.00`) if none exists yet. - `compute_k_calibration(lookback, db) -> dict` — averages `mean_proj / mean_actual` from the `k_count` metric over the lookback window (requires ≥3 scored days and ≥5 players/day to count), computes `bias_pct = (avg_proj - avg_actual) / avg_proj * 100`; if `|bias_pct| < 10%` (noise guard), keeps the current factor unchanged; otherwise nudges it. - `compute_hr_multiplier(lookback, db) -> dict` — same pattern using the `hr_brier` metric’s `mean_proj / mean_actual`, noise-guarded at `|ratio - 1| < 3%`. - `get_ip_context(lookback, db) -> dict` — reports average actual IP vs. the 5.0-inning baseline assumption (context only, doesn’t feed the multiplier). - `diagnose_k_miss(lookback, db)` — decomposes the average per-start K miss into an **IP contribution** (`(actual_ip - 5.0) * proj_k_pct/100 * 3.3`, i.e. extra/fewer Ks purely from pitching longer/shorter than the 5-inning baseline) and a **rate contribution** (`actual_ip * 3.3 * (actual_k_rate - proj_k_pct/100)`, i.e. the K-rate model itself being off at the innings actually thrown), then prints which one is the larger driver. - `diagnose_top_k_misses / diagnose_top_hr_misses` — print the 5 largest individual misses in the window, for manual inspection. - `build_calibration(lookback, dry_run, db)` — orchestrates all of the above and writes the new row (unless `--dry-run`).

Formula annotations - Calibration update rule (identical shape for both K and HR): `new_factor = current * (1.0 + LEARNING_RATE * (observed_ratio - 1.0))` where `observed_ratio = avg_actual / avg_proj` and `LEARNING_RATE = 0.30` — this is an exponential-moving-average-style correction: it moves 30% of the way toward “what would have made yesterday’s projections exactly right,” not all the way, to avoid over-correcting on noisy single-run signal. Clamped to `[0.60, 1.40]` for K, `[0.70, 1.40]` for HR. - **BF_PER_IP = 3.3** and **IP_BASELINE = 5.0** are hardcoded empirical constants (documented as “MLB average batters faced per inning” and “baseline projected IP used when pitcher history is thin”) used throughout the miss-decomposition math — not derived from the DB, just fixed assumptions. - This file is the authoritative source for **K_PCT_CALIBRATION**, referenced in the module’s own docstring: “The simulator produces raw K% ~27%; **K_PCT_CALIBRATION** (currently 0.830) scales it to match the league ~22.4%.” — i.e. the base multiplier exists to correct a structural over-projection baked into the Monte Carlo simulator itself (see `simulate.py` in the simulation/projections section), and this script’s job is to keep that correction factor tracking reality over time rather than staying frozen at 0.830 forever.

CLI usage

```
python build_calibration.py --lookback 14
python build_calibration.py --lookback 30 --dry-run
```

Schedule — `run_build_calibration()` runs nightly at 11:22pm PT (`calibration_nightly`), right after `build_model_scores` , and also inside the `run_mlb_morning()` 6am chain (near the end, after Model Scores).

`pipeline/mlb/compute_bapv.py`

Purpose — Computes BAPV+ (Batter-Adjusted Pitch Value) per pitcher × pitch type × game: scores every individual pitch by outcome (whiff/called-strike/foul/ball/in-play/HBP), adjusted for the specific batter's known tendencies (a whiff against a contact hitter counts for more than a whiff against a free swinger), then normalizes to 100 = league average per pitch type.

Inputs — `mlb.pitches` + `mlb.at_bats` + `mlb.games`; `mlb.batter_pitch_type_tendencies` and `mlb.batter_zone_tendencies` (batter-specific whiff/chase/take/hard-hit rates by pitch type and zone, built by `build_batter_tendencies.py`); `mlb.linear_weights` (season wOBA-style event weights, falls back to 2024 if the target season isn't populated yet).

Outputs — `mlb.pitch_quality_scores` (full delete + `bulk_save_objects` insert, scoped to whichever of `--game-pk` / `--start .. --end` / `--season` was passed — not a true upsert, but functionally equivalent since it deletes the matching scope first).

Key functions - `load_tendencies(season, db) -> tuple` — loads the batter tendency lookup tables and linear weights for a season, plus league-average fallback rates. - `load_pitches(season, db, start_date=None, end_date=None, game_pk=None) -> DataFrame` — loads raw pitch rows with at-bat/game context for the requested scope. - `compute_bapv_vectorized(pitches, type_lookup, zone_lookup, weights, league_avg) -> DataFrame` — the core scoring function (see Formula annotations). - `aggregate_per_game(pitches) -> DataFrame` — groups to pitcher × pitch_type × game, computing `avg_bapv` , `whiff_rate` , `cs_rate` , `csw_rate` , `hard_hit_rate` , and average physical characteristics. - `normalize_bapv_plus(agg, league_avg_bapv) -> DataFrame` — converts raw average BAPV to a 100-indexed “BAPV+” score per row (needs ≥3 same-pitch-type rows in the current aggregation batch to use an in-batch average; otherwise falls back to the global league average). - `store_scores` , `show_leaderboard` , `show_pitcher_profile` — persistence and CLI display helpers.

Formula annotations - Batter whiff/chase adjustment: `whiff_adj = 1.0 + max(0, league_avg_whiff_rate - batter_whiff_rate)` — only *increases* the whiff's value when the batter's own whiff rate is *below* league average (i.e. they're a tough hitter to whiff); if the batter is a free-swinger with an above-average whiff rate, the adjustment floors at 1.0 (whiffing a free-swinger gets no bonus, but also no penalty). Same clipped-at-zero pattern for `cs_adj` (called-strike value scaled by how far below league-average the batter's chase rate is — patient hitters taking a strike is more impressive) and `hard_hit_surprise` (contact quality relative to that batter's typical hard-hit rate). - **Per-pitch BAPV value:** `np.select` over the outcome flags — whiff → `BASE_WHIFF_VALUE(0.170) * whiff_adj` ; called strike → `BASE_CS_VALUE(0.025) * cs_adj` ; foul → flat `0.020` ; ball → flat `-0.040` ; in-play → `-adj_event_weight` (negative of the batter-adjusted linear-weights value of the actual outcome — a positive outcome for the batter like a double is a negative BAPV for the pitcher); HBP → flat `-0.150` . - **In-play event weight:** base linear weight per outcome type (single 0.888, double 1.271, etc., matching `bapv_config.WOBA_WEIGHTS` , though again note this file hardcodes its own copy rather than importing them) times a `hard_hit_surprise` multiplier: full penalty (`hard_hit_surprise` computed from the batter's own hard-hit-rate baseline) for a hard-hit hit, flat `0.5` for a hard-hit out (bad process, good result for the pitcher — half credit), flat `0.85` for soft-contact outs (slight bonus), `1.0` otherwise. - **BAPV+ normalization:** `bapv_plus = avg_bapv / abs(type_avg) * 100` , where `type_avg` is the mean BAPV for that pitch type across the current aggregation batch (if ≥3 pitcher-game rows exist for that type) or else the overall league-average BAPV per pitch — so 100 = league average for that specific pitch type, consistent with every other “+”-suffixed metric in this codebase.

CLI usage

```
PYTHONPATH=/app python3 /pipeline/mlb/compute_bapv.py --season 2025
PYTHONPATH=/app python3 /pipeline/mlb/compute_bapv.py --season 2025 --start 2025-04-01 --end 2025-04-30
PYTHONPATH=/app python3 /pipeline/mlb/compute_bapv.py --game-pk 745528
PYTHONPATH=/app python3 /pipeline/mlb/compute_bapv.py --season 2025 --leaderboard
PYTHONPATH=/app python3 /pipeline/mlb/compute_bapv.py --season 2025 --pitcher 694973
```

Schedule — `run_compute_bapv()` (hardcoded `--season 2026`) runs as part of `run_mlb_morning()` at 6:00am PT.

`pipeline/mlb/build_stuff_score.py`

Purpose — “Stuff Score”: a physics-first pitch-quality model combining (a) a **physical score** from velo/spin/movement weighted by each characteristic's empirically-measured correlation with whiff/CSW/SLG outcomes per pitch type, (b) a **location score** from zone-distribution quality, (c) a **tunnel score** from movement similarity/deceptiveness vs. the pitcher's own fastball, and (d) a batter-quality-weighted whiff-rate component — blended into one 100-indexed number per pitcher × pitch type, then aggregated to one overall Stuff Score per pitcher.

Inputs — `mlb.pitches` + `mlb.at_bats` + `mlb.games` (velo/spin/movement/call codes); writes and reads back its own `mlb.zone_values` table (per pitch-type × zone outcome quality) as an intermediate.

Outputs — `mlb.zone_values` (baseline zone-quality lookup, upsert on (`baseline_season` , `pitch_type_code` , `zone`)), `mlb.stuff_scores` (per pitcher × pitch type, upsert on (`pitcher_id` , `season` , `pitch_type_code`)), `mlb.stuff_overall` (per pitcher,

usage-weighted, upsert on (pitcher_id, season).

Key functions - `compute_batter_whiff_weights(season, db) -> dict` — same batter-quality weighting concept as BAPV+: `weight = min(LEAGUE_AVG_WHIFF / batter_whiff_rate, 5.0)`, capped at 5x, min 50 competitive swings. - `compute_zone_values(baseline_season, db) -> DataFrame` — league-wide outcome quality (batter-weighted whiff%, CSW%, SLG-against) per pitch type x zone (1–14, i.e. the 9 strike-zone cells plus 5 “ball” zones), combined into one `zone_value` per cell. - `compute_league_physical_avgs`, `compute_league_zone_avg`, `compute_league_zone_std` — league mean/stddev of physical characteristics and zone quality, aggregated to the *pitcher* level first (so std reflects pitcher-to-pitcher variance, not pitch-to-pitch noise). - `compute_physical_score(pitcher_vals, league_avgs, pitch_type) -> float` — z-score-normalizes velo/IVB/hbreak/spin against league mean/std, combines them with the `EMPIRICAL_WEIGHTS` correlation table (see below), scales to 100±. - `compute_location_score(...) -> tuple` — usage-weighted average zone value for that pitcher’s actual pitch distribution, z-scored against the league. - `compute_tunnel_score(...) -> float` — for fastball-family pitches (FF/SI/FC), scores based on release-point consistency (lower std of x_0 / z_0 = more deceptive = better, inverted z-score); for offspeed/breaking pitches, scores based on movement **divergence** from the pitcher’s own fastball (bigger divergence from FF/SI movement = better tunnel/deception, z-scored against the league’s average divergence for that pitch type). - `build_stuff_scores(score_season, baseline_season, db)` — orchestrates all of the above into the final composite per pitcher x pitch type, then a usage-weighted overall score per pitcher. - `validate_stuff_scores(season, db)` — prints Pearson-r correlations of Stuff Score against K rate/HR rate/ERA+/Goose+1/Goose+2 as a sanity check, plus a top-15 leaderboard.

Formula annotations - EMPIRICAL_WEIGHTS: a hardcoded table of Pearson-r correlation coefficients between each physical characteristic (velo/IVB/hbreak/spin) and each outcome (whiff/CSW/SLG), separately fit per pitch type (FF, SI, SL, CH, CU, FC, ST). For SLG specifically, the raw correlations are stored as-is (negative r = higher characteristic → lower SLG → good for pitcher) and the code multiplies by `sign = -1.0` when combining, so that “positive contribution” consistently means “better for the pitcher” across all three outcome types. - **Physical score**: `composite_z = Σ_outcome[ o_weight * sign * Σ_char(normalized_weight[char] * z_score[char]) ]` across the three outcomes (`OUTCOME_WEIGHTS = {whiff: 0.40, slg: 0.35, csw: 0.25}`), where each outcome’s per-characteristic weights are normalized so their absolute values sum to 1 (`normalize_weights`). Final: `score = 100 + composite_z * PHYSICAL_SCALE(18.0)`, clamped to `[50, 160]`. - **Location score**: `z_loc = (usage_weighted_zone_value - league_avg_zone_value) / league_std_zone_value`; `score = 100 + z * LOCATION_SCALE(18.0)`, clamped `[50, 160]`. - **Tunnel score**: dampened by 0.5× relative to the physical/location scale (`score = 100 + z * SCALE(18.0) * 0.5`), clamped to a narrower `[70, 130]` band — the file’s comments indicate this is intentional, tunnel is treated as the least statistically reliable of the three components. - **Weighted-whiff score**: separately z-scored batter-quality-weighted whiff rate vs. league, same `100 + z*18` scaling, clamped `[50, 160]`. - **Final composite**: `stuff_raw = whiff_score*0.45 + physical*0.35 + location*0.15 + tunnel*0.05` (`COMPONENT_WEIGHTS`), then regressed to the mean: `regress_to_mean(score, n) = 100 + (n/(n+200)) * (score - 100)` — a pitcher with exactly 200 pitches of data gets pulled halfway back to 100; more pitches = more trust in the raw score, matching the same `REGRESSION_K=200` shrinkage pattern used in BAPV+, Goose+2, and pK+/pHR+. - Overall per-pitcher Stuff Score is the **pitch-count-weighted average** of the per-pitch-type scores, and `top_pitch_type / top_pitch_stuff` record whichever single pitch type scored highest.

CLI usage

```
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/build_stuff_score.py --baseline-season 2025 --score-season 2026
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/build_stuff_score.py --score-season 2026 --skip-zones
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/build_stuff_score.py --score-season 2026 --validate-only
```

Schedule — `run_build_stuff_score()` (`--score-season 2026`, using default `--baseline-season 2025`) runs as part of `run_mlb_morning()` at 6:00am PT, after Runners and before pK+/pHR+.

pipeline/mlb/build_goose.py

Purpose — The original (“v1”) Goose+ pitcher pitch-quality model and its batter counterpart Juiced+: buckets every pitch type into velocity quintiles from 2 seasons of historical data, scores each pitcher’s physical characteristics + outcomes relative to those buckets, and scores each batter’s OPS against the bucket of pitcher quality they actually faced.

Inputs — `mlb.pitches / mlb.at_bats / mlb.games` for 2024+2025 (`HISTORICAL_SEASONS`) to build quintile boundaries and bucket outcome stats; current-season data to score pitchers/batters.

Outputs — `mlb.goose_pitch_boundaries` (quintile cutpoints per pitch type x characteristic, upsert), `mlb.goose_bucket_outcomes` (league outcome stats per pitch type x velo quintile), `mlb.goose_scores` (per pitcher x pitch type, includes platoon-split `goose_plus_vs_lhh / goose_plus_vs_rhh`), `mlb.goose_overall`, `mlb.juiced_scores` (per batter x pitch type x velo quintile faced).

Key functions - `compute_boundaries(db)` — Step 1: computes q20/q40/q60/q80 percentile cutpoints (plus mean/std) per pitch type for velo/spin/hbreak/vbreak, from 2024+2025 combined. - `get_quintile(value, boundaries) -> int` / `get_percentile_score(value, boundaries) -> float` — maps a raw value to a 1–5 quintile bucket, or to a continuous 0–100 percentile score via piecewise-linear interpolation between the stored breakpoints (with linear extrapolation beyond q20/q80 using the stored std dev). - `compute_bucket_outcomes(db, bounds)` — Step 2: for each pitch type x velo quintile, computes league strike%/whiff%/pitch-OPS/pitch-SLG outcome baselines (min 50 pitches per bucket). - `compute_goose_scores(db, bounds, season)` — Step 3: for each pitcher x pitch type (aggregated across bat_side, with separate LHH/RHH variants computed too), combines a **physical_score**

(percentile-based, not z-score-based like Stuff Score) with an **outcome_score** relative to that pitch's velo-quintile bucket. - `compute_overall_goose(db, season)` — Step 4: usage-weighted overall Goose+ per pitcher. - `compute_juiced_scores(db, season)` — Step 5: Juiced+ batter scores (see Formula annotations — contains an OBP bug).

Formula annotations - physical_score = `velo_pct*0.35 + spin_pct*0.20 + hbreak_pct*0.15 + vbreak_pct*0.15` (weights sum to 0.85, not 1.0 — the remaining 0.15 is implicitly absorbed into the outcome blend below, not a bug per se but worth noting the physical sub-weights alone don't normalize to 1). - **outcome_score** = `(strike_rel*0.30 + whiff_rel*0.40 + ops_rel*0.30) * 50`, where `strike_rel = pitcher_strike_pct / league_strike_pct`, `whiff_rel = pitcher_whiff_pct / LEAGUE_AVG_BATTER_WHIFF (0.267, a fixed constant, not the bucket-specific league average)`, and `ops_rel = max(0.5, 2.0 - pitch_ops/league_pitch_ops)` — an inverted ratio so that a pitcher allowing lower OPS scores higher, floored at 0.5 to prevent runaway values for extremely bad OPS-against. - **Final Goose+**: `raw_score = physical_score*0.85 + outcome_score*0.15`; `goose_plus = raw_score * 2.0` — the x2 scaling exists because `physical_score / outcome_score` are both roughly centered on a 0–100ish percentile scale, and the final Goose+ metric is meant to read on the same “100 = average” scale as every other model here (so a raw composite of ~50 maps to ~100). - **Juiced+ OBP bug**: `obp = hits / pa` (line ~666) — this treats *only hits* as “on base,” entirely omitting walks and HBP from the numerator, despite the query's own denominator (`pa`) correctly counting all plate appearances. A batter who walks a lot in this specific pitch-type/velo-quintile bucket will show an artificially depressed `obp` here, and thus `ops = obp + slg` and the final `juiced_plus = 100 * ops/league_ops` will understate that batter's Juiced+ score. This is a genuine formula bug (not just a differently-scoped stat), and it's the oldest of several inconsistent OBP implementations found across the codebase during this review — see Section 4. - **Juiced+ scaling**: `juiced_plus = 100 * (ops / league_ops_for_that_bucket)` — straightforward ratio-to-100, same convention as everywhere else, but built on top of the broken `obp` above.

CLI usage

```
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/build_goose.py --season 2026
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/build_goose.py --season 2026 --skip-boundaries --skip-buckets
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/build_goose.py --season 2026 --juiced-only
```

Schedule — `run_build_goose()` runs `["--season", "2026", "--skip-boundaries", "--skip-buckets"]` as part of `run_mlb_morning()` at 6:00am PT — i.e. the daily scheduled run **always skips re-computing the historical quintile boundaries and bucket outcomes** (steps 1–2), only re-running steps 3–5 (scoring) against whatever boundaries/buckets were last computed. Boundaries/buckets must be manually rebuilt (without the skip flags) if the 2024+2025 historical pitch data changes or a full recalibration is wanted.

pipeline/mlb/build_goose2.py

Purpose — Goose+2 (pitcher) and Juiced+2 (batter): a rolling-baseline redesign of the v1 model — instead of static historical quintile buckets, league baselines blend current-season and prior-season data by data volume, and pitcher/batter scores are 6-component weighted composites indexed directly against those baselines (no percentile/quintile bucketing).

Inputs — `mlb.pitches / mlb.at_bats / mlb.games`; `mlb.bat_tracking` (bat speed/squared-up%/blast-rate, for Juiced+2's hard-hit component when available).

Outputs — League baseline tables (blended current+prior), `mlb.goose2_scores / mlb.goose2_overall` (pitcher), `mlb.juiced2_scores` (batter).

Key functions - `get_season_weights(db, season) -> tuple` — see Formula annotations (pitch-volume-based current/prior blend weight). - `get_blended_baselines(score_season, db) -> dict` — computes league averages per pitch type × bat_side × pitch_hand, blended `w_current*current_season_baseline + w_prior*prior_season_baseline`. - `compute_league_baselines, compute_slg_baseline, store_baselines` — build and persist the raw single-season baseline components that get blended above. - `build_goose2(score_season, baseline_season, db)` — scores each pitcher × pitch type × bat_side against the blended baseline on 6 components (whiff/SLG/CSW/contact/chase/hard-hit), regresses to mean, aggregates to an overall usage-weighted score. - `build_juiced2(score_season, db)` — see Formula annotations for the “quality adjustment” concept (batter performance relative to the *quality of pitching actually faced*, not just raw league average). - `validate_goose2, validate_juiced2, store_2026_baselines_snapshot` — validation/reporting and a season-baseline snapshot utility.

Formula annotations - Season blend weight (`get_season_weights`): <50,000 pitches → 30% current / 70% prior season; >150,000 pitches → 100% current / 0% prior; linear interpolation between `(0.30 + 0.70*(n-50000)/100000)` in between — early in a season, baselines lean heavily on last year's league norms; by ~mid-season (150k+ league-wide pitches), baselines are purely current-year. -

GOOSE2_WEIGHTS: {whiff: 0.30, slg: 0.30, csw: 0.15, contact: 0.10, chase: 0.10, hard_hit: 0.05} — each component converted to a `safe_idx()` ratio-to-baseline (100 = league average), with `invert=True` used for “lower is better for the pitcher” metrics (contact%, SLG-against, hard-hit%) so the index direction is consistent (higher index = better pitcher) across all 6 components. -

goose2_raw = weighted sum of the 6 indices; final `goose2_plus = regress_to_mean(goose2_raw, pitches)` with `REGRESSION_K=200` (same shrinkage constant as Stuff Score and `pK+/pHR+`). - **Juiced+2 “quality adjustment”** (`JUICED2_WEIGHTS = {quality_adj: 0.40, slg: 0.28, ops: 0.22, hard_hit: 0.10}`), with the docstring noting these weights were chosen from the models' own correlation to actual outcomes: `r=0.480` for `quality_adj` vs OPS, `0.327` for SLG, `0.288` for OPS, `0.161` for hard-hit: `expected_slg = league_avg_slg * (100 / avg_goose2_of_pitchers_faced)` — a batter who faced tougher-than-average pitching (Goose+2 > 100) gets a *lower* expected

SLG bar to clear; `quality_idx = 100 * (actual_slg / expected_slg)` — so a batter posting average SLG against elite pitching scores above 100 on this component, correcting for strength of pitching faced. - ⚠️ **Non-standard SLG/OBP labels:** the `batter_pa` CTE computes `slg = AVG(bases_per_PA)` (divides total bases by **PA**, not AB — see the note under `build_team_defense.py`; same pattern, same caveat) and `obp = AVG(is_on_base_flag)` where the on-base flag correctly includes hits + BB/IBB + HBP over PA — this OBP formula is actually **correct enough** (missing only SF from the denominator, a minor deviation), in contrast to `build_goose.py`'s Juiced+ v1 OBP bug above. Worth flagging in Section 4 as an example of the same codebase getting the same stat right in one model and wrong in another. - **Hard-hit index** (Juiced+2): when bat-tracking data is available, a 3-component composite — `bat_speed/lg_bat_speed*100*0.50 + squared_up%/lg_squared_up%*100*0.30 + blast_rate/lg_blast_rate*100*0.20`; falls back to an HR-rate proxy (`hr_rate/league_hr_rate*100`) when a batter has no bat-tracking data, tagging the source (`bat_tracking` vs `hr_proxy`) in the stored row.

CLI usage

```
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/build_goose2.py --score-season 2026 --baseline-season 2025
```

Schedule — `run_build_goose2()` and `run_build_juiced2()` both call this exact same script with identical args (`--score-season 2026 --baseline-season 2025`) as part of `run_mlb_morning()` at 6:00am PT — the script computes both Goose+2 and Juiced+2 in a single run, so `run_build_juiced2()` is a redundant second invocation logged under a different name (harmless, just wasted compute — the second run recomputes and re-upserts identical output).

pipeline/mlb/build_goose3.py

Purpose — The “unified” pitcher quality model: Bayesian-style blend of Stuff Score (physics-based prior — what *should* happen given a pitcher’s raw stuff) with the average of pK+ and pHR+ (observed-outcomes likelihood — what *has* happened), weighted by sample size so that pitchers with little current-season data lean on their physical profile and pitchers with a lot of data lean on their actual results.

Inputs — `mlb.stuff_scores`, `mlb.pk_scores`, `mlb.phr_scores` (all pre-computed by their respective builder scripts — this script does no raw pitch/at-bat querying itself, it’s purely a combiner).

Outputs — `mlb.goose3_scores` (per pitcher × pitch type × bat_side), `mlb.goose3_overall` (per pitcher, with LHH/RHH split).

Key functions - `get_outcomes_weight(pitches) -> float` — see Formula annotations. - `build_goose3(score_season, db)` — for every (`pitcher_id`, `pitch_type`, `bat_side`) key present in either the pK+ or pHR+ maps, looks up the matching Stuff Score (keyed only by pitcher+pitch_type, no bat_side split — Stuff Score doesn’t split by opposing batter side), blends, regresses, and stores; also computes an overall usage-weighted composite with LHH/RHH breakdowns. - `validate_goose3(season, db)` — correlates Goose+3 (and each of its inputs — Stuff, pK+, pHR+, Goose+1) against actual K rate/HR rate/ERA+ as a sanity check, and prints the biggest “gap” pitchers (where outcomes and Stuff strongly disagree — flagged as likely regression/improvement candidates).

Formula annotations - **Outcomes weight ramp:** `outcomes_weight = min(1.0, pitches/300) * 0.65` — at 0 pitches, 0% outcomes weight (pure Stuff Score); reaches the 0.65 ceiling once a pitcher has thrown 300+ pitches of that pitch type in the current season; `stuff_weight = 1 - outcomes_weight` always. So **outcomes never fully dominate** — even at massive sample size, Stuff Score retains at least 35% weight (`1 - 0.65`), per the file’s own docstring table (300 pitches → 54/46 split; 1200+ pitches → 35/65 split, the asymptotic floor). - **outcome_score** = simple average of `pk_plus` and `phr_plus` (not weighted by their own sample sizes or correlation strength — a flat 50/50 blend of the strikeout model and the home-run model). - `goose3_raw = stuff_score * stuff_weight + outcome_score * outcomes_weight`, then `regress_to_mean(goose3_raw, pitches)` (same `k=200` shrinkage as the other models) — meaning there are technically **two** separate regression-to-mean effects stacked here: the sample-size-based blend weight itself already discounts thin-sample outcome data, and then the final composite is *again* pulled toward 100 based on total pitch count. This double-shrinkage is intentional per the architecture described in the docstring, but worth flagging in Section 8 as a place where “just tune the weight” isn’t a single-constant change — there are two independent shrinkage mechanisms to consider. - **stuff_outcomes_gap** = `outcome_score - stuff_score` — a diagnostic (not used in the final score) showing which pitchers are over/under-performing their raw stuff, used by `validate_goose3`’s “likely to regress” / “likely to improve” lists. - Missing pK+ or pHR+ data defaults that component to `100.0` (the anchor pitcher record — whichever of `pk_data`/`phr_data` exists — is used for name/hand/pitch-count metadata even if the other model has no row for that key).

CLI usage

```
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/build_goose3.py --score-season 2026
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/build_goose3.py --score-season 2026 --validate-only
```

Schedule — `run_build_goose3()` (`--score-season 2026`) runs as part of `run_mlb_morning()` at 6:00am PT, after Team Defense — i.e. last in the pitcher-model chain, since it depends on Stuff Score, pK+, and pHR+ all having already run earlier in the same 6am sequence.

pipeline/mlb/build_pk_phr.py

Purpose — pK+ (strikeout-rate quality) and pHR+ (home-run-rate suppression quality): two empirically-weighted composite indices built from the same 6-component league-baseline machinery as Goose+2 (in fact reads the same `mlb.league_pitch_baselines` table that `build_goose2.py` populates), but recombined into two separate, narrower composites specifically validated against K rate and HR rate respectively.

Inputs — `mlb.league_pitch_baselines` (produced by `build_goose2.py`'s baseline-computation step — this script does **not** rebuild baselines itself, it only reads them, so it must run after Goose+2 in the daily chain), `mlb.pitches / mlb.at_bats / mlb.games` for current-season pitcher stats, batter-quality whiff weighting (same pattern as Stuff Score/BAPV+).

Outputs — `mlb.pk_scores / mlb.pk_overall`, `mlb.phr_scores / mlb.phr_overall` (all per pitcher × pitch type × bat_side, plus usage-weighted overall with LHH/RHH splits).

Key functions - `safe_idx(pitcher_val, league_val, invert=False) -> float` — the shared ratio-to-100 indexing helper (same pattern as Goose+2's `safe_idx`). - `build_pk_phr(score_season, baseline_season, db)` — single big loop: for each pitcher × pitch type × bat_side, computes 4 pK+ components (whiff/CSW/contact/chase, with batter-quality-weighted whiff) and 2 pHR+ components (SLG-against, hard-hit%), indexes each against the matched baseline (falling back to a same-pitch-type-any-hand baseline, then to a simple league mean if no baseline row exists at all), weights and regresses to two separate scores. - `validate(season, db)` — correlates pK+/pHR+/Goose+2/Stuff against May-onward K rate and HR rate.

Formula annotations - **pK_WEIGHTS** (docstring-documented correlation to actual K rate): whiff 0.290 ($r=0.473$), csw 0.269 ($r=0.439$), contact 0.268 ($r=0.437$, inverted — lower contact% is better), chase 0.173 ($r=0.281$) — note these four weights were derived by normalizing the four component correlation coefficients to sum to 1 ($0.473+0.439+0.437+0.281=1.630$; each $\div 1.630 \approx$ the weights shown), i.e. **weights are literally proportional to how well each raw component predicted actual K rate out-of-sample.** - **pHR_WEIGHTS**: slg 0.868 ($r=-0.311$ SLG against), hard_hit 0.132 ($r=-0.047$) — SLG-against overwhelmingly dominates the HR-suppression model; hard-hit% contributes only marginally, per the file's own noted out-of-sample correlation to actual HR rate: $r=-0.215$ (weaker than pK+'s K-rate correlation, an intentional acknowledgment that HR is a noisier outcome to predict). - **Weighted whiff** (pK+'s whiff component): identical batter-quality-weighting logic to Stuff Score and BAPV+ — `weight = min(LEAGUE_AVG_WHIFF(0.267) / batter_own_whiff_rate, 5.0)`, applied per-pitch before aggregating the pitcher's whiff rate, so a whiff against a low-whiff-rate (contact) hitter counts up to 5× more than a whiff against a free swinger. - **SLG-against source** (pHR+): computed from `mlb.at_bats.event` (English event strings like 'Home Run') joined to the **last pitch of the at-bat** (`ORDER BY pitch_number DESC LIMIT 1` per at-bat, via a lateral join) to attribute the outcome to whichever pitch type ended the PA — a different join pattern than the whiff/CSW/contact/chase components, which attribute every individual *pitch* (not just the AB-ending one) to its own pitch type. This is a deliberate and correct distinction (SLG-against is an at-bat-outcome stat, only meaningful once per AB; whiff/CSW are pitch-level events), but worth knowing when reconciling pitch counts between the two halves of this model. - Both `pk_plus / phr_plus` use the same `regress_to_mean(score, n, k=200)` shrinkage as every other “+”-model in this codebase.

CLI usage

```
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/build_pk_phr.py --score-season 2026 --baseline-season 2025
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/build_pk_phr.py --score-season 2026 --validate-only
```

Schedule — `run_build_pk_phr()` (`--score-season 2026 --baseline-season 2025`) runs as part of `run_mlb_morning()` at 6:00am PT, after Stuff Score and before Juiced+2 — positioned after Goose+2 (whose baseline table it depends on) and before Goose+3 (which depends on this script's output).

pipeline/mlb/generate_homepage.py

Purpose — Server-renders the “morning brief” — a single static HTML page (newspaper-styled, monospace) recapping yesterday's results and previewing today's slate: scores, projection-vs-actual grades, focus-team box scores, Kalshi market comparison, K/HR/TB projections, game win picks, matchup mismatches, and (as of this session's integrity-check work) a data-integrity status strip at the bottom. Written to `/app/static/homepage.html`, served at `/briefing`, and separately emailed via `send_email.send_daily_brief_email`.

Inputs — Extremely broad — reads from nearly every model/projection table: `mlb.games`, `mlb.daily_actuals`, `mlb.model_scores`, `mlb.daily_projections`, `mlb.daily_batter_projections`, `mlb.game_projections`, `mlb.goose3_overall`, `mlb.pk_overall`, `mlb.phr_overall`, `mlb.kalshi_markets`, `mlb.boxscore_batting / boxscore_pitching`, `mlb.pipeline_integrity` (added this session).

Outputs — `/app/static/homepage.html` (file write, no DB writes).

Key functions (partial list — this is the largest file in the pipeline at 1,589+ lines, mostly HTML-template string-building rather than logic) - `poisson_cdf(lam, n) / _poisson_prob(lam, n)` — **two near-duplicate implementations** of $P(K \geq n)$ given a Poisson-distributed strikeout total with mean `lam`, computed via an iterative log-space running product (avoids overflow from computing `lam^k` directly for large `k`) rather than calling a stats library, clamped to `[0.03, 0.97]` (never reports a market-line probability as fully certain either direction). Used to translate a pitcher's projected strikeout mean into “probability of clearing a given Kalshi strikeout line.” -

`_get_optimal_parlay_line(proj_ks, lines, min_prob=0.55, min_edge=0.10)` — among all available Kalshi strike lines for a pitcher, finds the highest line where both `our_computed_prob >= min_prob` AND `our_prob - market_implied_prob >= min_edge`; falls back to the single highest-edge line if none clears both bars. - `compute_parlay_line(proj_ks, std_k, confidence, slider_level, all_lines)` — computes a risk-adjusted “buffer” below the raw projection based on a user-facing risk slider (0=conservative/2=aggressive), pitcher confidence, and that pitcher’s own K-count variance, then targets a Kalshi line at `proj_ks - total_buffer` (see Formula annotations). - `get_yesterday_scores`, `get_focus_team_boxscore`, `get_model_scores`, `get_projection_results`, `get_batter_call_results`, `get_pitcher_k_variance`, `get_today_outlook`, `get_today_game_projections` — one query-and-shape function per homepage section. - `get_latest_integrity_check(db)` (added this session) — reads the most recent `mlb.pipeline_integrity` row, wrapped in try/except with `db.rollback()` on failure so a missing table before the integrity check has ever run doesn’t crash homepage generation. - `get_report_mode()` — ‘morning’ before 9am PT, ‘daytime’ otherwise; determines whether the recap block or the outlook block renders first on the page. - `render_homepage(yesterday, today, db) -> str` — the master orchestrator that calls everything above and assembles the final HTML string.

Formula annotations - Parlay buffer formula (`compute_parlay_line`): `total_buffer = base_buffer(slider) + conf_adj + var_adj`, where `base_buffer = {0: 3.0, 1: 2.0, 2: 1.0}[slider_level]` (more conservative slider = bigger safety margin subtracted from the raw projection), `conf_adj = (1 - confidence/100) * 1.5` (lower pitcher-confidence score → bigger buffer), `var_adj = max(0, (std_k - 2.0) * 0.3)` (only pitchers with K-count stddev above 2.0 get an extra buffer, scaled 0.3 per extra stddev unit — low-variance pitchers get no penalty). Final target line = `proj_ks - total_buffer`, then the highest actually-available Kalshi line at or below that target is selected. - **Poisson K-probability**: standard $P(K \geq n) = 1 - \text{CDF}(n-1; \lambda)$ computed via $\sum_{k=0}^{n-1} e^{-\lambda} \lambda^k / k!$, but implemented with an incremental log-space update (`log_term += log_lam - log(k+1)` each iteration) rather than computing factorials/powers directly, which is the numerically-stable way to do this in pure Python without a stats library import. - **K variance**: `get_pitcher_k_variance` computes `STDDEV(actual_k)` per pitcher from their last 45 days of starts (min 3 starts), defaulting to 2.5 if stddev is null (e.g., exactly one qualifying start) — this feeds directly into the parlay buffer’s `var_adj` term above. - Every table on the page pulls pre-computed values from upstream tables — this file computes **no** batting/pitching stat formulas of its own (no OBP/SLG/ERA anywhere in it); it’s purely a presentation/aggregation layer over other scripts’ output, plus the Poisson/parlay math described above which is specific to Kalshi market modeling.

CLI usage

```
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/generate_homepage.py
```

(No CLI arguments — always renders for `today / yesterday = date.today()`.)

Schedule — Run very frequently: `run_generate_homepage()` fires at 6:30am PT, every hour at :02 from 9am–9pm PT (paired with the hourly game-projection rebuild), and again at 9:00am/12:00pm/4:00pm PT (each followed 5 minutes later by `run_send_daily_brief()` emailing the freshly-rendered page). Also available via `--now homepage`.

pipeline/mlb/live_poller.py

Purpose — The one pipeline script that is **not** part of the `daily_pipeline.py` scheduler at all — it runs as its own always-on Docker service (`sports_live_poller`), continuously polling MLB’s live GUMBO feed for any currently in-progress game, storing/transforming state on every poll, and triggering a full BAPV+ compute + a projection “snapshot” the moment a game goes Final.

Inputs — MLB Stats API `GET /v1/schedule` (to discover live/Final games for today+yesterday, UTC-safety margin) and `GET /v1.1/game/{game_pk}/feed/live` (GUMBO feed, same endpoint `fetch.py` uses); the running FastAPI service itself, via `API_BASE_URL` (defaults to `http://api:8000` in Docker), for the snapshot-projections callback.

Outputs — `mlb.raw_events` (continuous upsert as the game progresses), then the **full transform pipeline** (`transform_game_pk`) re-run on every single poll cycle for every live game, plus a `compute_bapv`-driven live BAPV update; on game completion, calls `POST {API_BASE_URL}/mlb/matchup/snapshot/{game_pk}` (an API endpoint, not covered in this batch — see Section 5) to freeze batter projections at game-start state.

Key functions - `get_live_games(target_date=None) -> list` — schedule lookup filtered to `detailedState` in (“In Progress”, “Manager challenge”, “Delayed”), checking both today and yesterday’s date (UTC-safety margin for games that started before midnight UTC but are “today” in US time zones). - `get_completed_games(target_date=None) -> list` — same lookup filtered to “Final”, used to catch games that finished between poll cycles without ever being seen “live” by this process (e.g. if the poller restarted mid-game). - `fetch_live_feed(game_pk) -> dict` — raw GUMBO fetch, no caching/skip logic (unlike `fetch.py`’s 7-day-stale skip — this always re-fetches, since the whole point is picking up in-progress changes). - `store_raw_event(game_pk, data, db) -> bool` — upserts `mlb.raw_events`, returns whether the game’s `status` field changed since the last poll (used only for a boolean — not consumed differently in this version of the code). - `get_current_pitcher_stats(game_pk, db) -> list` — quick in-game CSW/whiff/velo/spin summary per pitcher × pitch type from `mlb.pitches`, for console display (not the full BAPV+ pipeline). - `transform_and_score(game_pk, db) -> int` — on a completed game, runs `transform_game_pk` then the full `compute_bapv` pipeline (load tendencies using **prior**-season tendencies, `load_pitches` scoped to that one game, vectorized BAPV compute, normalized against the prior season’s league-average BAPV, stored). - `poll_game(game_pk, db) -> dict` — the main per-cycle worker: fetch feed → store raw → **re-run full transform on every poll** (not just on completion) → read back current pitcher stats for display. - `main()` —

infinite loop (`while True`), default 120s interval, tracks `completed_games` / `snapshotted_games` sets in-process (lost on restart — no persistence of “have I already snapshotted this game” beyond the current process lifetime) to avoid re-triggering the snapshot POST or re-scoring a completed game repeatedly.

Formula annotations — No new stat formulas of its own; reuses `transform.py`’s and `compute_bapv.py`’s logic verbatim (imported directly, not reimplemented) via `sys.path.insert(0, '/pipeline')` inside the functions that need them (a slightly unusual pattern — most other scripts insert their path once at module load time; this file does it lazily inside `transform_and_score` / `poll_game`).

CLI usage

```
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/live_poller.py          # poll all live games, 120s
interval
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/live_poller.py --game-pk 823244 # poll one specific game
PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/live_poller.py --interval 120
```

Schedule — Not part of `daily_pipeline.py`’s cron at all. Runs continuously as the dedicated `sports_live_poller` Docker service (docker-compose.yml / docker-compose.aws.yml, command `python3 /pipeline/mlb/live_poller.py`), restarting automatically (restart: unless-stopped) and looping forever with its own internal sleep interval rather than being externally scheduled.

Section 4 — Stat Calculation Reference (CRITICAL)

This section was audited personally against the standard MLB formulas (not delegated) using raw code evidence gathered from `api/routers/mlb.py`, `pipeline/mlb/build_actuals.py`, `pipeline/mlb/build_daily_projections.py`, `pipeline/mlb/build_calibration.py`, and `pipeline/mlb/transform.py`. The single biggest finding: **the same stat is computed differently in different endpoints**, so “is our OBP right?” doesn’t have one answer — it depends which endpoint you’re looking at. Full line-by-line evidence lives in the companion `FORMULAS.md` reference sheet.

4.1 Batting stats

BA — Batting Average

Formula used (all locations, consistent):

```
ROUND(SUM(bb.hits)::numeric / NULLIF(SUM(bb.at_bats), 0), 3)
```

Correct MLB formula: $BA = H / AB$ **Verdict:**  CORRECT — matches everywhere it’s found (`/batter/{id}/season-stats`, `/lineup-stats/{game_pk}`, `/hitter/{id}/profile`, `/lineup-batter-stats`).

OBP — On-Base Percentage

Correct MLB formula: $OBP = (H + BB + HBP) / (AB + BB + HBP + SF)$

Five different formulas were found in `api/routers/mlb.py`, and they disagree with each other:

Variant	Endpoint(s)	File:line	Formula	Verdict
A	GET <code>/batter/{batter_id}/season-stats</code> , GET <code>/lineup-stats/{game_pk}</code> , <code>batter-vs-hand</code> helper	2781-2783, 2850-2852, 2214-2215	$(H + HBP) / (AB + HBP + SF)$	 WRONG — walks (BB) omitted from both numerator and denominator entirely
B	GET <code>/hitter/{batter_id}/profile</code>	2963-2967	$(H + BB + HBP) / (AB + BB + HBP + SF)$	 CORRECT — this is the textbook formula
C	GET <code>/hitter/{batter_id}/juiced-splits</code> (<code>obp</code> , <code>lg_obp</code>)	4352-4354, 4371-4377	event-list AVG (1B/2B/3B/HR/BB/IBB/HBP = 1, else 0) over qualifying <code>at_bats</code> rows; denominator doesn’t exclude sac bunts	 WRONG (minor) — sac bunts (SH) should be excluded from the OBP denominator and aren’t; numerator logic itself is fine
D	GET <code>/game/{game_pk}/batter-splits/{batter_id}</code> → <code>game_state_split</code> subquery	5141-5142	event-list AVG, but list is (1B/2B/3B/HR/BB/IBB) — HBP is missing	 WRONG — HBP dropped from the numerator (the sibling <code>ops_vs_hand</code> subquery at 5011-5019 in the same endpoint does include HBP)

Variant	Endpoint(s)	File:line	Formula	Verdict
				— internally inconsistent within one response payload)
E	GET /lineup-batter-stats	5193-5195	(H + BB) / (AB + BB)	✗ WRONG — HBP omitted from both numerator and denominator; also missing COALESCE around walks, so a single NULL walks row nulls the whole aggregate

Fix: Standardize every OBP computation on Variant B's formula: $(H + BB + HBP) / (AB + BB + HBP + SF)$. Since `mlb.boxscore_batting` already carries `hits`, `walks`, `hit_by_pitch`, `at_bats`, `sac_flies` as MLB's own authoritative per-game numbers, every `boxscore_batting`-based endpoint (Variants A, E) should just copy Variant B's SQL verbatim. The two event-based endpoints (C, D) should add `Hit By Pitch` to variant D's event list and exclude `sac-bunt` events from variant C's denominator.

SLG — Slugging Percentage

Correct MLB formula: $SLG = TB / AB$

Variant	Endpoint(s)	File:line	Formula	Verdict
A	/batter/{id}/season-stats, /lineup-stats, /hitter/{id}/profile, /lineup-batter-stats	2784, 2853, 2968, 5196	SUM(total_bases) / SUM(at_bats)	✓ CORRECT
B	/hitter/{batter_id}/juiced-splits, /game/{game_pk}/batter-splits/{batter_id} (several subqueries)	4348-4351, 5014-5016, 5020-5022, 5037-5041, 5143-5145	AVG(CASE event WHEN Single→1, Double→2, Triple→3, HR→4, ELSE→0 END) over all qualifying at_bats rows (BB/HBP/SF/SH rows included as 0s)	✗ WRONG — the denominator is every qualifying plate-appearance-like row, not just AB. This silently dilutes SLG downward for any batter with walks; it's really computing $TB / (AB + BB + HBP + SF + SH)$, not TB / AB

Fix: Variant B's WHERE clause needs to also exclude BB/HBP/SF/SH events (same non-AB-event list used correctly elsewhere in the codebase), or switch to `boxscore_batting.total_bases / boxscore_batting.at_bats` like Variant A.

OPS

Correct MLB formula: $OPS = OBP + SLG$ **Verdict:** ✓ **CORRECT** *mechanically* — every location just adds its own query's `obp + slg` (e.g. 2785-2790, 2969-2974, 4393, 5097-5098, 5197-5200), never recomputing independently. But since OBP/SLG themselves have multiple broken variants (above), **OPS inherits whichever error its host endpoint's OBP/SLG has**. There is no case of OPS being wrong independently of its inputs.

PA — Plate Appearances

Correct MLB formula: $PA = AB + BB + HBP + SF + SH$

- **One place computes it explicitly and correctly:** GET /hitter/{batter_id}/profile, lines 2951-2952 — `SUM(at_bats) + SUM(walks) + SUM(hit_by_pitch) + SUM(sac_flies) + SUM(sac_bunts)`. ✓ **CORRECT**.
- **Everywhere else**, “PA” is approximated as `COUNT(*)` of `mlb.at_bats` rows after excluding non-PA event types (`Runner Out`, `Caught Stealing 2B/3B`, `Wild Pitch`, `Passed Ball`) — e.g. lines 3212, 5023, 5075, 5111, 5138. ⚠ **REASONABLE PROXY, not verified equivalent** — `transform_at_bats()` in `pipeline/mlb/transform.py` writes one `at_bats` row per GUMBO `allPlays` entry, and each play is one real plate appearance, so this proxy should track true PA closely as long as the excluded-event list is complete. It is not “wrong” the way the OBP/SLG variants are, but it's a second, parallel PA definition living alongside the `boxscore`-based one above — worth consolidating onto a single source of truth.

K% for batters

Correct MLB formula: $K\% = K / PA$ (a very common error is using K / AB instead — the user flagged this exact risk, and it exists in this codebase)

Variant	Endpoint(s)	File:line	Formula	Verdict
1	<code>/batter/{id}/season-stats</code> , <code>/lineup-stats/{game_pk}</code>	2791, 2860	<code>SUM(strikeouts) / SUM(at_bats)</code>	❌ WR — this exactly K/AB -instead K/PA e AB is a PA, so system oversta (a batte walks a looks lil strike o more o than th relative their re opportu
2	<code>/hitter/{batter_id}/profile</code>	2975-2980	<code>SUM(strikeouts) / (AB+BB+HBP+SF+SH)</code>	✅ COI — true
3	<code>/game/{game_pk}/batter-splits/{batter_id}</code> (event-based), <code>build_daily_projections.py::get_pitcher_season_k_pct / get_team_k_rate</code> (pitcher-side, same pattern)	5073-5074, 5146-5147, <code>build_daily_projections.py:185-199</code>	<code>K / (qualifying at_bats rows)</code>	✅ REASC — sam proxy c as the f section

Fix: Same remedy as OBP — standardize `/batter/{id}/season-stats` and `/lineup-stats/{game_pk}` on the `/hitter/{id}/profile` PA-denominated formula.

BB% for batters

Finding: No endpoint in `api/routers/mlb.py` exposes an *actual* BB% stat at all — grepped for a BB-rate column and found none. The only BB-rate value anywhere in the system is `proj_bb_pct`, a **projected** (simulation-derived) figure from `pipeline/mlb/simulate.py`, surfaced via `build_daily_projections.py`. There's no “real BB%” counterpart to sit next to the real K% shown on `/hitter/{id}/profile`. **Verdict:** Not a formula bug — a **missing feature**. Flagged in Section 7.

4.2 Pitching stats

ERA

Correct MLB formula: $ERA = (ER / IP) \times 9$

- **Self-computed exactly once**, `GET /lineup-pitcher-stats`, lines 5230-5232: `SUM(earned_runs) * 9 / SUM(innings_pitched)`. This is formula-correct **but** `innings_pitched` itself is corrupted before it ever reaches this query — see **IP conversion** below. ❌ **WRONG in practice**, even though the ERA arithmetic itself is fine, because its input is bad.
- **Everywhere else** (main Pitcher Profile page, season comparisons), ERA is pulled straight from `mlb.fangraphs_pitching.era` (line 394) — Fangraphs' own authoritative value, computed on Fangraphs' side from real innings. ✅ **CORRECT** (trustworthy, not locally derived).

WHIP

Correct MLB formula: $WHIP = (BB + H) / IP$ **Finding:** Not computed locally anywhere in `api/routers/mlb.py` or `pipeline/mlb/build_actuals.py`. The only WHIP value in the system is `mlb.fangraphs_pitching.whip`, passed through as-is. ✅ **CORRECT** (by virtue of not being locally computed) — but flagged as a **landmine**: if anyone ever adds a local WHIP calc using `boxscore_pitching.innings_pitched`, it will inherit the same IP-notation bug that's already broken the local ERA calc.

Finding: Not computed anywhere in the codebase (grepped `k_9`, `k9`, `per_9`, `/9`, `bb_9`, `bb9` — no matches), and not among the Fangraphs columns currently selected (`era`, `era_plus`, `fip`, `whip`, `k_pct`, `bb_pct`, `swstr_pct`, `war`, `xera`, `siera` — line 394).

Missing feature, not a wrong formula. Flagged in Section 7.

K% for pitchers (K/BF)

Only computed as a projection *input*, not a displayed actual stat: `build_daily_projections.py::get_pitcher_season_k_pct()` (lines 185-199) computes $K / (\text{qualifying at_bats rows})$ as a batters-faced proxy (same caveat as the batter-side PA proxy). No endpoint in `api/routers/mlb.py` computes and displays a real “K% (actual)” stat from `boxscore_pitching` (e.g. $\text{SUM}(\text{strikeouts}) / \text{SUM}(\text{batters_faced})$) the way the batter side does on `/hitler/{id}/profile`. **Asymmetry worth noting**, not itself wrong.

IP conversion — baseball notation vs decimal

This is the highest-impact bug in the whole audit. MLB reports innings pitched in *thirds notation*: “6.1” means 6⅓ innings, “6.2” means 6⅔ innings — the digit after the decimal point is thirds-of-an-inning, not tenths.

- **Where it's broken:** `pipeline/mlb/transform.py`, line 349:

```
innings_pitched=float(p["inningsPitched"]) if p.get("inningsPitched") else None,
```

This casts “6.1” straight to the Python float `6.1` — literal decimal, not 6.333. Every row of `mlb.boxscore_pitching.innings_pitched` is stored this way.

- **Where the corruption propagates uncorrected:**
 - `api/routers/mlb.py` lines 5225, 5230-5232 (`/lineup-pitcher-stats`'s `ip` column and its ERA calc) — sums the raw broken values.
 - `pipeline/mlb/build_actuals.py` line 39 — copies `boxscore_pitching.innings_pitched` straight into `mlb.daily_actuals.actual_ip` with no conversion. Since `daily_actuals` is what `build_model_scores.py` grades projections against, **this means our own model-accuracy tracking is scored against a systematically distorted “actual IP” value.**
- **Where it's done correctly:** `pipeline/mlb/build_daily_projections.py`, `get_pitcher_pa_per_inn()`, lines 215-216:

```
FL00R(bp.innings_pitched) + (bp.innings_pitched - FL00R(bp.innings_pitched)) * 10.0 / 3.0 as ip_decimal
```

This is the right conversion (multiply the tenths-fraction by 10/3 to recover thirds) — it's just applied in exactly one place, while every other consumer of `innings_pitched` reads the raw broken value.

- **Effect:** because `.1 / .2` as literal decimal (0.1, 0.2) is always *less* than the true fraction ($\frac{1}{3} \approx 0.333$, $\frac{2}{3} \approx 0.667$) it's meant to represent, every aggregate that sums raw `innings_pitched` **understates total innings**, which **overstates** ERA/WHIP/any rate-per-IP stat computed from it. The error compounds across a season as more fractional-inning appearances accumulate.

Fix: Either (a) store innings pitched correctly at write time in `transform.py` — convert to true decimal on transform, and keep a separate display column in baseball notation for UI — or (b) never `SUM()`/divide `boxscore_pitching.innings_pitched` directly; always route through `get_pitcher_pa_per_inn`'s conversion (or a shared SQL helper) first. Option (a) is the correct systemic fix.

4.3 Model metrics

These are proprietary, not external MLB stats, so there's no “correct formula” to grade against — this section documents what they mean and flags one real internal inconsistency.

proj_k_pct

A **percentage number** (e.g. `24.5` = 24.5%), representing a **per-PA (batters-faced) rate**, built in `build_daily_projections.py` in three stages: 1. Raw simulation output × calibration: `proj_k_pct = summary["k_pct"] * k_calibration` (line 372). 2. Rescaled per opposing lineup: `lineup_k_calib = k_calib * clamp(lineup_raw_k / 0.224, 0.60, 1.30)` (lines 444-447). 3. Blended toward the pitcher's actual season K%, weighted by batters-faced sample size (`blend_weight = min(0.60, bf/300)`) (lines 452-459).

Confirmed per-PA (not per-AB) by its own downstream usage — it's multiplied against `pa_per_inn` (“batters faced per inning”) at line 461, which only makes dimensional sense if `proj_k_pct` is itself a per-PA rate.

proj_ks_6inn

```
proj_ks_6inn = (proj_k_pct / 100) × proj_ip × pa_per_inn × lineup_k
```

where `proj_ip` = `avg_ip(pitcher) × acwr_factor` (a workload/fatigue multiplier — see Section 3/8 for ACWR), and `pa_per_inn` is the pitcher's own historical batters-faced-per-inning rate. **Note the field name is misleading** — `proj_ip` is not hardcoded to 6 innings, it floats with the pitcher's actual projected workload, so `proj_ks_6inn` really means “projected Ks over projected innings,” not literally “Ks in exactly 6 innings.” An `f5` (first-5-innings) variant caps `proj_ip` at 5.0 for that specific calc.

proj_ks_locked

Not a formula — a **write-once snapshot**. Set at lines 544-546 only when a real posted lineup (≥ 7 batters) was used for that projection run; on every subsequent run for the same `(date, pitcher)`, the upsert's `COALESCE(existing_locked_value, new_value)` means **whichever value was locked first is permanent for the day** — later re-runs (even with a corrected/updated posted lineup) cannot change it. `build_actuals.py` falls back to `COALESCE(proj_ks_locked, proj_ks_6inn)` when nothing was ever locked (e.g. a game where a lineup never got posted before first pitch).

K_PCT_CALIBRATION

Two things share the name: a module-level fallback constant (`0.830`, `build_daily_projections.py` line 40) used only if the DB has no calibration row yet, and the real runtime value loaded fresh each run from `mlb.projection_calibration` (most recent row). That table is updated nightly by `build_calibration.py::compute_k_calibration()`, which nudges the multiplier toward the ratio of actual-to-projected K counts over a rolling lookback window (`new_calib = current × (1 + LEARNING_RATE × (observed_ratio - 1))`), clamped to a min/max band) — this is the “self-learning” piece referenced in Section 3.

lineup_k_factor

```
raw_k = SUM(strikeouts) / SUM(at_bats)   for the batters in the lineup (min 50 AB)
ratio = raw_k / 0.224                    (0.224 = hardcoded "league average" K rate)
lineup_k = 1.0 + (clamp(ratio, 0.80, 1.25) - 1.0) × 0.5    ("dampened" 50% toward neutral)
```

⚠ **Internal inconsistency worth flagging:** this function's own SQL computes the lineup's K rate as **K/AB**, while every sibling function in the same file (`get_pitcher_season_k_pct`, `get_team_k_rate`) computes K rate as **K/PA (batters faced)**. Since $AB < PA$, K/AB is always numerically *higher* than the equivalent K/PA — comparing an AB-denominated `raw_k` against the `LEAGUE_K_PCT = 0.224` constant (which represents real MLB's league-average K% — a PA-denominated figure) is an apples-to-oranges comparison that will systematically overstate how extreme a lineup's true strikeout tendency is. Also, `0.224` is hardcoded independently in two places in the file (this function and `get_team_k_rate`'s fallback) rather than defined once.

Section 5 — API Endpoints Reference

`api/routers/mlb.py` (5,814 lines) defines all MLB routes on `router = APIRouter()`, mounted under prefix `/mlb` (see `api/main.py`). All routes take `db: Session = Depends(get_db)` unless noted. A simple in-memory `_cache dict (cache_get / cache_set, TTL in seconds)` backs a handful of hot endpoints (`live-games`, `schedule/today`, `leaderboard`, `today-with-lineups`, `parlay/builder`, `score-projection`).

Legend: **Tables** = primary tables queried (not exhaustive for very large endpoints). **UI** = best-guess dashboard section.

Pitcher Profile (`/mlb/pitcher/{pitcher_id}/...`, plus `pitcher-search/live-pitcher/marcel`)

Method & Path	Params	Returns	Tables	UI
GET <code>/live-pitcher/{pitcher_id}</code>	today only	per-pitch-type velo/CSW/whiff for pitcher's outing today	<code>mlb.pitches</code> , <code>mlb.games</code>	Pitcher Profile (live)
GET <code>/pitcher/{pitcher_id}/season-comparison</code>	—	2025 vs 2026 pitch-arsenal comparison (BAPV+, velo, spin, movement), Goose+, pK+/pHR+, Fangraphs/BBref stats, headshot	<code>mlb.pitch_quality_scores</code> , <code>mlb.goose_scores</code> , <code>mlb.goose_overall</code> , <code>mlb.fangraphs_pitching</code> , <code>mlb.boxscore_pitching</code> , <code>mlb.bbref_pitching</code> , <code>mlb.pk_overall</code> , <code>mlb.phr_overall</code>	Pitcher Profile

Method & Path	Params	Returns	Tables	UI
GET /pitcher-search?q=	q	name/id search (LIKE match)	mlb.at_bats	Pitcher Profile (search)
GET /pitcher/{pitcher_id}/game-log?season=	season	per-game BAPV+ log + per-pitch-type breakdown	mlb.pitch_quality_scores , mlb.games	Pitcher Profile
GET /pitcher/pk-phr/leaderboard?season=&min_pitches=	season, min_pitches	pK+/pHR+/Stuff/Goose+3/ERA+ leaderboard	mlb.pk_overall , mlb.phr_overall , mlb.stuff_overall , mlb.goose3_overall , mlb.bbref_pitching	Metrics/Leaderboard
GET /pitcher/{pitcher_id}/pitch-counts?season=&bat_side=	season, bat_side	ball/strike counts per pitch type	mlb.pitches , mlb.at_bats , mlb.games	Pitcher Profile
GET /pitcher/{pitcher_id}/pitch-counts-full?season=	season	season + last-14-day counts w/ velo/whiff/CSW/usage, split by bat side, + league avg	mlb.pitches , mlb.at_bats , mlb.games	Pitcher Profile
GET /pitcher/{pitcher_id}/pitch-group-splits?season=	season	Hard/Breaking/Soft group splits, season/14-day, vs LHH/RHH, + league avg	mlb.pitches , mlb.at_bats , mlb.games	Pitcher Profile
GET /marcel/{pitcher_id}?season=	season	Marcel-style 3-yr weighted (5/4/3) K%/BB%/FIP/ERA/SIERA projection w/ regression to league mean	mlb.fangraphs_pitching , mlb.pitch_quality_scores	Pitcher Profile
GET /pitcher/{pitcher_id}/pitch-physics?season=	season	avg release point/velocity/spin/movement per pitch type (for 3D trajectory animation)	mlb.pitches , mlb.at_bats , mlb.games	Pitcher Profile (3D)
GET /pitcher/{pitcher_id}/percentiles?season=	season	percentile rank vs qualified pitchers (Goose+, whiff/CSW/hard-hit, ERA+, K%, BB%)	mlb.pitch_quality_scores , mlb.goose_overall , mlb.bbref_pitching , mlb.fangraphs_pitching , mlb.goose3_overall , mlb.pk_overall , mlb.phr_overall	Pitcher Profile
GET /pitcher/{pitcher_id}/pitch-percentiles?season=&bat_side=	season, bat_side	per-pitch-type percentile vs all pitchers throwing that pitch	mlb.pitches , mlb.at_bats , mlb.games	Pitcher Profile
GET /pitcher/{pitcher_id}/context-splits?season=	season	K%/BB%/HR%/AVG/OBP/SLG/OPS by split_type (precomputed)	mlb.pitcher_splits	Pitcher Profile
GET /goose/pitcher/{pitcher_id}?season=	season	Goose+ v1 by pitch type + overall	mlb.goose_scores , mlb.goose_overall , mlb.goose_bucket_outcomes	Metrics
GET /goose2/pitcher/{pitcher_id}?season=	season	Goose+ 2 by pitch type + overall	mlb.goose2_scores , mlb.goose2_overall	Metrics
GET /goose3/pitcher/{pitcher_id}?season=	season	Goose+ 3 by pitch type + overall (stuff/outcomes blend)	mlb.goose3_scores , mlb.goose3_overall	Metrics
GET /stuff/pitcher/{pitcher_id}?season=	season	Stuff Score by pitch type + top pitch	mlb.stuff_scores , mlb.stuff_overall	Metrics

Hitter Profile (/mlb/hitter/{batter_id}/... , /mlb/batter/{batter_id}/...)

Method & Path	Params	Returns	Tables	UI
GET /batter-tendencies?name=&season=	name, season	per-pitch-type whiff/chase/hard-hit/exit-velo/CSW tendencies	mlb.batter_pitch_type_tendencies , mlb.at_bats	Hitter Profile
GET /batter/{batter_id}/recent?days=&pitcher_hand=	days, pitcher_hand	last-N-day + season stats/pitch-group/zone splits vs a handedness	mlb.boxscore_batting , mlb.pitches , mlb.at_bats , mlb.games	Hitter Profile
GET /matchup/attack-plan/{pitcher_id}/{batter_id}	—	USE/AVOID/NEUTRAL recommendation per pitch type with reasons	mlb.pitcher_mix_profile , mlb.batter_pitch_type_tendencies , mlb.at_bats	Hitter Profile / Matchup
GET /batter/{batter_id}/season-stats?season=	season	AVG/OBP/SLG/OPS/K% season totals	mlb.boxscore_batting , mlb.games	Hitter Profile — ⚠️ OBP formula omits BB, see below

Method & Path	Params	Returns	Tables	UI
GET /lineup-stats/{game_pk}? season=	season	today's lineup batters' season AVG/OBP/SLG/OPS/K%/Juiced+2, enriched w/ BBref OPS+ & FG wRC+/WAR	mlb.boxscore_batting , mlb.games , mlb.juiced2_scores , mlb.bbref_batting , mlb.player_id_map , mlb.fangraphs_batting	Hitler Profile / Game Companion — ⚠️ same OBP bug (omits BB)
GET /hitter-search?q=	q	name search (≥5 games, 2025+)	mlb.at_bats , mlb.games	Hitler Profile (search)
GET /hitter/{batter_id}/profile? season=	season	full profile: season stats (OBP here is correct, includes BB), plus_stats (OPS+/wRC+/WAR/hard-hit/barrel), pitch-group splits (season/14d/7d, vs LHP/RHP, league avg), zone heatmap, game log w/ per-PA pitch sequences, Juiced+ and Juiced+2	mlb.boxscore_batting , mlb.bbref_batting , mlb.fangraphs_batting , mlb.player_id_map , mlb.pitches , mlb.at_bats , mlb.games , mlb.juiced_scores , mlb.juiced2_scores , mlb.goose_pitch_boundaries	Hitler Profile
GET /juiced/batter/{batter_id}? season=	season	Juiced+ v1 splits by pitch type/velo quintile + PA-weighted overall	mlb.juiced_scores , mlb.goose_bucket_outcomes	Metrics
GET /juiced2/batter/{batter_id}? season=	season	Juiced+2 detail w/ weighted component breakdown (OPS idx 30%, quality-adj idx 30%, SLG idx 20%, hard-hit idx 20%)	mlb.juiced2_scores , mlb.bbref_batting	Metrics
GET /hitter/{batter_id}/percentiles? season=	season	swing-decision percentiles (swing%/contact%/whiff%/chase%), Backwards Helmet %, Juiced+ by pitch group	mlb.pitches , mlb.at_bats , mlb.games , mlb.juiced_scores	Hitler Profile
GET /hitter/{batter_id}/juiced- splits?season=	season	Juiced+2 splits by pitcher hand vs league, incl. OPS+/Juiced+ calc	mlb.juiced2_scores , mlb.at_bats , mlb.games , mlb.goose2_overall	Hitler Profile
GET /batter/{batter_id}/context- splits?season=	season	precomputed split_type stats (AVG/OBP/SLG/OPS “for”)	mlb.batter_splits	Hitler Profile
GET /lineup-batter-stats? ids=&season=	ids (csv), season	bulk AVG/OBP/SLG/OPS/HR/K/BB + Juiced+2 for a list of batter ids	mlb.boxscore_batting , mlb.games , mlb.juiced2_scores	Game Companion (lineup card) — ⚠️ third OBP variant, omits HBP & SF too

batter_season_stats OBP formula (api/routers/mlb.py:2780-2791) — quoted verbatim:

```
ROUND(SUM(bb.hits)::numeric / NULLIF(SUM(bb.at_bats), 0), 3) as avg,
ROUND((SUM(bb.hits) + COALESCE(SUM(bb.hit_by_pitch),0))::numeric /
  NULLIF(SUM(bb.at_bats) + COALESCE(SUM(bb.hit_by_pitch),0) +
    COALESCE(SUM(bb.sac_flies),0), 0), 3) as obp,
ROUND(SUM(bb.total_bases)::numeric / NULLIF(SUM(bb.at_bats), 0), 3) as slg,
ROUND(
  (SUM(bb.hits) + COALESCE(SUM(bb.hit_by_pitch),0))::numeric /
  NULLIF(SUM(bb.at_bats) + COALESCE(SUM(bb.hit_by_pitch),0) +
    COALESCE(SUM(bb.sac_flies),0), 0) +
  SUM(bb.total_bases)::numeric / NULLIF(SUM(bb.at_bats), 0)
, 3) as ops,
ROUND(SUM(bb.strikeouts)::numeric / NULLIF(SUM(bb.at_bats), 0) * 100, 1) as k_pct
```

This never references `bb.walks` at all — OBP = $(H+HBP)/(AB+HBP+SF)$, missing BB from both numerator and denominator. OPS inherits the same bug since it's OBP+SLG. K% is K/AB , not K/PA .

Contrast — hitter_profile OBP formula (api/routers/mlb.py:2963-2980), which is correct:

```

ROUND(
  (SUM(bb.hits) + COALESCE(SUM(bb.walks),0) + COALESCE(SUM(bb.hit_by_pitch),0))::numeric /
  NULLIF(SUM(bb.at_bats) + COALESCE(SUM(bb.walks),0) +
    COALESCE(SUM(bb.hit_by_pitch),0) + COALESCE(SUM(bb.sac_flies),0), 0)
, 3) as obp,
...
ROUND(
  SUM(bb.strikeouts)::numeric /
  NULLIF(SUM(bb.at_bats) + COALESCE(SUM(bb.walks),0) +
    COALESCE(SUM(bb.hit_by_pitch),0) + COALESCE(SUM(bb.sac_flies),0) +
    COALESCE(SUM(bb.sac_bunts),0), 0) * 100
, 1) as k_pct

```

This correctly includes BB in OBP, and correctly computes K% as K/PA (PA including sac bunts too).

Third variant — lineup_batter_stats OBP (api/routers/mlb.py:5192-5200):

```

ROUND(SUM(bb.hits)::numeric / NULLIF(SUM(bb.at_bats), 0), 3) as avg,
ROUND(
  (SUM(bb.hits) + SUM(bb.walks))::numeric /
  NULLIF(SUM(bb.at_bats) + SUM(bb.walks), 0), 3) as obp,
ROUND(SUM(bb.total_bases)::numeric / NULLIF(SUM(bb.at_bats), 0), 3) as slg,

```

This one includes BB but omits HBP and SF from both sides — a third, differently-wrong formula. Net effect: three endpoints (batter_season_stats, batter_recent, lineup_stats) share the (H+HBP)/(AB+HBP+SF) bug (missing BB); lineup_batter_stats has its own (H+BB)/(AB+BB) bug (missing HBP/SF); only hitter_profile gets the full formula right.

Game Companion (/mlb/live-games, /mlb/game/..., /mlb/schedule/...)

Method & Path	Params	Returns	Tables	UI
GET /fetch-game/{game_pk}	—	fetch one game's raw GUMBO JSON into mlb.raw_events (manual/debug)	mlb.raw_events	Admin/debug
GET /explore/{game_pk}	—	structural summary of a stored raw GUMBO blob	mlb.raw_events	Admin/debug
GET /live-games	— (60s cache)	in-progress/final games today+yesterday w/ live pitcher CSW stats, score, current batter/pitcher	mlb.pitches, mlb.at_bats, mlb.linescore + live MLB API	Game Compani
POST /transform/{game_pk}	—	runs transform_game_pk for a game, returns row counts	mlb.games, mlb.at_bats, mlb.pitches	Admin/debug
GET /schedule/today	— (300s cache)	today's (+yesterday fallback) full schedule w/ probable starters, live score	live MLB API only	Today's Games
GET /live-game/{game_pk}/pitcher-scores	—	live in-game BAPV+ per pitcher/pitch-type vs season baseline (pandas-based)	mlb.pitches, mlb.at_bats, mlb.batter_pitch_type_tendencies, mlb.pitch_quality_scores	Game Compani
POST /matchup/snapshot/{game_pk}	—	store pre-game per-batter K%/wOBA/TB projections (idempotent, ON CONFLICT DO NOTHING)	writes mlb.matchup_projections; reads mlb.pitch_quality_scores, mlb.pitcher_mix_profile, mlb.batter_pitch_type_tendencies	Game Compani (internal — caller before first pitch)
POST /matchup/grade/{game_pk}	—	back-fills actual outcomes into mlb.matchup_projections after game ends, returns accuracy metrics (MAE/RMSE/Pearson/Brier/skill score)	mlb.boxscore_batting → mlb.matchup_projections	Game Compani (internal)
GET /matchup/grade/summary?season=	season	season-wide aggregate accuracy for matchup projections	mlb.matchup_projections	Metrics/model-C
GET /game/{game_pk}/score-projection?n_sims=	n_sims	Monte Carlo game score projection (15-min cache)	calls mlb.simulate_innings.run_game_simulation → mlb.game_lineups etc.	Today's Games / Game Compani
GET /schedule/today-with-lineups	— (300s cache)	today's games w/ lineup-posted flag, pK+/Goose+3 for both starters	mlb.game_lineups, mlb.pk_overall, mlb.goose3_overall	Today's Games
GET /weather/today	—	today's park weather + HR modifiers	mlb.game_weather, mlb.games	Game Compani Today's Games

Method & Path	Params	Returns	Tables	UI
GET /game/{game_pk}/summary	—	full Game Companion state: score/inning/count/runners, full at-bat + pitch-by-pitch log (with live GUMBO runner/count overlay), current pitcher/batter today's-game stats, live pitch mix vs season	mlb.games , mlb.at_bats , mlb.pitches , mlb.raw_events	Game Companic (main feed)
GET /game/{game_pk}/batter-splits/{batter_id}	pitcher_id, outs, on_first/second/third	contextual batter splits for current game state: vs current pitcher hand, vs pitcher's specific pitch mix, RISP, base/out state	mlb.pitches , mlb.at_bats , mlb.games , mlb.juiced_scores	Game Companic
GET /lineup-pitcher-stats?ids=&season=	ids (csv), season	bulk IP/K/BB/H/ER/ERA/Goose+3 for pitcher ids	mlb.boxscore_pitching , mlb.games , mlb.goose3_overall	Game Companic (lineup card) — ERA formula su innings_pit directly, see bel
GET /props/live/{game_pk}	player_id, player_type, stat	live current stat value + game state from MLB boxscore/linescore API directly	none (live MLB API only)	Prop Tracker

lineup_pitcher_stats ERA formula (api/routers/mlb.py:5225-5232):

```
ROUND(SUM(bp.innings_pitched)::numeric, 1) as ip,
...
ROUND(
    SUM(bp.earned_runs)::numeric * 9 /
    NULLIF(SUM(bp.innings_pitched)::numeric, 0), 2) as era,
```

bp.innings_pitched (mlb.boxscore_pitching.innings_pitched, a Float column) is populated in pipeline/mlb/transform.py:349 via float(p["inningsPitched"]), a **raw cast of MLB's baseball-notation IP string** (e.g. "6.1" = 6⅓ innings, "6.2" = 6⅔ innings) straight to float — so "6.1" becomes 6.1 instead of 6.333, and "6.2" becomes 6.2 instead of 6.667. Summing and dividing these mis-scaled values produces a mathematically wrong ERA (and would equally corrupt any WHIP/K9/BB9 computed the same way). This is a systemic issue anywhere `innings_pitched` is aggregated across games — see Section 7.

Leaderboard (/mlb/leaderboard , /mlb/leaderboard/expanded , model leaderboards)

Method & Path	Params	Returns	Tables	UI
GET /leaderboard? season=&min_pitches=&pitch_type=	season, min_pitches, pitch_type	BAPV+ leaderboard w/ pK+/pHR+/Goose+3/Stuff Score joined	mlb.pitch_quality_scores , mlb.pk_overall , mlb.phr_overall , mlb.goose3_overall , mlb.stuff_overall	Leaderboard
GET /leaderboard/expanded? season=&min_pitches=	season, min_pitches	expanded leaderboard w/ Fangraphs rate stats + hard/break/soft pitch-mix %	mlb.pitch_quality_scores , mlb.fangraphs_pitching , mlb.pitcher_mix_profile , mlb.goose3_overall , mlb.pk_overall , mlb.phr_overall , mlb.stuff_overall	Leaderboard
GET /goose/leaderboard? season=&min_pitches=	season, min_pitches	Goose+ v1 leaderboard w/ per-pitch breakdown JSON	mlb.goose_overall , mlb.goose_scores	Leaderboard/Metrics
GET /goose2/leaderboard? season=&min_pitches=	season, min_pitches	Goose+ 2 leaderboard	mlb.goose2_overall , mlb.bbref_pitching	Leaderboard/Metrics
GET /goose3/leaderboard? season=&min_pitches=	season, min_pitches	Goose+ 3 leaderboard	mlb.goose3_overall , mlb.goose_overall , mlb.bbref_pitching	Leaderboard/Metrics
GET /juiced2/leaderboard? season=&min_pa=	season, min_pa	Juiced+2 leaderboard	mlb.juiced2_scores , mlb.bbref_batting	Leaderboard/Metrics

Today's Games (/mlb/schedule/today , /mlb/leaderboard/daily-projections , etc.)

Method & Path	Params	Returns	Tables	UI
GET /leaderboard/daily-projections?date=	date	pitcher K/HR/hit/OPS projections snapshot for a date + pK+/Goose+3 join + available dates list	mlb.daily_projections , mlb.pk_overall , mlb.goose3_overall	Today's Games
POST /leaderboard/run-daily-projections?date=	date	triggers build_daily_projections.py as subprocess, returns stdout tail	subprocess only	Admin (manual trigger)
GET /leaderboard/daily-batter-projections?snapshot_date=	snapshot_date	batter AVG/OBP/SLG/OPS/TB/HR%/K%/BB% projections snapshot	mlb.daily_batter_projections	Today's Games
GET /matchup/pitcher/{pitcher_id}/vs-lineup/{game_pk}	—	per-batter projected K% vs today's actual posted lineup, aggregated to proj Ks over 6/9 innings	live MLB boxscore API, mlb.pitch_quality_scores , mlb.pitcher_mix_profile , mlb.fangraphs_pitching , mlb.boxscore_batting , mlb.batter_pitch_type_tendencies	Today's Games / Matchup engine
GET /matchup/batter/{batter_id}/vs-pitcher/{pitcher_id}	—	per-pitch-type projected K%/wOBA/TB for one batter vs one pitcher	mlb.pitch_quality_scores , mlb.batter_pitch_type_tendencies , mlb.fangraphs_pitching , mlb.boxscore_batting , mlb.pitcher_mix_profile	Today's Games / Matchup engine
GET /matchup/{pitcher_id}/vs/{batter_id}	—	career pitcher-vs-batter head-to-head, regressed K%/HR% (shrinkage toward league avg with 20-PA prior)	mlb.matchup_history	Today's Games / Matchup

Regression formula used here (api/routers/mlb.py:5326-5333) — worth noting for Section 4 model-metrics context:

```
(SUM(k)::NUMERIC / NULLIF(SUM(pa), 0) * 100) * (SUM(pa)::NUMERIC / (SUM(pa) + 20))
+ 22.0 * (20.0 / (SUM(pa) + 20)) AS k_pct_reg
```

Empirical-Bayes shrinkage: observed K% weighted by PA/(PA+20) , blended with a fixed 22.0% league-average K% prior weighted by 20/(PA+20) . Same pattern for HR% with a 3.5% prior.

Metrics (/mlb/goose/... , /mlb/pk/... via pk-phr leaderboard, /mlb/juiced/... , /mlb/stuff/...)

Covered above under Pitcher/Hitter Profile and Leaderboard — these are the same endpoints, just the “Metrics” tab of the dashboard reusing them. Also:

Method & Path	Params	Returns	Tables	UI
GET /goose/buckets	—	velo-quintile boundaries per pitch type + bucket outcome rates	mlb.goose_pitch_boundaries , mlb.goose_bucket_outcomes	Metrics
GET /goose/bucket-labels	—	human labels (Soft/Below Avg/Average/Above Avg/Elite) + velo ranges per quintile	mlb.goose_pitch_boundaries	Metrics
GET /model-scores?days=	days (1-365)	daily MAE/RMSE/bias per player_type+metric, + 7-day rolling and season-to-date averages	mlb.model_scores	Metrics (model-QA)

Other (simulation, workload, props, parlay, misc.)

Method & Path	Params	Returns	Tables	UI
POST /simulate/at-bat	pitcher_id, batter_id, n_simulations, outs, runners, balls/strikes, game_pk, season	Monte Carlo at-bat sim results, outcome/pitch distributions, sample PAs, Goose+/Juiced+/Goose+3/weather context	calls pipeline/mlb/simulate.py::run_simulation (reads mlb.pitches , mlb.at_bats , count-mix tables)	At-Bat Simulator
GET /player/{player_id}/workload?player_type=	player_type	28-day workload history + ACWR (acute:chronic workload ratio) w/ red/yellow/green flag (>1.5 / >1.2)	mlb.player_workload	Metrics (workload)

Method & Path	Params	Returns	Tables	UI
GET /props/players/search?q=	q	today's projected pitchers/batters matching a name for prop entry	mlb.daily_projections , mlb.daily_batter_projections	Prop Tracker
GET /props/confidence	player_id, player_type, stat, line, direction	Poisson (or Bernoulli for HR) over/under probability for a prop line vs projection	mlb.daily_projections , mlb.daily_batter_projections	Prop Tracker
GET /parlay/builder?date=	date (120s cache)	per-pitcher proj Ks (+locked value), K std-dev (45-day), a 0-100 confidence blend, and all Kalshi K lines	mlb.daily_projections , mlb.kalshi_markets , mlb.daily_actuals	Parlay Builder

Parlay confidence formula (api/routers/mlb.py:5791-5796):

```

sample_conf = min(1.0, starts / 12)
variance_conf = max(0.0, 1.0 - (std_k - 1.5) / 3.5)
lineup_conf = 1.0 if row["has_locked_proj"] else 0.7
stability_conf = 1.0 - min(0.5, abs(proj_ks - proj_lock) / max(proj_ks, 1))
confidence = round((sample_conf*0.25 + variance_conf*0.40 +
                    lineup_conf*0.15 + stability_conf*0.20) * 100)

```

Weighted blend: 25% sample size (starts/12, capped at 1), 40% K-count variance (lower std_k over trailing 45 days = higher confidence), 15% whether the lineup is locked (posted) vs still projected, 20% stability (how much the live proj has drifted from the pre-lock snapshot proj_ks_locked).

proj_ks_locked (referenced here, and defined in mlb.daily_projections) is set once when the lineup locks and is not overwritten by later re-projections — it's the frozen "pre-game" number, whereas proj_ks_6inn keeps updating. (Confirm exact write site in build_daily_projections.py for Section 3/4 — not in this file.)

Raw formula snippets found in api/routers/mlb.py (for formula audit)

- batter_season_stats** OBP/SLG/OPS/K% — lines 2780-2791 (see full quote above under Hitter Profile). Missing BB in OBP; K% = K/AB.
- hitter_profile** OBP/SLG/OPS/K% — lines 2962-2980 (see full quote above). Correct: includes BB, HBP, SF (and SAC bunts in K% denominator).
- lineup_batter_stats** OBP/SLG/OPS — lines 5192-5200 (see full quote above). Includes BB but omits HBP/SF.
- batter_recent** (get_stats nested fn) — lines 2200-2215:

```

ROUND(SUM(bb.hits)::numeric / NULLIF(SUM(bb.at_bats),0), 3) as avg,
ROUND(SUM(bb.total_bases)::numeric / NULLIF(SUM(bb.at_bats),0), 3) as slg,
ROUND((SUM(bb.hits) + COALESCE(SUM(bb.hit_by_pitch),0))::numeric /
      NULLIF(SUM(bb.at_bats) + COALESCE(SUM(bb.hit_by_pitch),0) + COALESCE(SUM(bb.sac_flies),0),0), 3) as obp

```

Same "missing BB" bug as batter_season_stats.

- lineup_stats** — lines 2849-2860: identical formula/bug to batter_season_stats (see Hitter Profile section above).
- ERA** — lineup_pitcher_stats, lines 5225-5232 (quoted above under Game Companion). Sums mlb.boxscore_pitching.innings_pitched, which is stored via a naive float() cast of MLB's baseball-notation innings-pitched string in pipeline/mlb/transform.py:349 (innings_pitched=float(p["inningsPitched"])). No other endpoint in this file computes ERA/WHIP/K9/BB9 from raw pitch/at-bat/boxscore data — all other ERA/FIP/WHIP/K%/BB% values shown elsewhere (season-comparison, marcel, percentiles, expanded leaderboard) are sourced pre-computed from mlb.fangraphs_pitching or mlb.bbref_pitching, not calculated in this file.
- IP display cast** — props_live, line 5692: ip_pitched = float(ip_str) where ip_str = pit.get("inningsPitched", "0") straight from the live MLB boxscore API. Same baseball-notation-as-float issue, but this one is display-only (not summed/aggregated), so its blast radius is smaller — a "6.1" (6% IP) just displays as "6.1" (mildly confusing but not compounding).
- K% "common error" check** — batter_season_stats (line 2791) and batter_recent / lineup_stats compute batter K% as K/AB (K% = strikeouts / at_bats), not K/PA. hitter_profile (lines 2975-2980) computes it correctly as K/PA (PA = AB+BB+HBP+SF+SAC). No batter endpoint in this file computes BB% at all (not present in the returned stat sets scanned).
- Regression/shrinkage formulas** (not in the requested list, but relevant to Section 4 "model metrics"): matchup_h2h K%/HR% regression (lines 5326-5333, quoted above under Today's Games) and Marcel-style projection weighting (lines 2662-2724, marcel_projection) — both are legitimate empirical-Bayes shrinkage, not bugs.

10. proj_ks_6inn and pitcher_vs_lineup projection formula (lines 1361-1365, 1389-1392): per-batter K% is a 3-way blend — $\text{pitcher_k_adj} * \text{ip_weight} + \text{batter_k_adj} * 0.35 + \text{csw_proj_k} * \text{csw_fill}$ where $\text{csw_fill} = 1 - \text{ip_weight} - 0.35$ — then $\text{projected_ks_6inn} = \text{avg_k_pct} * 18$ (18 = 3 times through a 6-batter... actually 6 innings × 3 batters/inning = 18 batters faced assumption). This is the live matchup-engine version of the K projection; the canonical `daily_projections.proj_ks_6inn` is written by `build_daily_projections.py` (out of scope for this file — flag for Section 3/4 cross-reference).

Route count: 74 `@router.*` decorated endpoints documented above (73 read/compute + `run-daily-projections` trigger). None are omitted from the grouping; a few (`/fetch-game`, `/explore`, `/transform/{game_pk}`) are admin/debug utilities not tied to a specific dashboard tab.

Section 6 — Frontend Reference

`frontend/mlb.html` (6,291 lines; also deployed as `api/static/mlb.html`) is a single-file SPA: static HTML shell + one giant inline `<script>` block (lines 862–5762, plus two small trailing scripts for mobile-nav at 6271–6289). No build step, no framework — vanilla JS with template-string HTML injection. `const API = window.location.origin;` (line 863) — all API calls are relative to whatever host serves the page.

Navigation is a left sidebar (`.left-nav-item`, `onclick="showSection('name')"`) driving a set of `.panel` divs (`id="panel-<name>"`). `showSection(name)` (line 1014) toggles `.active` class and, for a few panels, lazily triggers the load function the first time they're shown:

```
function showSection(name) {
  document.querySelectorAll('.panel').forEach(p => p.classList.remove('active'));
  document.querySelectorAll('.left-nav-item').forEach(n => n.classList.remove('active'));
  document.getElementById('panel-'+name)?.classList.add('active');
  document.getElementById('nav-'+name)?.classList.add('active');
  if (name === 'matchup') initGameCompanion();
  if (name === 'games') loadTodaysGames();
  if (name === 'metrics') { ... lazy-render metricsContent() ... }
  if (name === 'model-scores') loadModelScores();
  if (name === 'props') initPropsTracker();
  if (name === 'parlay') loadParlayBuilder();
}
```

Panel-id map (nav name → panel): `profile` (Pitcher Profile), `matchup-engine` (Matchup/Simulation Engine, **hidden from nav**), `matchup` (Game Companion — confusingly named; the panel id is “matchup” but the feature is “Game Companion”), `hitter` (Hitter Profile), `metrics` (Metrics), `model-scores` (Model Scores), `props` (Props Tracker, **hidden from nav**), `parlay` (Parlay Builder, **hidden from nav**), `leaderboard` (Leaderboard), `games` (Today's Games).

Pitcher Profile

- **Panel:** `panel-profile`, lines 574–592 (markup: search box `#profile-search` → `#search-pills` → `#profile-content`).
- **Load path:** `searchProfile()` (line 1893) reads `#profile-search`, hits pitcher search, renders result pills; clicking a pill (or Enter) calls `loadProfile(id, name)` (line 1909).
- **Endpoints called** (from `loadProfile` and its sub-loaders):
 - `GET /mlb/pitcher-search?q=` — name autocomplete
 - `GET /mlb/pitcher/{id}/season-comparison` — core season vs. career/prior-season stat block
 - `GET /mlb/pitcher/{id}/pitch-counts-full?season=2026` — arsenal usage counts
 - `GET /mlb/pitcher/{id}/percentiles?season=2026` — league percentile rings
 - `GET /mlb/pitcher/{id}/pitch-percentiles?season=2026[&bat_side=L|R]` — per-pitch percentile bars, split vs. LHH/RHH
 - `GET /mlb/pitcher/{id}/pitch-group-splits?season=2026` — grouped-pitch splits table
 - `GET /mlb/pitcher/{pitcherId}/game-log?season=` — game log (`loadGameLog`, line 2500)
 - `GET /mlb/live-pitcher/{pid}` — live in-game stat overlay (`loadLiveForPitcher`, line 2615)
 - `GET /mlb/pitcher/{id}/pitch-physics?season=2026` — feeds the 3D pitch-trajectory visualizer (`loadPitchViz`, line 4562)
- **Rendering:** `renderProfile()` (line 2159) — builds percentile rings (`makeRing`), pitch-mix table (`buildPitchCountsRows`), tabs for season/last-14/vs-hand splits (`buildPitcherGroupRows`), and wires up the Three.js pitch visualizer (`initVizScene` / `buildVizStrikeZone` / `animatePitch`, ~4278–4626) plus the game log accordion (`toggleGameLogRow`).
- **Displayed data:** ERA/WHIP/K%/BB%/Stuff-type scores, pitch-by-pitch velo/spin/movement, percentile rings vs. league, per-pitch-type usage %, game-by-game log, live win-probability-style overlay during active games, animated 3D pitch trajectory into the

strike zone.

- **Known issues:** none flagged via comments; `loadProfile` fires 6 endpoints in parallel via `Promise.all`-style chained `fetch` — no visible error-boundary per-card (a single failed endpoint appears to blank that card rather than showing a retry).

Hitter Profile

- **Panel:** `panel-hitter`, lines 709–727 (`#hitter-search` → `#hitter-search-pills` → `#hitter-content`).
- **Load path:** `searchHitter()` (line 3206) → `loadHitterProfile(batterId, name)` (line 3231).
- **Endpoints called:**
 - `GET /mlb/hitter-search?q=`
 - `GET /mlb/hitter/{batterId}/profile?season=2026` — primary profile payload
 - `GET /mlb/hitter/{batterId}/percentiles?season=2026`
 - `GET /mlb/hitter/{batterId}/juiced-splits?season=2026` — Juiced+2 power-model splits (`renderJuicedBreakdown`, line 3251)
 - `GET /mlb/batter/{id}/season-stats?season=2026` and `GET /mlb/batter/{id}/recent?days=7&pitcher_hand=` used from the Matchup Engine's attack-plan flow, not the profile page itself — see Matchup Engine below.
- **Rendering:** `renderHitterProfile()` (line 3329) builds swing-rate bars (`swingRow`, `buildMBar`), pitch-group split tabs (`buildHitterGroupRows` / `setHitterGroupSplitsTab`), and a zone heat-map grid (`renderZoneGrid`, line 3834; `setZoneMetric` toggles the stat shown per zone cell).
- **Displayed data:** season slash line, swing-rate metrics (chase/whiff/zone-contact) vs. league percentile, Juiced+2 exit-velo/launch-angle-driven power splits, 5x5 strike-zone heat map (selectable metric), game log (`toggleHitterGameRow`).
- **Known issues:** `renderJuicedBreakdown`'s `veloRange` sub-function (line 3266) buckets exit velocity — worth a formula sanity check if Juiced+2 numbers look off; not verified here (out of scope for this section, flagged for Section 4/7 owner).

Game Companion

- **Panel:** `panel-matchup` (note: panel id is `matchup`, not `game-companion`), lines 685–708. Game picker renders into `#gc-game-pills`; content into `#gc-content` (sub-divs: `#gc-header`, `#gc-players`, `#gc-prob`, `#gc-pitch-mix`, `#gc-perf-chart`, `#gc-3d`, `#gc-lineup`, `#gc-sequence`, `#gc-splits`).
- **Load path:** entering the tab calls `initGameCompanion()` (line 4632) → populates game pills from `GET /mlb/schedule/today` → selecting one calls `loadGameCompanion(gamePk)` (line 4673) → `renderGameCompanion()` (line 4702) which fans out to the render* functions below.
- **Endpoints called:**
 - `GET /mlb/schedule/today`
 - `GET /mlb/game/{gamePk}/summary`
 - `GET /mlb/weather/today`
 - `GET https://statsapi.mlb.com/api/v1/game/{gcGamePk}/boxscore` — calls the MLB Stats API directly from the browser, bypassing our own API layer (used in `renderGCLineup`, line 4736, to get players not yet at bat)
 - `GET /mlb/lineup-batter-stats?ids=...&season=2026`, `GET /mlb/lineup-pitcher-stats?ids=...&season=2026`
 - `GET /mlb/hitter/{batterId}/juiced-splits?season=2026` (`loadJuicedSplitsForGC`, line 4944)
 - `GET /mlb/live-game/{gamePk}/pitcher-scores`
- **Rendering:** `renderGCHeader` (score/inning state), `renderGCPlayers` (line 4951), `renderGCProbability` (win-prob-style bar, line 5044), `renderGCPitchMix` (line 5105), `renderGC3D` / `initGC3D` / `buildGCField` / `renderGCPitches` / `animateGCPitch` (Three.js live pitch-by-pitch replay, ~5141–5391), `renderGCPerformanceChart` (line 5391, toggled by `toggleGCMetric` / `setGCChartPitcher` / `setGCChartPitchType` / `setGCXAxis`), `renderGCSequence` (line 5575), `renderGCSplits` (line 5608), with `selectGCab` / `selectGCPitch` for click-through at-bat/pitch selection.
- **Displayed data:** live lineup with real-time batting stats, in-game win/score probability, starter pitch-mix breakdown, animated 3D pitch trajectories per at-bat, rolling performance chart (selectable metric/pitcher/pitch-type/x-axis), pitch sequence log, matchup splits.
- **Known issues:** direct browser call to `statsapi.mlb.com` is an architectural inconsistency (CORS-dependent, no caching/rate-limit protection our API would otherwise provide) — worth flagging in Section 7.

Leaderboard

- **Panel:** `panel-leaderboard`, lines 455–573. Three tabs via `setLBTab(tab)` (line 1623): Season Stats (`#lb-season-content`), Pitcher Projections (`#lb-daily-content`), Hitter Projections (`#lb-batters-content`).
- **Load path:** `loadLB()` (line 1554) for season tab; `loadDailyProjections(date)` (line 1637) for pitcher-projections tab; `loadDailyBatterProjections(snapDate)` (line 1751) for hitter-projections tab. Manual admin buttons trigger `runDailyProjections()` (line 1661) and `runAndReloadBatters()` (line 1878).
- **Endpoints called:**

- `GET /mlb/leaderboard?season=2025&min_pitches=50` — season Goose+/Stuff leaderboard
- `GET /mlb/leaderboard/daily-projections?date=` and `GET /mlb/leaderboard/daily-batter-projections?snapshot_date=`
- `POST /mlb/leaderboard/run-daily-projections` — manual re-run trigger (used by both `runDailyProjections` and `runAndReloadBatters`)
- **Rendering:** `renderLB()` (line 1576, sortable via `sortLB(col)`), `renderDailyProjections()` (line 1679, sortable via `sortDaily(col)`, team-filterable via `renderTeamFilter / toggleBatterTeam`), `renderDailyBatterProjections()` (line 1769, with its own `th()` sortable-header helper at 1811).
- **Displayed data:** season pitch-quality leaderboard (Goose+ and related model scores) filterable by pitch type (`setType`) and min-pitch-count (`setMin`); today's per-pitcher K/IP projections; today's per-batter TB/HR projections, filterable by team.
- **Known issues:** none flagged in comments.

Today's Games

- **Panel:** `panel-games`, lines 440–452 (`#games-content`).
- **Load path:** `loadTodaysGames()` (line 1035), auto-triggered on `showSection('games')`. Per-game score-projection cards load lazily via `loadScoreProjection(gamePk, containerEl)` (line 1182).
- **Endpoints called:**
 - `GET /mlb/schedule/today`
 - `GET /mlb/game/{gamePk}/score-projection?n_sims=200` — per-game Monte Carlo run/win projection, rendered into `#proj-{gamePk}`
- **Displayed data:** today's slate with starters, projected score/total and win-probability lean per game (Monte Carlo-backed).
- **Known issues:** none flagged in comments.

Metrics

- **Panel:** `panel-metrics`, lines 732–734 (`#metrics-content`, populated once lazily by `metricsContent()`, line 936).
- **Load path:** static content generator, not an async loader — `metricsContent()` returns a template string of stat-glossary/model cards; individual pitcher/batter score lookups happen elsewhere via `loadGooseBucketLabels()` (line 904, populates bucket labels used by `scoreColor / profileScoreColor / scoreClass`) and are actually rendered inline in Pitcher/Hitter Profile via:
 - `GET /mlb/goose/pitcher/{id}?season=2026`
 - `GET /mlb/stuff/pitcher/{id}?season=2026`
 - `GET /mlb/juiced/batter/{id}?season=2026`
 - `GET /mlb/goose/bucket-labels`
- **Displayed data:** static explainer/reference content for the model metric system (Goose+, Stuff Score, Juiced+2, pK+/pHR+ presumably), color-coded score legends.
- **Known issues:** this panel is mostly a glossary, not a live dashboard — the actual metric *values* surface inside Pitcher/Hitter Profile instead, so “Metrics” as a nav destination is thinner than the other sections.

Parlay Builder

- **Panel:** `panel-parlay`, lines 832–855. **Nav item hidden** (`id="nav-parlay" style="display:none"`, line 431, comment `<!-- temporarily hidden -->`) — currently unreachable from the sidebar even though the panel and its logic are fully implemented.
- **Load path:** `loadParlayBuilder()` (line 6202), triggered by `showSection('parlay')` (only reachable today via direct JS console call or restoring the nav link). Risk slider (`#parlay-risk-slider`) bound via `parlayBindSlider()` (line 6225).
- **Endpoints called:** `GET /mlb/parlay/builder?date=` (`todayStr()`).
- **Rendering:** `parlayRenderTable(data, level)` (line 6156) using `poissonProb(lam, n)` (line 6128) and `computeParlayLine(projKs, stdK, confidence, level, allLines)` (line 6138) client-side to pick the highest Kalshi K-line clearing the confidence/edge thresholds for the selected risk level (0=conservative ... 2=aggressive).
- **Displayed data:** per-starter recommended Kalshi strikeout line at the selected risk tier, confidence color-coding (green ≥ 70 , orange ≥ 50 , red below).
- **Known issues:** **feature is built but not linked in the nav** — flag for Section 7 as either intentional (in testing) or an oversight to re-enable.

Other (not in the 7 requested sections)

Matchup Engine (`panel-matchup-engine`, lines 594–685) — **nav item hidden** (line 407, `<!-- temporarily hidden -->`). Pick a pitcher (`searchMEPitcher / selectMEPitcher`) and batter (`searchMEBatter / selectMEBatter`), set count/outs/runners (`setME0outs`, `initCountGrid / setMECount`, `toggleRunner`), then `runSimulation()` (line 4067) calls `POST /mlb/simulate/at-bat` and

`renderSimResults()` (line 4100) shows the Monte Carlo at-bat outcome distribution. Also embeds a live scouting-report style matchup flow (`loadMatchupGames / onMatchupGameSelect / loadMatchupPitcherFromGame / runMatchupAnalysis / renderMatchupResult`, lines 2647–2849) hitting `GET /mlb/matchup/pitcher/{pitcherId}/vs-lineup/{gamePk}`, `GET /mlb/lineup-stats/{gamePk}`, `GET /mlb/lineup-stats/pitchers`, `GET /mlb/matchup/attack-plan/{pitcherId}/{batterId}`, `GET /mlb/batter/{batterId}/recent`.

Model Scores (`panel-model-scores`, lines 737–747) — `loadModelScores()` (line 1226) → `GET /mlb/model-scores?days=30` → `renderModelScores()` (line 1241) with `fmtMAE / fmtBias / metricRow` helpers building a MAE/RMSE/bias table for pitcher (K, ER, IP) and batter (TB, K, HR) projections vs. actuals. This is the frontend surface for `build_model_scores.py`'s output.

Props Tracker (`panel-props`, lines 749–830) — **nav item hidden** (line 428). Full CRUD UI for tracking manually-entered prop bets with live confidence scoring: `propsSearchPlayers / propsSelectPlayer` (search), `propsUpdateConfPreview` → `GET /mlb/props/confidence?... (pre-add confidence check)`, `propsAddProp` → same confidence endpoint, `propsRenderList / propsStatus / propsStatusBadge / propsGameLabel` (render), `propsRefreshLive / propsStartPolling` (live status polling during games), `propsDelete / propsClearAll`. State is persisted client-side only (`propsGetAll / propsSaveAll / propsGenId` — appears to use `localStorage`, not a DB table) — worth confirming in Section 7 whether that's intentional (ephemeral, per-browser tracking) vs. a gap (no server-side persistence/multi-device sync).

Mobile nav (lines 6271–6289): `toggleMobileNav / closeMobileNav / mobileNav(section)` — a small separate script block wrapping `showSection()` for a hamburger-menu mobile layout; not a dashboard section, just responsive-nav plumbing.

3 of 10 nav items are currently hidden (Matchup Engine, Props Tracker, Parlay Builder) despite being fully wired up — this is the single most notable cross-cutting observation for Section 7: either these are intentionally in soft-launch/testing and should stay hidden, or they're finished features that should be re-enabled.

Section 7 — Known Issues & Fix List

Prioritized by blast radius (how many downstream consumers are affected × how visible/consequential the error is), not by file order. Items 1-9 come from the Section 4 formula audit; 10-15 are architecture/product gaps surfaced while documenting Sections 3, 5, and 6.

ISSUE: Innings-pitched stored in baseball notation but treated as decimal
FILE: `pipeline/mlb/transform.py`, line 349
CURRENT: `innings_pitched=float(p["inningsPitched"])` # "6.1" → 6.1, "6.2" → 6.2
CORRECT: "6.1" should become 6.333 (6⅓), "6.2" should become 6.667 (6⅔) — the digit after the decimal is thirds-of-an-inning, not tenths
FIX: Convert at write time in `transform.py`:
 `whole, frac = divmod(round(raw*10), 10); ip_decimal = whole + frac/3.0`
 Keep the original baseball-notation string in a separate display column if the UI wants to show "6.1" rather than "6.33". Also backfill existing rows.
IMPACT: Corrupts every aggregate built on `mlb.boxscore_pitching.innings_pitched`:
 — `/lineup-pitcher-stats ERA` (`api/routers/mlb.py:5230–5232`) — wrong ERA shown on the Game Companion lineup card for any pitcher with fractional innings
 — `mlb.daily_actuals.actual_ip` (`pipeline/mlb/build_actuals.py:39`) — means our own projection-accuracy grading (`build_model_scores.py`) is scored against a distorted "actual IP," which can bias the nightly self-calibration in `build_calibration.py`
 Only `pipeline/mlb/build_daily_projections.py`'s `get_pitcher_pa_per_inn()` (lines 215–216) converts correctly — everything else reads the raw value.

ISSUE: OBP calculation missing walks entirely
FILE: `api/routers/mlb.py`, lines 2781–2783 (`batter_season_stats`), 2850–2852 (`lineup_stats`), 2214–2215 (`batter_recent` helper)
CURRENT: $(H + HBP) / (AB + HBP + SF)$
CORRECT: $(H + BB + HBP) / (AB + BB + HBP + SF)$
FIX: Add `COALESCE(SUM(bb.walks),0)` to both numerator and denominator — copy the formula already used correctly in `hitter_profile` (lines 2963–2967).
IMPACT: Affects `/batter/{id}/season-stats`, `/lineup-stats/{game_pk}`, and the `batter-vs-hand recent-form` helper — all under-report OBP for any player who walks. OPS on these same endpoints is wrong too, since `OPS = this broken OBP + SLG`.

ISSUE: SLG denominator includes non-AB plate appearances
FILE: api/routers/mlb.py, lines 4348-4351 (hitter/{id}/juiced-splits) and the same pattern repeated at lines 5014-5016, 5020-5022, 5037-5041, 5143-5145 (game/{game_pk}/batter-splits/{batter_id})
CURRENT: AVG(CASE event WHEN Single=1 ... HR=4 ELSE=0 END) over ALL qualifying at_bats rows (walks/HBP/SF/SH rows included as 0s in the denominator)
CORRECT: SLG = TB / AB - walks/HBP/SF/SH must be excluded from the denominator
FIX: Add event-type exclusion for BB/HBP/SF/SH to the WHERE clause (or switch to boxscore_batting.total_bases / boxscore_batting.at_bats, as done correctly elsewhere in the same file).
IMPACT: Understates SLG (and therefore OPS) for any batter with a normal walk rate, on the Hitter Profile "Juiced+2 splits" card and the live Game Companion batter-splits panel.

ISSUE: Batter K% computed as K/AB instead of K/PA
FILE: api/routers/mlb.py, line 2791 (batter_season_stats), line 2860 (lineup_stats)
CURRENT: SUM(strikeouts) / SUM(at_bats) * 100
CORRECT: SUM(strikeouts) / PA * 100, where PA = AB + BB + HBP + SF + SH
FIX: Reuse the PA expression already correct in hitter_profile (lines 2975-2980).
IMPACT: Overstates K% for any batter who draws walks (AB < PA, so the same K count divided by a smaller number looks worse than reality) - visible on /batter/{id}/season-stats and the Game Companion lineup card.

ISSUE: OBP missing HBP (and using a different formula than every other endpoint)
FILE: api/routers/mlb.py, lines 5193-5195 (lineup_batter_stats)
CURRENT: (H + BB) / (AB + BB) -- no HBP, no SF, and SUM(bb.walks) unguarded
CORRECT: (H + BB + HBP) / (AB + BB + HBP + SF)
FIX: Same as the other OBP fix - standardize on hitter_profile's formula, and wrap SUM(bb.walks) in COALESCE(..., 0) to avoid NULLing the whole aggregate when any row has a NULL walks value.
IMPACT: Third distinct wrong OBP formula in the codebase (see FORMULAS.md for the full comparison table) - affects the Game Companion lineup card.

ISSUE: HBP silently dropped from one live-game OBP subquery, inconsistent with its sibling subquery in the same endpoint response
FILE: api/routers/mlb.py, lines 5141-5142 (game_state_split subquery inside /game/{game_pk}/batter-splits/{batter_id}) vs. lines 5011-5019 (ops_vs_hand subquery, same endpoint, correctly includes HBP)
CURRENT: event list (1B,2B,3B,HR,BB,IBB) - HBP omitted
CORRECT: event list must include Hit By Pitch
FIX: Add 'Hit By Pitch' to the CASE/IN list in the game_state_split subquery.
IMPACT: A single API response returns two different OBP numbers computed two different ways for the same batter, depending which split the frontend reads.

ISSUE: Internal inconsistency - lineup_k_factor uses K/AB while every sibling projection function uses K/PA (batters faced)
FILE: pipeline/mlb/build_daily_projections.py, get_lineup_k_factor(), lines 328-334
CURRENT: SUM(bb.strikeouts) / NULLIF(SUM(bb.at_bats), 0)
CORRECT (for consistency with get_pitcher_season_k_pct / get_team_k_rate in the same file): should be K / PA (or K / batters-faced), matching how LEAGUE_K_PCT=0.224 is itself defined
FIX: Change the lineup-K SQL to the same PA-denominated pattern used elsewhere in this file, or explicitly document why AB is intentional here (it isn't obviously intentional based on the surrounding code).
IMPACT: Model-internal, not directly user-visible - but because K/AB is always higher than K/PA, this makes lineups look more extreme (more or less K-prone) than they really are relative to the 0.224 league-average benchmark, which biases proj_k_pct / proj_ks_6inn for every game via the lineup_k adjustment factor.

ISSUE: OBP denominator doesn't exclude sacrifice bunts
FILE: api/routers/mlb.py, lines 4352-4354, 4371-4377 (hitter/{id}/juiced-splits, both the batter's own OBP and the league-average OBP CTE)
CURRENT: denominator = COUNT of all at_bats rows except Runner Out/CS/WP/PB (sac bunts remain in the denominator)
CORRECT: sacrifice bunts (SH) should be excluded from the OBP denominator (unlike sac flies, which ARE included)
FIX: Add sac-bunt event types to the WHERE-clause exclusion list for this query only (don't touch the PA-for-K%/BB% denominators elsewhere, which correctly include SH).
IMPACT: Minor - sac bunts are rare - but technically incorrect, and this is the league-average comparator too, so it very slightly understates both sides.

ISSUE: Game Companion calls the MLB Stats API directly from the browser, bypassing our own API layer
FILE: frontend/mlb.html, renderGCLineup(), line ~4736
CURRENT: fetch('https://statsapi.mlb.com/api/v1/game/{gamePk}/boxscore') directly from client-side JS
CORRECT: N/A – architectural choice, not a formula bug
FIX: Route through a new/existing /mlb/... endpoint on our own API so it benefits from the same caching, rate-limit protection, and consistent data shape as every other Game Companion data source.
IMPACT: CORS-dependent (breaks if MLB ever tightens CORS policy), no caching (extra load on MLB's API + slower for the user), and it's the one place the dashboard's data doesn't flow through our own pipeline/DB layer at all.

ISSUE: Props Tracker has no server-side persistence
FILE: frontend/mlb.html, propsGetAll/propsSaveAll (~line 749-830 region)
CURRENT: Tracked props appear to be stored in browser localStorage only (no DB table referenced by propsGetAll/propsSaveAll)
CORRECT: N/A – needs a product decision, not obviously "wrong"
FIX: If cross-device/persistent tracking is wanted, add an mlb.tracked_props table and CRUD endpoints; if per-browser scratch tracking is intentional, no fix needed – just confirm with the user and note it, since it isn't documented anywhere.
IMPACT: Props tracked on one browser/device don't show up on another; clearing browser storage silently loses all tracked props.

ISSUE: 3 of 10 dashboard nav items are fully built but hidden from navigation
FILE: frontend/mlb.html – nav items at lines 407 (Matchup Engine), 428 (Props Tracker), 431 (Parlay Builder), all with a `<!-- temporarily hidden -->` comment
CURRENT: style="display:none" on the nav link; panel + load functions + endpoints all fully wired and functional if reached directly
CORRECT: N/A – needs a product decision
FIX: Either re-enable the nav links (`display:none` → removed) once satisfied with each feature, or remove the dead code if they're permanently shelved.
IMPACT: No functional bug, but worth a deliberate decision rather than leaving three working features invisible indefinitely.

ISSUE: fetch_weather.py's --date flag is accepted but not applied
FILE: pipeline/mlb/fetch_weather.py, main()/fetch_today_weather()
CURRENT: main() parses --date but fetch_today_weather() always uses date.today() internally regardless of what was passed, unless --game-pk/--team are also given
CORRECT: fetch_today_weather() should accept and use a date parameter
FIX: Thread the parsed --date value into fetch_today_weather(target_date=...) and use it instead of date.today() when present.
IMPACT: Low – the scheduled job always runs same-day anyway (run_fetch_weather() in daily_pipeline.py passes today's date), so this only bites someone manually backfilling weather for a past date and assuming --date alone is sufficient.

ISSUE: LEAGUE_K_PCT hardcoded independently in two places, not derived from data
FILE: pipeline/mlb/build_daily_projections.py, line 257 (inside get_team_k_rate fallback) and line 316 (inside get_lineup_k_factor)
CURRENT: LEAGUE_K_PCT = 0.224 defined twice as a local constant
CORRECT: Should be a single module-level constant, or better, computed from mlb.boxscore_batting each run so it tracks the actual current league K rate
FIX: Hoist to one shared constant at minimum; ideally replace with a query.
IMPACT: Low today (0.224 is a reasonable real-world MLB league K% and both copies currently agree), but a latent risk – if one gets updated and the other doesn't, the two functions silently drift out of sync.

ISSUE: pHR+ "hard-hit" gate uses pitch velocity, not exit velocity
FILE: pipeline/mlb/build_pk_phr.py, line 96
CURRENT: p.start_speed >= 92 -- start_speed is PITCH velocity (how hard it was thrown)
CORRECT: should gate on batted-ball exit velocity, i.e. launch_speed >= 95 – the threshold used correctly everywhere else in the codebase (build_goose2.py, build_stuff_score.py, bapv_config.py, build_pitcher_mix.py all use launch_speed >= 95)
FIX: Change build_pk_phr.py's hard-hit gate to launch_speed >= 95 to match the rest of the model family.
IMPACT: pHR+ (home-run-rate quality score) is silently computing its "hard-hit against" component from how hard the PITCHER threw the pitch, not how hard the BATTER hit it – the opposite of what "hard-hit against" is supposed to measure. This is a real, likely consequential bug in a model that's directly blended into Goose+3 (the headline pitcher grade).

ISSUE: Team defense "SLG allowed" divides by PA, not AB
FILE: pipeline/mlb/build_team_defense.py, line 70 (pa_count defined line 45, exclusion list lines 56-57 only removes baserunning events, not BB/HBP)
CURRENT: $slg_allowed = total_bases / \max(1, pa_count)$ -- pa_count includes BB/HBP
CORRECT: $SLG = TB / AB$ (walks/HBP should not be in the denominator)
FIX: Exclude BB/HBP/SF/SH events from the denominator, or compute from boxscore_pitching/boxscore_batting's own at_bats column instead.
IMPACT: Understates every team's "SLG allowed" figure, which feeds directly into simulate.py's opposing-defense adjustment for every batted-ball outcome simulation – a systemic (if probably modest) downward bias on defense-adjusted power projections league-wide.

ISSUE: bapv_config.py is dead/orphaned – the live BAPV+ script doesn't import it
FILE: pipeline/mlb/bapv_config.py (whole file); pipeline/mlb/compute_bapv.py lines 42-58
CURRENT: compute_bapv.py re-declares BASE_WHIFF_VALUE/BASE_CS_VALUE/etc. inline rather than importing them from bapv_config.py; grep confirms no other file imports bapv_config anywhere in the repo
CORRECT: N/A – not a formula bug, a maintenance hazard
FIX: Either delete bapv_config.py (if it's meant purely as documentation, rename it clearly, e.g. BAPV_FORMULA_NOTES.md) or make compute_bapv.py actually import its constants from there so there's one source of truth.
IMPACT: The two copies currently agree, but nothing enforces that – a future edit to one and not the other would silently desync the documented formula from the live one, exactly the kind of drift this CODEBASE.md audit is meant to catch.

ISSUE: mlb.pitcher_count_mix (the simulation engine's primary pitch-selection input) is not on any schedule
FILE: pipeline/mlb/build_count_mix.py – no reference in pipeline/daily_pipeline.py
CURRENT: Not called by any cron job or --now option; must be run manually
CORRECT: N/A – operational gap
FIX: Add a scheduled job (daily or weekly is probably sufficient – count-mix shouldn't change drastically day to day) to daily_pipeline.py, similar to how build_batter_tendencies.py runs weekly Monday 4am PT.
IMPACT: If nobody has re-run this manually recently, every Monte Carlo simulation (build_daily_projections.py, the live at-bat simulator, game simulations) is selecting pitches from a stale count-mix snapshot rather than reflecting each pitcher's current-season count tendencies.

ISSUE: Juiced+ (v1) OBP omits walks and HBP from the numerator
FILE: pipeline/mlb/build_goose.py (compute_juiced_scores / the batter OBP calc feeding Juiced+ v1's OPS-vs-bucket comparison)
CURRENT: $obp = hits / pa$ -- PA denominator is correct, but numerator is hits-only
CORRECT: $obp = (hits + BB + HBP) / PA$
FIX: Add walks and HBP to the numerator, matching the corrected formula used elsewhere (see hitter_profile's OBP).
IMPACT: Depresses Juiced+ (v1) for high-walk batters specifically, since OBP feeds directly into the OPS-vs-league-bucket ratio that Juiced+ is built from. Juiced+2 (build_goose2.py) does NOT have this bug – only the legacy v1 model does.

ISSUE: No "actual" BB% stat, K/9, or BB/9 exposed anywhere for batters/pitchers
FILE: api/routers/mlb.py – grepped, not found as columns anywhere
CURRENT: hitter_profile shows actual K% but no actual BB%; no endpoint computes K/9 or BB/9 locally, and they aren't among the Fangraphs columns currently selected (era, era_plus, fip, whip, k_pct, bb_pct, swstr_pct, war, xera, siera)
CORRECT: N/A – feature gap, not a wrong formula
FIX: Add BB% alongside K% in hitter_profile (same PA denominator, trivial addition since boxscore_batting.walks is already summed there); add k_9/bb_9 to the Fangraphs pitching SELECT if those columns exist in mlb.fangraphs_pitching (verify – fetch_fangraphs.py may already be storing them even if unused here).
IMPACT: Low – cosmetic completeness gap on the Pitcher/Hitter Profile pages.

Section 8 — How To Make Common Edits

1. Change a model weight (e.g. adjust pK+ whiff weighting)

1. Find the weights dict — most models keep them as a module-level constant near the top of the file: pK_WEIGHTS / pHR_WEIGHTS in pipeline/mlb/build_pk_phr.py, G00SE2_WEIGHTS / JUICED2_WEIGHTS in build_goose2.py, COMPONENT_WEIGHTS / OUTCOME_WEIGHTS in build_stuff_score.py, G00SE_WEIGHTS -equivalent inline percentages in build_goose.py (lines ~406-419), get_outcomes_weight() in build_goose3.py.

2. Edit the constant. Most of these dicts are documented with the Pearson-r correlation each component was empirically weighted from (e.g. `build_pk_phr.py`'s comments) — if you change a weight, update that comment too so the next person knows it's no longer derived from the stated correlation.
3. Rebuild that model standalone first to sanity check: e.g. `PYTHONPATH=/app:/pipeline python3 pipeline/mlb/build_pk_phr.py --score-season 2026 --baseline-season 2025 --validate-only` (most `build_*` scripts have a `--validate-only / validate()` path that prints correlation-to-actuals without writing to the DB — use it before committing to a full rebuild).
4. If satisfied, rebuild for real (drop `--validate-only`), then rebuild anything **downstream** that consumes this table: `pk_scores / phr_scores feed build_goose3.py`, which feeds `mlb.goose3_overall` (the headline pitcher grade) and `simulate.py`'s `load_goose3_adjustments`. Changing `build_pk_phr.py` without re-running `build_goose3.py` afterward leaves Goose+3 built on stale pK+/pHR+ inputs until the next 6am scheduled run.
5. `docker cp` the changed file(s) into `sports_api` and `sports_scheduler` (see Section 8.5) if testing against the running containers rather than a fresh `docker compose up --build`.

2. Add a new column to a projection

Follow the data flow from Section 1.3 in reverse:

1. **DB:** Every projection table (`mlb.daily_projections`, `mlb.daily_batter_projections`, `mlb.game_projections`, etc.) is a raw-SQL table with `CREATE TABLE IF NOT EXISTS` in its owning pipeline script — but `CREATE TABLE IF NOT EXISTS` does **not** add a column to a table that already exists. For a brand-new environment this is fine (the new column shows up automatically); for the existing dev/prod DB you need an explicit `ALTER TABLE mlb.daily_projections ADD COLUMN IF NOT EXISTS new_col NUMERIC`; run once (there's no Alembic migration workflow actively used here despite `migrations/env.py` existing — `Base.metadata.create_all()` in `api/main.py` only handles ORM-defined tables in `api/models/mlb.py`, and raw-SQL tables aren't ORM-defined, so plan to run the `ALTER TABLE` by hand via `docker exec ... psql` or pgAdmin).
2. **Pipeline:** Compute the new value in whichever `build_*.py` script owns that table, add it to the `INSERT ... ON CONFLICT ... DO UPDATE SET` column list (both the insert column list and the `SET` clause — easy to add to one and forget the other, which silently means the column only ever gets its initial value and never updates on re-runs).
3. **API:** Add the column to the relevant `SELECT` in `api/routers/mlb.py` and to the response dict/model if the endpoint builds an explicit response shape rather than returning `dict(row)` directly.
4. **Frontend:** Add the field to wherever `frontend/mlb.html` renders that endpoint's data — find the `render*()` function for that dashboard section (Section 6 has the map of load-function → render-function → line numbers per section).
5. Test locally end-to-end (run the pipeline script manually, hit the endpoint with `curl`, reload the dashboard) before deploying.

3. Fix a stat formula

Using this session's own OBP audit as the template:

1. **Find every place the formula is computed** — don't assume there's only one. Grep `api/routers/mlb.py` for the stat name (e.g. `obp`, `slg`) — Section 4/FORMULAS.md shows this codebase frequently has 3-5 independent copies of the “same” formula that have drifted apart.
2. **Fix the SQL** in each location, or better, extract it into one shared SQL snippet/CTE if several endpoints need it (there's currently no shared query-builder layer — every endpoint inlines its own SQL — so consolidating is a larger refactor than a one-line fix; a pragmatic middle ground is a Python helper function that returns the SQL fragment string).
3. **Rebuild `mlb.daily_actuals`** if the formula affects anything that flows into actuals/scoring: `python pipeline/mlb/build_actuals.py --date <date>` (or `--backfill` if historical data needs correcting — check the script's flags first, this can be slow over a full season).
4. **Verify** by re-running `build_model_scores.py` and eyeballing whether bias/MAE numbers moved in the expected direction, or by directly querying the fixed vs. old formula side by side for a few known players.
5. If the formula feeds `build_calibration.py`'s nightly self-learning loop (K% and HR% specifically), a formula fix will shift the calibration factor over the next few days as the rolling window picks up corrected data — don't be alarmed by calibration drift right after a formula fix, it's expected.

4. Add a new pipeline step

Use `build_integrity_check.py` (built this session) as the reference example:

1. Create `pipeline/mlb/build_whatever.py` following the existing convention: `argparse` CLI with sensible defaults, a `main()` guarded by `if __name__ == "__main__":`, functions that take `db` as an explicit parameter (never a global session) so the script works both standalone and when imported/called in-process by `daily_pipeline.py`.
2. In `pipeline/daily_pipeline.py`: add a one-line wrapper function (`def run_whatever(): _run_script("Whatever", "/pipeline/mlb/build_whatever.py", [...args])` for the common case, or call the module's functions directly in-process if you need finer control, following the `run_mlb_daily() / run_pa_validation()` pattern).
3. Add the new choice string to the `--now argparse choices` list and to the `if args.now in (...)` dispatch block in `main()`.

4. Add a `scheduler.add_job(run_whatever, CronTrigger(...), id="...", name="...", misfire_grace_time=600)` call in the scheduled-mode section of `main()`, and add a line to the startup `log.info(...)` summary block so the schedule stays self-documenting in the container logs.
5. Deploy per Section 8.5 below — remember the scheduler container needs a restart (see note there), not just a file copy.

5. Deploy a change to production

Based on this session's actual working pattern with the running Docker containers:

1. **Local test first.** Bind mounts (`./api:/app`, `./pipeline:/pipeline`) mean `sports_api`'s `uvicorn --reload` picks up API/router changes automatically in local dev — but pipeline scripts run as one-off subprocess/module calls, so just re-run the script manually to test (`docker exec sports_api bash -c "PYTHONPATH=/app:/pipeline python3 /pipeline/mlb/whatever.py ..."`).
2. **git commit** (only when explicitly asked — this repo's working convention, per the instructions governing this session, is to never commit without being asked).
3. **Push** to the remote (again, only on explicit request).
4. **On the EC2 host:** `git pull`, then either `docker compose -f docker-compose.aws.yml up -d --build` (full rebuild — safest, picks up dependency/Dockerfile changes) or, for a quick pipeline-only fix, `docker cp` the changed file(s) directly into the already-running containers the way this session did:

```
docker cp pipeline/mlb/whatever.py sports_api:/pipeline/mlb/whatever.py
docker cp pipeline/mlb/whatever.py sports_scheduler:/pipeline/mlb/whatever.py
```

5. **Restart the scheduler if you touched anything it schedules or imports.** `sports_api` auto-reloads on file change (`--reload` flag), but `sports_scheduler` runs `daily_pipeline.py` as a long-lived `BlockingScheduler` process that only reads the file once at startup — a `docker cp` into that container updates the file on disk but **won't** take effect until the process restarts: `docker restart sports_scheduler`. (`sports_live_poller` is the same story — it's a long-running loop, not a reloading server.)
6. Verify with a manual `--now <job>` run before trusting the next scheduled firing, the same way `build_integrity_check.py` was smoke-tested against `sports_api` in this session before being wired into the schedule.